International Journal of
INTELLIGENT SYSTEMS AND APPLICATIONS IN ENGINEERING
IJISAE
ISSN:2147-6799 www.ijisae.org Original Research Paper

A Critical Study of Pragmatic Ambiguity Detection in Natural Language Requirements

Ms. Reena S. Satpute, Dr. Avinash Agrawal

Submitted: 02/11/2022 **Accepted**: 01/02/2023

Abstract: An approach for pragmatic ambiguity detection in natural language requirements is presented in this paper. Pragmatic ambiguities are determined by the requirements' context, which includes the reader's background knowledge. Readers with different backgrounds may interpret requirements differently. To determine whether a requirement is ambiguous or not, various pragmatic interpretations are compared. In this paper, we will discuss the significance of pragmatic ambiguity detection in NLRs, applications of NLP, ambiguities in NLP, and pragmatic ambiguities, as well as review various techniques used for identifying and resolving ambiguities in natural language requirements. Our objective is to motivate further research in this field by providing a thorough understanding of the difficulties and opportunities related to pragmatic ambiguity detection in NLRs. The tool might be enhanced in the future to support more file types, like PDF. There is ongoing research in the area of pragmatic ambiguity detection, and new approaches and procedures are constantly being developed. It is likely that improvements in pragmatic ambiguity detection and resolution will come as a result of developments in artificial intelligence, machine learning, and natural language processing in the future. Additionally, the growing accessibility of expansive, varied datasets will make it possible to train more reliable and accurate models. Pragmatic ambiguity detection is likely to become a more crucial tool as the field develops in fields like automated language translation, dialogue systems, and natural language understanding.

Keywords*: Natural Language Processing, Ambiguity, Pragmatic Ambiguity.*

1. Introduction

Language is a channel of communication with the help of which we are able talk, write and read. For instance, we use natural language - more specifically, words - to think, decide, plan, and do other things. The key question, though, is whether we can converse similarly with computers in the age of AI [1][2]. In other words, is it possible for people to speak naturally to computers? Because computers require organized data but human speech is unstructured and frequently confusing in nature, it is difficult for us to create NLP applications. There are several advantages to using natural language in software specifications, including improved stakeholder comprehension and communication [3]. [4]. However, ambiguities introduced by natural language can also result in misinterpretations and mistakes during the software development process. In order to increase the overall quality and correctness of the requirements, pragmatic ambiguity detection in NLRs aims to identify and clear out ambiguities in NLRs [5]. Multiple readings of a sentence or phrase depending on the context in which it is used are referred to as pragmatic ambiguity. These misunderstandings can be brought about by a number of things, including a lack of knowledge, inconsistent terminology use, and implicit presumptions. Pragmatic ambiguity detection in NLRs is a difficult task since it calls for knowledge of the domain being produced for the system as well as the context in which the requirements are written [6]. It is impossible to emphasize the significance of pragmatic ambiguity detection in NLRs. The software development process might be delayed and expensive errors can occur as a result of ambiguous specifications. They might also lead to the formation of a system that fails to satisfy the requirements of its users. Therefore, identifying and resolving ambiguities in NLRs is essential to guaranteeing the effectiveness and efficiency of the software development process [7]. The steps in pragmatic ambiguity detection is given in fig 1 can be summarized as follows:

- Identify potential ambiguities: The first step is to identify potential ambiguities in the natural language requirements. This can be done by using natural language processing (NLP) tools to automatically identify and classify ambiguities, or by having domain experts review and validate the requirements.
- Analyze the context: Once potential ambiguities are identified, the next step is to analyze the context in

Assistant Professor Datta Meghe Institute of Higher Education & Research, Sawangi (M)
Email – reenasatpute2017@gmail.com
Associate Professor & Dean Shri Ramdeobaba College of Engineering and Management, Nagpur
Email – agrawalaj@rknec.edu

which they appear. This includes understanding the domain knowledge of the system being developed and the intended meaning of the requirements.

- Resolve the ambiguities: After analyzing the context, the ambiguities can be resolved by clarifying the requirements and ensuring that they are unambiguous. This may involve rephrasing, adding additional information, or removing implicit assumptions.
- Validate the resolution: The final step is to validate that the resolved ambiguities are accurate and that they meet the needs of the stakeholders. This may involve testing the requirements with users or conducting a formal review by domain experts.
- Document the resolution: After validating, it's important to document the resolution of the ambiguities, the reason why it was resolved that way and who was involved in the process.

Fig. 1. Pragmatic Ambiguities Detection Steps

The significance of pragmatic ambiguity detection in NLRs, applications of NLP, ambiguities in NLP, pragmatic ambiguities, and a review of several strategies used for finding and resolving ambiguities in natural language requirements are all covered in this paper. Our objective is to further this field of research by providing a thorough knowledge of the challenges and opportunities related with pragmatic ambiguity detection in NLRs.

2. Overview of NLP

Natural language processing (NLP), which can describe and interpret language and communication digitally, has attracted a lot of attention lately. Machine translation, spam email recognition, information retrieval and summarizing, in addition to healthcare and inquiry answering, are just a few of the many functions it presently serves [8]. The study of how we may use machines to understand and alter natural language for further investigation is known as natural language processing, or NLP. Numerous computer methods for the automatic analysis and interpretation of human language are included in NLP. In NLP, the basic words of language play a significant role. Examples of some basic words include the following: bad, partly, old, magnificent, extremely, etc. A composite is a group of these essential words. [9] [10]. Examples of several composite words are young guy, not really startled, and very nice movie. Simple terminology for atomic and composite terms are words and phrases, respectively. Language's fundamental building component is the word. Words make up human languages, whether they are spoken or written. One of the first stages toward understanding the language is the NLP word-level techniques. The effectiveness of NLP systems, such as automated question answering, information retrieval, and machine translation, relies on the text's intended meaning. Ambiguity, or ambiguous or open meaning depending on use context, is the biggest difficulty. NLP are used in number of applications as presented in fig 2 as presented in [11]-[17].

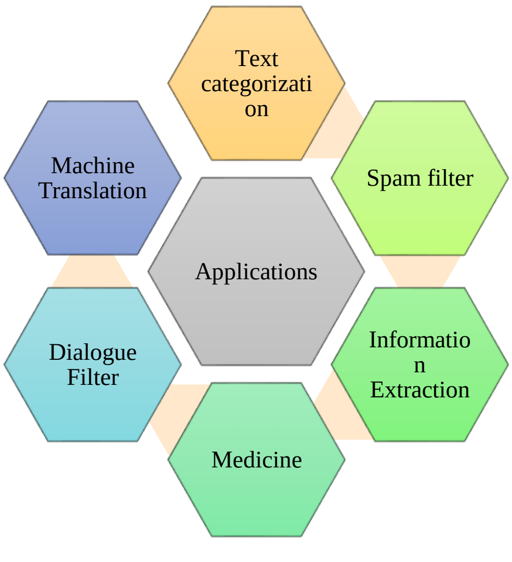

Fig. 2. Application of NLP

3.

4. Ambiguities in NLP

Language is a method of communication with the help of which we can speak, read and write. For example, we think, we use natural language—more specifically, words—to make choices, plans, and other things. The key issue, however, is whether humans can converse similarly with machines in the age of AI. In other words, is it possible for people to speak naturally to computers? Because computers need organized data but human speech is unstructured and often confusing in nature, it is difficult for humans to create NLP applications [18]. This makes it possible to define Natural Language Processing (NLP) as the area of computer science, particularly Artificial Intelligence (AI) that deals with teaching computers how to comprehend and use human language. According to technical definitions, the primary function of NLP would be to train computers to process and analyze vast amounts of natural language data [19]. One of the main problems in NLP is ambiguity. We take into account a number of various factors when attempting to interpret the meaning of a term, including the context in which it is used, our own worldview, and the way a word is typically used in society. Words may imply various things in different contexts and can also alter their meaning over time. Homographs, which are two words with the same spelling but distinct etymologies, and polysemy, which is one word with many meanings, are examples of this phenomena. Ambiguity is a term often used in natural language processing and is defined as the capacity to be interpreted in several contexts. Uncertainty is the capacity to be comprehended in more than one manner, to put it simply. Language in general is highly pragmatic. Fig 3 represents the ambiguities in NLP.

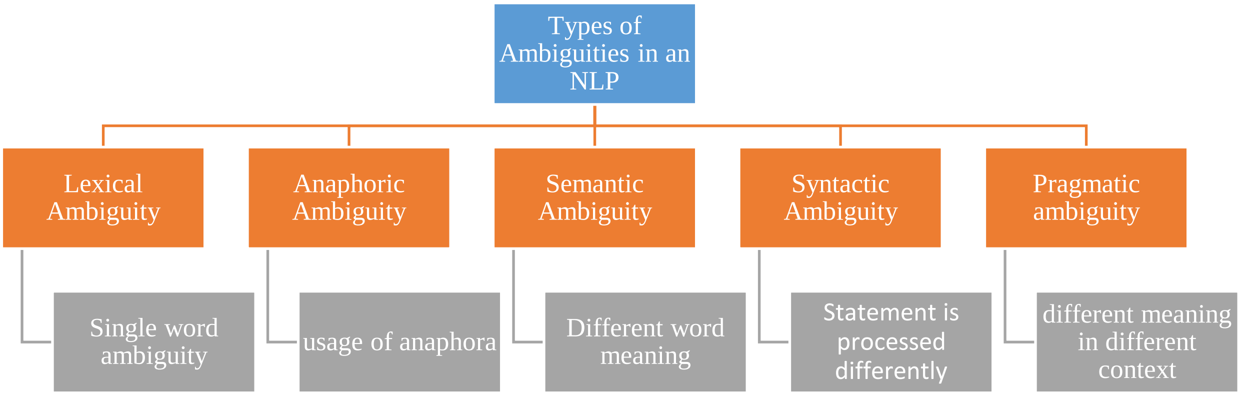

Fig. 3. Types of Ambiguities in an NLP

Ambiguity in Natural Language Processing can be removed using:

- Word Sense Disambiguation
- Part of Speech Tagger
- HMM (Hidden Markov Model) Tagger
- Hybrid combination of taggers with machine learning techniques.

In order to clarify the intended meanings of words (word senses) in a certain context, word sense disambiguation (WSD) is used. WSD classifies an occurrence of a word in context into one or more of its sense classes for a particular word and its potential meanings [20]. The characteristics of the context, such as words nearby, serve as the basis for categorization. Concepts of Part of Speech (POS) tagging [21] and the use of probabilistic grammars in parsing are examples of statistical strategies for ambiguity resolution. Large corpora of word use data, such as the Brown Corpus, WordNet, and SentiWordNet, have been used to develop statistical techniques for reducing sentence ambiguity. The POS, grammatical structure, and frequency of use for words and sentences obtained from a large sample of written language are pre-tagged on the words and sentences in these various corpora. The process of choosing the most probable part of speech from among the potential meanings for each word in a phrase is known as POS tagging. The notion of grammar rules and rules connected to part-of-speech tags are explored by probabilistic grammar [22]. Each rule in a probabilistic language is assigned a probability depending on how often it is used in the corpus. When a sentence is syntactically confusing, this method helps choose the optimal option (that is, there are more than one different parse trees for the text).

Methods for using a knowledge dictionary are proposed by Kato et al. [23] recommended by analyzing requirement specifications and supporting documentation for transitive verbs and their objects. TaskLint is a system that was developed by Manam et al. [24], and its purpose is to automatically discover errors in task instructions. TaskLint is able to recognize words and sentences that may indicate uncertainty among workers by making use of a wide variety of previously developed NLP technologies. This is similar to the way that code analysis tools ("linters") detect probable traits in code that might signal the presence of flaws. In our evaluation of TaskLint, we used task instructions that were generated by inexperienced users. This confirmed that static tools have the ability to increase task clarity and the correctness of outputs, but it also brought to light a number of difficulties. In order to deal with anaphoric ambiguity in requirements, Ezzini et al.[25] created a comprehensive automated solution that takes into account both ambiguity detection and anaphora interpretation. When it comes to the problem of ambiguity identification, supervised machine learning performs better than both a large-scale language model known as SpanBERT and a solution that is put together using off-the-shelf NLP coreference re-solvers. Roop et al. [26] presented a supervised neural network model that makes use of numerous approaches in order to achieve the highest possible accuracy in sense detection. Kaddoura et al. [27] gave a comprehensive assessment of research works that seeks to resolve Arabic word meaning disambiguation using the AWSD datasets that are already in existence. Saxena et al. [28] compiled a list of SMT systems for five different translation tasks, using English and Indian languages as the test dataset and evaluating the systems using the BLEU and METEOR criteria. The investigations were conducted on the En-Kn, En-Ta, En-Te tasks, En-Hi, and they demonstrated an increase in BLEU points by 2.68, 0.78, 2.32, 2.3, respectively, and an increase in METEOR points by 1.34, 0.72, 0.693,1.07, respectively, over the baseline model. Yadav et al. [29] provided a comprehensive analysis of a variety of disambiguation techniques in their presentation. Certain tools are still in the process of being developed, and it is possible that in the future they will be able to eliminate ambiguity. Formal language is required to be used when writing a requirement document so that there is no room for ambiguity. However, consumers do not appreciate formal language because it is not as clear and is more difficult to comprehend. Handling ambiguity in the requirements is addressed by Ezzini et al. [30] proposed automated technique, which makes use of natural language processing. Our strategy involves using Wikipedia to automatically generate a corpus of content that is specific to a given domain. When domain knowledge is integrated, the accuracy of ambiguity detection and interpretation sees a large improvement, both of which are favorable. Our work is focused on coordination ambiguity, sometimes known as CA, as well as prepositional-phrase attachment ambiguity (PAA). Osama et al. [31] presented an effective and versatile automatic syntactic ambiguity detection solution for NL needs. The proposed method involves sifting through all of the many scoring interpretations that can be produced for a particular sentence by using the Core NLP library. When we tested our method on a collection of datasets including 126 requirements, we were able to achieve an average precision of 65% and recall of 99%. Mishra et al. [32] measured the ambiguity potential of the words that are most commonly used in computer science when those words were employed in various subdomains of engineering, such as civil, petroleum, biomedical, and environmental engineering, aerospace etc. This research makes use of a natural language processing method called word embedding in order to identify ambiguous terms. In Ferrari et al. [33] evaluated the degree to which natural language processing (NLP) may be practically applied to the task of finding errors in the requirements papers of a railway signaling manufacturer. In order to search for frequently ambiguous terms, the SREE tool was utilized, and it was applied to the criteria. Based on the results of the trials, it appears that SREE and our patterns may play roles in the detection of requirements deficiencies that are complementary to one another. Below table 1 presents the summary of the research contributions presented above:

Table 1. Research Contribution for Ambiguity Detection in NLP

Ref	**Year**	**Methodology**	**Technologies**	**Ambiguity**	**Results**
[23]	2022	Knowledge Dictionary	-	Pragmatic	Recall 99%
[24]	2018	TaskLint	-	-	-
[25]	2022	BERT	-	Anaphoric	Precision 60%
[26]	2022	Artificial Neural Network	-	Grammar	-
[27]	2022	Word Segmentation Based Dictionary	-	Grammar	-
[28]	2022	Unsupervised Learning	-	Grammar	-

[29]	2021	Controlled Language	POS Tagging	Lexical	Recall 80.12% and Precision 85.76%
		Knowledge Based & Ontology	Stanford	Lexical	Recall 92.85% Precision 92.85%
		Knowledge based & Ontology	WordNet	Lexical	Precision 83.4%
[30]	2021	Domain-Specific Corpora	-	Domain	Precision 80% recall 89%
[31]	2020	Filtering pipelines	Stanford	Syntactic	Recall 99% precision 65%
[32]	2019		Word embedding	Domain	-
[33]	2018	NLP	Word embedding	-	Precision 83% and recall 85%

5. Pragmatic Ambiguity Detection

Whenever a statement is pragmatic ambiguous and the context lacks the details necessary to make it clear, pragmatic ambiguity results. An example of pragmatic ambiguity is presented in table 2. The lack of data necessitates inference. When a statement is read or written sarcastically and the context lacks the details necessary to make it clear, pragmatic ambiguity results [34]. A requirement has a pragmatic ambiguity, roughly speaking, if various readers understand it differently, depending on the context of the demand. The data of a requirements encompasses both the requirements of the particular file that have an impact on how well the demand is understood as well as the reader's prior knowledge, which gives the ideas described in the requirement meaning.

A key component of natural language processing is pragmatic ambiguity identification, which aids in deciphering the intended meaning of a sentence or phrase. Language ambiguities may come from a variety of places, including homonyms, synonyms, and idiomatic expressions [35]. It is challenging for robots to comprehend and interpret human language because of these ambiguities, which may cause misunderstanding and confusion. We may increase the precision and dependability of natural language processing systems by spotting and resolving pragmatic ambiguities. This is necessary for a number of applications, including conversation systems, automatic language translation, and natural language comprehension. Additionally, pragmatic ambiguity detection may assist natural language based systems become more effective and efficient by enhancing their overall usability and user experience [36].

Table 2. Meaning of Pragmatic Ambiguities

Sentence	**Direct Meaning**	**Other meaning**
Do you know what time it is?	requesting the time	angering someone for not doing anything in a timely manner
Will you crack open the door? I'm getting hot	For breaking the door	For door opening
He chicken is ready to eat the breakfast	Small hen is ready to have its food	The prepared chicken is ready to eat

It's crucial to remember that pragmatic ambiguity detection is a continuous process that must be carried out at every stage of software development to guarantee that the specifications are accurate and clear. For pragmatic ambiguity detection in NLRs, various methods are employed. The most cutting-edge method uses artificial intelligence in tools for natural language processing (NLP) [37]. These instruments are capable of automatically locating and categorizing ambiguities in NLRs. Utilizing subject-matter experts to examine and confirm the requirements is another tactic. To find and resolve ambiguities in the requirements, this method depends on the domain experts' knowledge. There are several advantages to detecting and resolving pragmatic ambiguities in natural language requirements (as presented in fig 4), some of them are:

1) Improved understanding: By identifying and resolving ambiguities, stakeholders have a clearer understanding of the requirements and what the system being developed is supposed to do.
2) Reduced errors: Ambiguities in requirements can lead to misunderstandings and errors in the software development process. By detecting and resolving ambiguities, the likelihood of these errors is greatly reduced.
3) Increased efficiency: Detecting and resolving ambiguities in the requirements allows for a more efficient software development process. This is because ambiguities can cause delays and rework if they are not identified and resolved early on.
4) Better communication: Pragmatic ambiguities detection helps in better communication between stakeholders, and ensures that the requirements are clearly understood by all parties involved in the development process.
5) Improved user satisfaction: By ensuring that the requirements are clear and accurate, the likelihood of developing a system that meets the needs of its users is increased, which leads to improved user satisfaction.
6) Reduced costs: By detecting and resolving ambiguities early on in the development process, the costs associated with rework and delays can be reduced, which can lead to significant cost savings for the project.
7) Better traceability: By documenting the resolution of the ambiguities, it's easier to track any changes in the requirements and to understand the reason why certain decisions were made

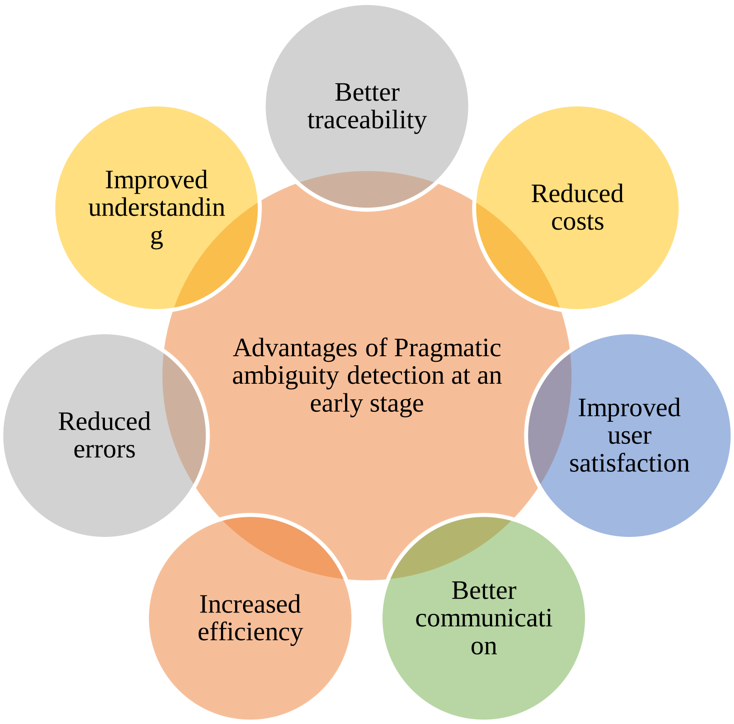

Fig. 4. Advantages of Pragmatic Ambiguity Detection

In [39] because natural language is often confusing, so are software needs. Errors in software development are caused by the requirements' ambiguity. As a consequence, several methods and procedures for identifying ambiguity in software requirements have been developed. In order to identify linguistic ambiguity in software requirements, this research employed three supervised machine learning techniques. With an accuracy of 86.66%, RF was the most accurate, followed by SVM and KNN with 80 and 70 percent respectively. In [40], author provided a method for locating these ambiguities depends on the concept of bidirectional encoder called BERT representations from and clustering. This method gives a list of terms that are most comparable to each situation in which a phrase has been used in the document, as well as some sample sentences from the corpus that illustrate the term's context-specific meaning. Uses a corpus that is specialized to computer science (CS) and a multi-domain corpus that contains material from eight distinct application areas. Our test results demonstrate how successful this technique is in locating and identifying intra-domain ambiguities. In [41] author provided an effective and adaptable automated method for NL requirements syntactic ambiguity identification. The suggested method is based on filtering the Stanford Core NLP library's potential scored interpretations of a given text. Additionally, it gives the user feedback along with potential accurate interpretations in order to clear up any misunderstanding. In order to offer the most probable right interpretations of a need. Tested proposed methodology using a collection of datasets with 126 criteria, and on average were able to obtain 65% accuracy and 99% recall. For locating and recognizing ambiguous terms in a requirement, word embedding was utilized by [42]. The research then use connected data to resolve ambiguity in words. In order to assess the performance of the suggested tool, open-source software specification documents were used to compare human detection and correction capabilities. The results show that, in comparison to the suggested tool, humans struggle to identify and correct ambiguity in software requirement specifications (SRS) that span multiple domains. The outcome demonstrates that, in comparison to humans, the created tool was accurate in identifying and resolving pragmatic ambiguities. In [43], author explored the possibility of discovering near-synonyms rapidly by combining natural language processing and human analytical ability. The results point to a manual examination combined with an enhanced version of our technology as the most efficient strategy. Accuracy is 66%. The goal of [44] is to locate and find ambiguous terms in natural language text that are peculiar to a certain area. This study uses a word embedding-based NLP approach to identify ambiguous terms. More precisely, when terminology from computer science (CS) are employed in other application fields or engineering subdomains. Proposed thorough and in-depth tests on several different subdomains demonstrate that word embedding-based approaches are particularly good at spotting ambiguities that are peculiar to a certain domain. Additionally, our studies show that this method may be used with papers of various sizes. Minimum and maximum similarity scores are both 16.6%. Finally, provide suggestions for more study. In [45] author suggested delving deeper to pragmatic and socio-pragmatic levels of analysis to clear up ambiguity and prevent erroneous interpretations of texts and social media postings, particularly in the sub-tasks of identifying hate speech, in order to avoid making mistakes.

6. Current Research Challenges and Future Research Directions

Despite a lot of recent effort, the applications of NLP have been expanding daily, and with that growth come new obstacles. Because people use a variety of words to convey the same idea, there are also challenges associated with dealing with synonyms in language. Designing models to handle ironic and sarcastic statements is a highly difficult problem in natural language processing (NLP), since ironic and sarcastic sentences may sometimes be received in the opposite manner by humans. Additionally, statements in the language that are ambiguous in the sense that they may be understood in several ways are another area that needs improvement so that greater accuracy can be attained. In reality, these kinds of problems also arise when dealing with several fields, for example, when phrases or sentences may have one meaning in the education sector but another in the fields of health, law, or defense. Therefore, although NLP models may be effective for a specific topic or region, these difficulties must be overcome for widespread application. In addition to the difficulties mentioned above, misspelled or improperly used words can also cause issues. Although autocorrect and grammar checkers have greatly advanced as a result of ongoing research, determining the writer's intent by taking into account sarcasm, expressions, informal language, etc. is still a very difficult task. There is no denying that NLP models for the majority of commonly spoken languages have been performing well and developing daily, but models for all people rather than specialized knowledge of a particular language and technology are still needed. Although there has been much discussion of ambiguity detection recommendations in the literature, this process has not yet been automated. One of the objectives of the proposed work is the automation of ambiguity detection. To be more precise, the methodology and tool described in this paper have three objectives: (1) automate ambiguity detection; (2) convince the analyst that the ambiguities it has found are real issues with the document under study; and (3) inform the analyst by outlining the origins of the ambiguities it has found. [Principal Concepts/Outcomes] In that it finds four times as many real ambiguities as the typical human analyst, the provided method offers trustworthy ambiguity identification. The program also provides very accurate ambiguity identification and does not overburden the human analyst with false positives. The instrument that is being used may both identify ambiguities and explain their causes. As a result, in addition to ambiguity identification, it may also be used to train analysts. Additionally, it offers a major opportunity for time and money savings while also improving the quality of industrial requirements engineering. Some of the challenges observed throughout the study are summarized in fig 5.

There are several challenges faced when developing a pragmatic ambiguity detector:

1. Natural language understanding: One of the main challenges is understanding the meaning of natural language text, as it can be affected by factors such as idiomatic expressions, synonyms, and antonyms, making the detection of ambiguities a complex task.

2. Domain-specific knowledge: Another challenge is the need for domain-specific knowledge to accurately detect and resolve ambiguities. This can be difficult to obtain and may require the involvement of domain experts.
3. Inconsistency: The use of natural language in requirements can lead to inconsistencies and variations in the way that similar concepts are expressed. This can make it difficult for a detector to accurately identify ambiguities.
4. Annotation: Annotating a large dataset of natural language requirements for training a detector is a challenging and time-consuming task.
5. Evaluation: It can be difficult to evaluate the performance of a pragmatic ambiguity detector, as it may require a human expert to review and validate the results.
6. Scalability: The detector should be able to handle large amount of data and to analyze it in a short period of time, otherwise it's not useful for real-world scenarios.
7. Adaptability: The detector should be able to adapt to different types of requirements, written in different languages and from different domains, without the need for extensive retraining.
8. False positives: The detector may produce false positives, i.e, it may identify certain parts of the requirements as ambiguous when they are not. This can lead to additional work and can be frustrating for stakeholders.

To overcome these challenges, it's essential to develop a detector that uses both NLP techniques and domain-specific knowledge, that is well-trained, and that can be evaluated and improved over time.

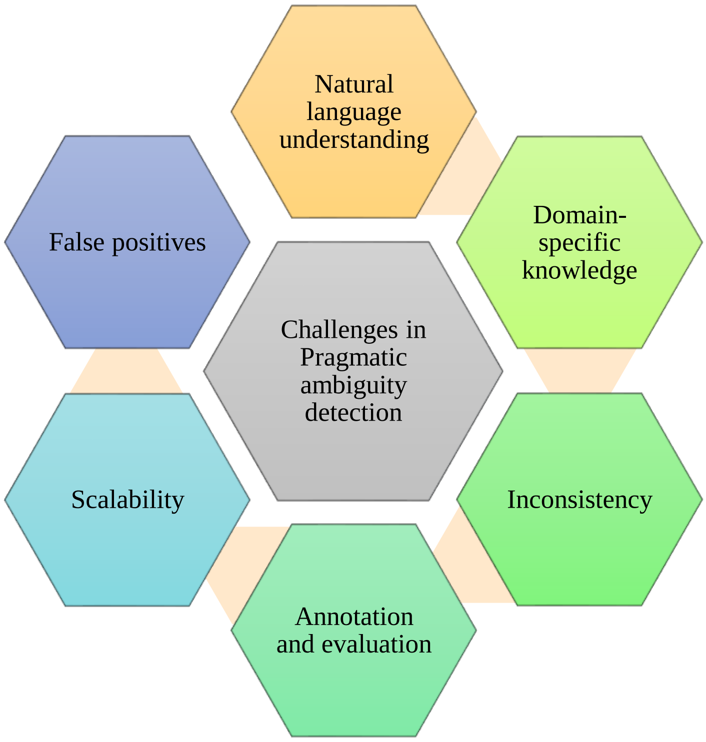

Fig. 5. Challenges in Pragmatic Ambiguity Detection

7. Conclusion

The software development process gains a lot from natural language requirements, but they also include ambiguities that might cause misunderstandings and mistakes. To ensure the overall quality and correctness of the requirements, pragmatic ambiguity identification in natural language requirements is essential. In this research, we have evaluated numerous methods for detecting and resolving pragmatic ambiguities in natural language requirements and we have addressed the significance of pragmatic ambiguity detection in NLRs. Our objective is to motivate additional study in this field by providing a thorough knowledge of the difficulties and possibilities related to pragmatic ambiguity identification in NLRs. This research aims to identify and resolve ambiguities in software requirements. A remark that is ambiguous has more than one meaning. Semantic, syntactic, lexical, syntax, and pragmatic ambiguities are the most prevalent kinds of ambiguity. Therefore, the goal of this research is to assess how ambiguous common computer science terms like "system," "application," and "database" are when used to various contexts. The outcome demonstrates that, in comparison to humans, the created tool was accurate in identifying and resolving pragmatic issues. Research in the area of pragmatic ambiguity identification is advancing quickly. Future improvements in technology, such as those made in NLP, ML, and AI, will probably result in more effective methods for locating and resolving ambiguities. Additionally, the utilization of extensive and varied datasets will result in the creation of models that are more accurate and trustworthy. Pragmatic ambiguity detection will be used increasingly often in conversation systems, automated language translation, and natural language comprehension as the area develops. The utility could be enhanced in the future to support new file types, such PDF. The program does not take into account other forms of ambiguity, such as semantic, syntactic, lexical, and syntax, and it can only identify one type of ambiguity. Future updates to the tool will allow for the addition of the additional ambiguities.

References

[1] Ferrari, G. Lipari, S. Gnesi and G. O. Spagnolo, "Pragmatic ambiguity detection in natural language requirements," 2014 IEEE 1st International Workshop on Artificial Intelligence for Requirements Engineering (AIRE), Karlskrona, Sweden, 2014, pp. 1-8, doi: 10.1109/AIRE.2014.6894849.

[2] Mishra, S., & Sharma, A. (2019). On the use of word embeddings for identifying domain specific ambiguities in requirements. Paper presented at the 2019 IEEE 27th International Requirements Engineering Conference Workshops (REW).

[3] M. Q. Riaz, W. H. Butt and S. Rehman, "Automatic Detection of Ambiguous Software Requirements: An Insight," 2019 5th International Conference on Information Management (ICIM), Cambridge, UK, 2019, pp. 1-6, doi: 10.1109/INFOMAN.2019.8714682.

[4] Sabriye, A. O. J. a., & Zainon, W. M. N. W. (2017). A framework for detecting ambiguity in software requirement specification. Paper presented at the 2017 8th International Conference on Information Technology (ICIT).

[5] Ali, S. W., Ahmed, Q. A., & Shafi, I. (2018). Process to enhance the quality of software requirement specification document. Paper presented at the 2018 International Conference on Engineering and Emerging Technologies (ICEET).

[6] D. Kokane, S. D. Babar and P. N. Mahalle, "Word Sense Disambiguation for Large Documents Using Neural Network Model," 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kharagpur, India, 2021, pp. 1-5, doi: 10.1109/ICCCNT51525.2021.9580101.

[7] Riaz, M. Q., Butt, W. H., & Rehman, S. (2019). Automatic detection of ambiguous software requirements: An insight. Paper presented at the 2019 5th International Conference on Information Management (ICIM).

[8] Nazir, F., Butt, W.H., Anwar, M.W., Khan Khattak, M.A. (2017). The Applications of Natural Language Processing (NLP) for Software Requirement Engineering - A Systematic Literature Review. In: Kim, K., Joukov, N. (eds) Information Science and Applications 2017. ICISA 2017. Lecture Notes in Electrical Engineering, vol 424. Springer, Singapore. https://doi.org/10.1007/978-981-10-4154-9_56

[9] Mathews, S.M. (2019). Explainable Artificial Intelligence Applications in NLP, Biomedical, and Malware Classification: A Literature Review. In: Arai, K., Bhatia, R., Kapoor, S. (eds) Intelligent Computing. CompCom 2019. Advances in Intelligent Systems and Computing, vol 998. Springer, Cham. https://doi.org/10.1007/978-3-030-22868-2_90

[10] Shankar, V., Parsana, S. An overview and empirical comparison of natural language processing (NLP) models and an introduction to and empirical application of autoencoder models in marketing. J. of the Acad. Mark. Sci. 50, 1324–1350 (2022). https://doi.org/10.1007/s11747-022-00840-3

[11] Yue Kang, Zhao Cai, Chee-Wee Tan, Qian Huang & Hefu Liu (2020) Natural language processing (NLP) in management research: A literature review, Journal of Management Analytics, 7:2, 139-172, DOI: 10.1080/23270012.2020.1756939

[12] Dhar, A., Mukherjee, H., Dash, N.S. et al. Text categorization: past and present. Artif Intell Rev 54, 3007–3054 (2021). https://doi.org/10.1007/s10462-020-09919-1

[13] P. Garg and N. Girdhar, "A Systematic Review on Spam Filtering Techniques based on Natural Language Processing Framework," 2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence), Noida, India, 2021, pp. 30-35, doi: 10.1109/Confluence51648.2021.9377042.

[14] S. Alves, J. Costa and J. Bernardino, "Information Extraction Applications for Clinical Trials: A Survey," 2019 14th Iberian Conference on Information Systems and Technologies (CISTI), Coimbra, Portugal, 2019, pp. 1-6, doi: 10.23919/CISTI.2019.8760639.

[15] Abualigah, L., Bashabsheh, M.Q., Alabool, H., Shehab, M. (2020). Text Summarization: A Brief Review. In: Abd Elaziz, M., Al-qaness, M., Ewees, A., Dahou, A. (eds) Recent Advances in NLP: The Case of Arabic Language. Studies in Computational

Intelligence, vol 874. Springer, Cham. https://doi.org/10.1007/978-3-030-34614-0_1
[16] Ni, J., Young, T., Pandelea, V. et al. Recent advances in deep learning based dialogue systems: a systematic survey. Artif Intell Rev (2022). https://doi.org/10.1007/s10462-022-10248-8
[17] Zhou, G. Yang, Z. Shi and S. Ma, "Natural Language Processing for Smart Healthcare," in IEEE Reviews in Biomedical Engineering, 2022, doi: 10.1109/RBME.2022.3210270.
[18] M. Bano, "Addressing the challenges of requirements ambiguity: A review of empirical literature," 2015 IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE), Ottawa, ON, Canada, 2015, pp. 21-24, doi: 10.1109/EmpiRE.2015.7431303.
[19] Chowdhary, K.R. (2020). Natural Language Processing. In: Fundamentals of Artificial Intelligence. Springer, New Delhi. https://doi.org/10.1007/978-81-322-3972-7_19
[20] R. Sahoo, B. R. Das and B. Kishore Mishra, "Analysis and Implementation of Odia Part of Speech Tagger in recent IoT based devices through Chatbot: A review," 2020 International Conference on Computer Science, Engineering and Applications (ICCSEA), Gunupur, India, 2020, pp. 1-4, doi: 10.1109/ICCSEA49143.2020.9132940.
[21] Das Dawn, D., Shaikh, S.H. & Pal, R.K. A comprehensive review of Bengali word sense disambiguation. Artif Intell Rev 53, 4183–4213 (2020). https://doi.org/10.1007/s10462-019-09790-9
[22] Harish, B.S., Rangan, R.K. A comprehensive survey on Indian regional language processing. SN Appl. Sci. 2, 1204 (2020). https://doi.org/10.1007/s42452-020-2983-x
[23] T. Kato and K. Tsuda, "A Method of Ambiguity Detection in Requirement Specifications by Using a Knowledge Dictionary," Procedia Comput. Sci., vol. 207, pp. 1482–1489, 2022, doi: https://doi.org/10.1016/j.procs.2022.09.205.
[24] Manam, VK Chaithanya, Joseph Divyan Thomas, and Alexander J. Quinn. "TaskLint: Automated Detection of Ambiguities in Task Instructions." Proceedings of the AAAI Conference on Human Computation and Crowdsourcing. Vol. 10. No. 1. 2022.
[25] S. Ezzini, S. Abualhaija, C. Arora and M. Sabetzadeh, "Automated Handling of Anaphoric Ambiguity in Requirements: A Multi-solution Study," 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, 2022, pp. 187-199, doi: 10.1145/3510003.3510157.
[26] Roopa, H. R., and S. Panneer Arockiaraj. "The Role of Artificial Neural Network in Word Sense Disambiguation (WSD)—A Survey." Rising Threats in Expert Applications and Solutions. Springer, Singapore, 2022. 221-227.
[27] Kaddoura, Sanaa, and Rowanda D. Ahmed. "A comprehensive review on Arabic word sense disambiguation for natural language processing applications." Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery (2022): e1447.
[28] Saxena, Shefali, et al. "Improved unsupervised statistical machine translation via unsupervised word sense disambiguation for a low-resource and Indic languages." IETE Journal of Research (2022): 1-11.
[29] Yadav, Apurwa, Aarshil Patel, and Manan Shah. "A comprehensive review on resolving ambiguities in natural language processing." AI Open 2 (2021): 85-92.
[30] S. Ezzini, S. Abualhaija, C. Arora, M. Sabetzadeh and L. C. Briand, "Using Domain-Specific Corpora for Improved Handling of Ambiguity in Requirements," 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), Madrid, ES, 2021, pp. 1485-1497, doi: 10.1109/ICSE43902.2021.00133.
[31] M. Osama, A. Zaki-Ismail, M. Abdelrazek, J. Grundy and A. Ibrahim, "Score-Based Automatic Detection and Resolution of Syntactic Ambiguity in Natural Language Requirements," 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), Adelaide, SA, Australia, 2020, pp. 651-661, doi: 10.1109/ICSME46990.2020.00067.
[32] Mishra, Siba, and Arpit Sharma. "On the use of word embeddings for identifying domain specific ambiguities in requirements." 2019 IEEE 27th International Requirements Engineering Conference Workshops (REW). IEEE, 2019.
[33] Ferrari, Alessio, et al. "Detecting requirements defects with NLP patterns: an industrial experience in the railway domain." Empirical Software Engineering 23.6 (2018): 3684-3733.
[34] (Walton D. (1996) A Pragmatic Synthesis. In: Fallacies Arising from Ambiguity. Applied Logic Series, vol 1. Springer, Dordrecht).
[35] R. Sonbol, G. Rebdawi and N. Ghneim, "The Use of NLP-Based Text Representation Techniques to Support Requirement Engineering Tasks: A Systematic Mapping Review," in IEEE Access, vol. 10, pp. 62811-62830, 2022, doi: 10.1109/ACCESS.2022.3182372.
[36] Khurana, D., Koli, A., Khatter, K. et al. Natural language processing: state of the art, current trends and challenges. Multimed Tools Appl 82, 3713–3744 (2023). https://doi.org/10.1007/s11042-022-13428-4
[37] Apurwa Yadav, Aarshil Patel, Manan Shah, A comprehensive review on resolving ambiguities in natural language processing, AI Open, Volume 2, 2021, Pages 85-92, ISSN 2666-6510, https://doi.org/10.1016/j.aiopen.2021.05.001.

[38] T. P. Nagarhalli, V. Vaze and N. K. Rana, "Impact of Machine Learning in Natural Language Processing: A Review," 2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), Tirunelveli, India, 2021, pp. 1529-1534, doi: 10.1109/ICICV50876.2021.9388380.

[39] Oo, K.H. (2023). Comparing Accuracy Between SVM, Random Forest, K-NN Text Classifier Algorithms for Detecting Syntactic Ambiguity in Software Requirements. In: Al-Emran, M., Al-Sharafi, M.A., Shaalan, K. (eds) International Conference on Information Systems and Intelligent Applications. ICISIA 2022. Lecture Notes in Networks and Systems, vol 550. Springer, Cham. https://doi.org/10.1007/978-3-031-16865-9_4

[40] Moharil and A. Sharma, "Identification of Intra-Domain Ambiguity using Transformer-based Machine Learning," 2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE), Pittsburgh, PA, USA, 2022, pp. 51-58, doi: 10.1145/3528588.3528651.

[41] M. Osama, A. Zaki-Ismail, M. Abdelrazek, J. Grundy and A. Ibrahim, "Score-Based Automatic Detection and Resolution of Syntactic Ambiguity in Natural Language Requirements," 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), Adelaide, SA, Australia, 2020, pp. 651-661, doi: 10.1109/ICSME46990.2020.00067.

[42] Khalid Abdikarim Mohamed, Jamilah Din, & Salmi Baharom. (2022). A Tool to Detect Pragmatic Ambiguity with Possible Interpretations Suggestion in Software Requirement Specifications. International Journal of Synergy in Engineering and Technology, 3(2), 52-60. Retrieved from https://tatiuc.edu.my/ijset/index.php/ijset/article/view/141

[43] Fabiano Dalpiaz, Ivor van der Schalk, Sjaak Brinkkemper, Fatma Başak Aydemir, Garm Lucassen, Detecting terminological ambiguity in user stories: Tool and experimentation, Information and Software Technology, Volume 110, 2019, Pages 3-16, ISSN 0950-5849, https://doi.org/10.1016/j.infsof.2018.12.007.

[44] S. Mishra and A. Sharma, "On the Use of Word Embeddings for Identifying Domain Specific Ambiguities in Requirements," 2019 IEEE 27th International Requirements Engineering Conference Workshops (REW), Jeju, Korea (South), 2019, pp. 234-240, doi: 10.1109/REW.2019.00048.

[45] S. Assem and S. Alansary, "Sentiment Analysis From Subjectivity to (Im)Politeness Detection: Hate Speech From a Socio-Pragmatic Perspective," 2022 20th International Conference on Language Engineering (ESOLEC), Cairo, Egypt, 2022, pp. 19-23, doi: 10.1109/ESOLEC54569.2022.10009298.

Using NLP Tools to Detect Ambiguities in System Requirements - A Comparison Study

Aleksandar Bajceta[1], Miguel Leon[1], Wasif Afzal[1], Pernilla Lindberg[2] and Markus Bohlin[1]

[1]*Mälardalens universitet, Box 325, 631 05 Eskilstuna*
[2]*Alstom Sweden, 721 14 Västerås*

Abstract
Requirements engineering is a time-consuming process, and it can benefit significantly from automated tool support. Ambiguity detection in natural language requirements is a challenging problem in the requirements engineering community. Several Natural Language Processing tools and techniques have been developed to improve and solve the problem of ambiguity detection in natural language requirements. However, there is a lack of empirical evaluation of these tools. We aim to contribute the understanding of the empirical performance of such solutions by evaluating four tools using the dataset of 180 system requirements from the electric train propulsion system provided to us by our industrial partner Alstom. The tools that were selected for this study are Automated Requirements Measurement (ARM), Quality Analyzer for Requirement Specifications (QuARS), REquirements Template Analyzer (RETA), and Requirements Complexity Measurement (RCM). Our analysis showed that selected tools could achieve high recall. Two of them had the recall of 0.85 and 0.98. But they struggled to achieve high precision. The RCM, an in-house developed tool by our industrial partner Alstom, achieved the highest precision in our study of 0.68.

Keywords
Requirements engineering, Natural language requirements, Ambiguity, Natural language processing

1. Introduction

Good requirements engineering is the backbone of every successfully developed system. It is a process where engineers define the stakeholders' needs and their transformation into clearly described systems' behavior. Many projects have failed due to a poor requirements engineering process. According to [1], bad requirements engineering is the cause in 71% of software project failures.

This paper considers a comparison study for ambiguity analysis applied to the requirements in a safety-critical application in the railway domain. The development of safety-critical systems, such as those used in the railway industry, must follow international standards, and they demand that requirements documents should be complete, clear, precise, unequivocal, verifiable, testable, maintainable, and feasible [2].

NLP4RE 2022: 5th Workshop on Natural Language Processing for Requirements Engineering @ REFSQ, Birmingham, UK (CEUR workshop): https://nlp4re.github.io/2022/
aleksandar.bajceta@mdh.se (A. Bajceta); miguel.leonortiz@mdh.se (M. Leon); wasif.afzal@mdh.se (W. Afzal); pernilla.lindberg@alstomgroup.com (P. Lindberg); markus.bohlin@mdh.se (M. Bohlin)

Most of the requirements documents are written in some form of natural language [3]. It is easy to write and understand natural language requirements, but they can be more ambiguous and error-prone. The ambiguous requirement could be understood differently between developers, testers, and other stakeholders involved in project development. The occurrence of underspecified or abstract requirements that can be interpreted differently is one of the most common requirements engineering problems [4]. This kind of miscommunication could lead to project delay or failure. Because of that, the identification of ambiguities in natural language requirements plays a vital role in requirements engineering.

A system requirement is ambiguous if there is more than one interpretation. Analyzing requirements for defects and ambiguities is a time-consuming process, and it can benefit from an automated solution. Using Natural Language Processing (NLP) for requirements analysis has been a popular research topic. Researchers created several tools for identifying ambiguities or defects in requirements using pattern or rule-based approaches [2, 5–10]. However, there is still a lack of industrial evaluation in using these tools in the research papers [11]. Our paper aims to fill the research gap by evaluating several requirements analysis tools used for ambiguity detection in natural language requirements in the railway domain.

Tools that we selected are Automated Requirements Measurement (ARM) [8], Quality Analyzer for Requirement Specifications (QuARS) [9], REquirements Template Analyzer (RETA) [5], and Requirements Complexity Measurement (RCM) [10]. We selected the tools based on suitability for the type of systems considered in this paper and their availability. The three first tools were publicly available for use by anyone, and the fourth tool was developed in-house at Alstom, and it is also publicly available [1]. Furthermore, they all use a similar approach for identifying ambiguities, making them a good candidate for comparison studies. A sample set of 180 requirements were provided to us by our industrial partner, Alstom Transport AB in Sweden. These requirements are used in one of the Alstoms' projects for defining specifications of a train propulsion system. After running selected tools on this set of requirements, we compared the tool's ambiguity identification results with ambiguities found by an expert responsible for requirements engineering in Alstom.

The remainder of the paper is structured as follows. The related work is presented in Section 2. In Section 3, we described the tools used in the analysis. The followed methodology is described in Section 4. Results of the study are presented in Section 5. We addressed validity threats in Section 6. Section 7 concluded the paper.

2. Related Work

We identified several papers that compare requirements analysis tools, whose focus is on reducing ambiguities and improving overall requirements quality.

In [12] researchers compared their unnamed tool that was in development with two additional requirements engineering tools, reconstructed ARM tool [8] and TIGER Pro 2.0 [2]. The study was performed on a set of requirements from two projects. Researchers concluded that their unnamed new tool is still not good enough and that it is identifying many false positive

[1] https://github.com/eduardenoiu/NALABS/tree/master/RCM
[2] http://www.therightrequirement.com/TigerPro/TigerPro.html

examples. A similar comparison study was performed in [13], where the authors presented their experience report using three different tools for requirement analysis. They compared two commercial tools, Requirement Scout and QVscribe, and one academic tool, QuARS. The study was performed by running the tools on the set of requirements from a simple E-shop software. The conclusions were that all three tools are performing quite similarly and that the two newer tools, Requirement Scout and QVscribe, did not outperform QuARS which was developed almost 20 years ago.

Different techniques, tools, and their technologies are reviewed and discussed in [14–17]. In these papers, researchers were reviewing the tools and technologies, and they were not experimenting on an actual set of requirements. In [14] researchers performed a literature review focusing on the tools for automatic detection of ambiguities in the requirements. They extracted 25 tools developed between 2008 and 2018. Ten of the most popular tools were compared with the addressed ambiguity, technologies used, and approaches in detecting ambiguities. In [15] researchers reviewed different techniques used for ambiguity detection in the NL requirements. They identified three approaches in detecting ambiguities: manual, semi-automatic using natural language processing techniques, and semi-automatic using machine learning techniques. They classified the detection of ambiguities with patterns as a Semi-Automatic approach using NLP. Similar studies were done in [16], [17]. A literature review was performed on detecting and resolving ambiguities in the NL requirements. Researchers compared tools and approaches by different parameters such as used technologies, targeting ambiguity, user interaction, etc.

In our work, we introduce two tools that were not compared in the literature. Also, tools are evaluated using requirements that were assessed and used in an industrial project.

3. Selected Tools

The availability of tools for requirements engineering is poor, and most of them are not publicly available [11]. Public versions of several tools do not contain all features, such as the set of patterns needed for the tool's full functionality. We were also unable to run specific tools in our environment. For our study, we selected four tools. The tools ARM and QuARS are among the oldest tools for requirement analysis. But they are still quite popular, and they built a foundation for further research and tools development. RETA is a bit newer tool, and comparison with older tools is relevant to us. The fourth tool selected for this study is RCM, a tool developed by Alstom, primarily to measure requirements complexity in Alstom's internal projects, but it can be used to detect ambiguities. The focus of these tools is to improve the quality, reduce complexity, and identify ambiguity in requirements. They automatically detect ambiguous requirements by comparing requirements with their patterns or rules.

3.1. Automated Requirements Measurement (ARM)

NASA developed the ARM tool in the 1990s. In 2011, it was discontinued, and for our study, we are using a reconstructed ARM tool developed by Carlson and Laplante [8]. The ARM tool has several quality indicators that are used for requirement analysis. Those quality indicators are imperatives, directives, continuances, size, specification depth, text structure, options, and weak phrases. Imperatives indicate absolute necessity using command words such as "shall" or “must“. Continuances are words or phrases that follow imperative words, and their extensive use could increase requirement complexity. Examples of continuances are "following ", "as follows ", "and", etc. Size includes counts of three indicators: total lines of text, the total number of imperative words, and the total number of subjects. Specification depth indicates how concise the document specifies the requirement by calculating the number of imperative statements at each level of the requirement document's text structure. The text structure measures the number of indicators identified at each hierarchical level of the requirement document. Based on NASA's requirements quality model, indicators of ambiguity are optional and weak phrases. The terms for options are “can“, “may“, and “optionally“. These terms loosen the specification, and they could allow the developer to judge what should be implemented in more than one way. Weak phrases terms are a set of words that could leave requirements with multiple or subjective interpretations. There are 12 pre-defined weak phrases in the ARM tool, and some of those terms are “adequate“, “be able to“, “timely“, “as appropriate“, etc.

3.2. Quality Analyzer for Requirement Specifications (QuARS)

QuARS is a requirement analysis tool that focuses on ambiguity detection developed by Lami et al. [9]. QuARS performs lexical and syntactic analysis to find ambiguities in natural language requirements. To perform lexical analysis, QuARS uses a set of patterns or keywords to identify optionality, subjectivity, vagueness, and weakness in the requirements. Optionality indicates that the requirement contains an optional part. Examples of terms used to identify optionality are “possibly“, “eventually“, “optionally“ etc. Subjectivity represents the occurrence of terms that indicates personal opinion. For example, “having in mind“ or “take into account“. Vagueness indicates non-quantifiable terms such as “significant“ or “adequate“. Weakness indicates that the requirement does not have an imperative, and it is identified with terms such as “can“, “could“, “may“ etc. The goal of syntactic analysis is to find patterns that could identify implicity, multiplicity, and under-specification among requirements. Implicity occurs when pronouns or other indirect references represent a subject or object in a sentence. Multiplicity occurs when the sentence has more than one main verb, subject, or object. Under-specification represents the occurrence of words that should be instantiated, for example, testing instead of functional testing or unit testing.

3.3. REquirements Template Analyzer (RETA)

RETA is a requirement analysis tool for automatically checking natural language requirements against templates for conformance [5]. The tool is implemented as a plugin for an open-source framework called GATE workbench [3]. The RETA tool supports analysis of the requirements against two templates, Rupp's [18] and the EARS templates [19]. Templates represent a requirement structure, and when followed, it should reduce ambiguity in the requirement. Also, RETA tool checks for potentially dangerous terms and phrases that could cause ambiguity. These terms are looking for constructions such as passive voice, pronouns, quantifiers (terms used for quantification, such as “all“, “any“, “every“), and/or, vague terms, plural terms, etc.

[3]https://gate.ac.uk/

3.4. Requirements Complexity Measurement (RCM)

RCM is a tool developed in Alstom used for analyzing requirements complexity [10]. The tool's metrics for measuring complexity are the following: number of words, number of vague phrases, number of conjunctions, number of reference documents, optionality, subjectivity, weakness, automated readability index, imperatives, and continuances. The number of words is a metric used to measure the length of the requirement. The number of vague phrases represents the total number of terms that could cause problems in understanding the requirement, such as "adequate", "appropriate", "normal", etc. The number of conjunction counts the total number of phrases such as "and", "after", "although", etc. The number of reference documents indicates a need for additional reading to understand the requirement that contains references to other documents. Optionality metrics is implemented in the same way as in the ARM tool, and it uses keywords such as "can", "may", and "optionally" to identify it. Subjectivity indicates a personal feeling or opinion in requirement. To identify subjectivity, the tool uses keywords such as "similar", "better", "worse", etc. Weakness indicates phrases that could introduce uncertainty into requirements by using keywords such as "timely", "be able to", "be capable of", etc. The automated readability index is dependent on the total number of words in the requirement and the average number of letters per word. Imperatives indicate command words, and it is identified using keywords such as "shall", "must", "will", etc. Continuances are implemented in the same way as in the ARM tool. Measures selected to identify ambiguity among requirements are optionality, subjectivity, vague phrases, and weak phrases. We chose these indicators since Alstom's engineers identified them during tool development as rules that should be followed to avoid ambiguity in requirements.

4. Methodology

To evaluate selected tools, we used a dataset provided to us by our industrial partner Alstom. Alstom is one of the leaders in the railway industry and the largest railway company in Sweden. The dataset contains 180 requirements, written in the English language, for the train propulsion system development used in one of their previous projects. The average size of the requirement is 32.46 words. The shortest requirement contains 8 words, and the longest one has 205 words. Most of the requirements, or some parts of them, are written in passive form. Requirements were developed from the customer specifications, and they describe both hardware and software components of the system.

We used selected tools described in Section 3 to analyze the requirements and detect potential ambiguities in them. We compared results from tools' analysis with the analysis from Alstom's engineer, a person responsible for requirements engineering with years of experience in the railway domain. We asked the engineer to "Identify requirements in the dataset that have defects and could cause ambiguity among different stakeholders". If the requirement is potentially ambiguous, the engineer was supposed to mark it with "yes" and with "no" if it is not. We presented several examples of ambiguous sentences to the engineer. The ambiguous sentences were developed using definitions and examples from the Ambiguity Handbook [20].

- "I saw a man at the bank." - the bank can be a financial institution or ground bordering a river

- "I saw Peter and Paul and Mary saw me." - can be understood as I saw (Peter and Paul) and Mary saw me, or I saw Peter and (Paul and Mary) saw me
- "All linguists prefer a theory." - can be understood as all linguists love the same one theory, or each linguist loves a, perhaps different, theory
- "The trucks shall treat the roads before they freeze." - pronoun "they" can be either trucks or roads
- "Avoid long C functions." - we do not know the value of "long"

Results from the engineer's analysis are taken as a ground truth. If the requirement was marked as ambiguous by the engineer and it was identified by the tool, it was counted as a true positive result (TP). A false-positive result (FP) is if the requirement was marked as ambiguous by the tool but not by the engineer. A false-negative result (FN) is if the requirement was marked as ambiguous by the engineer but not identified as ambiguous by the tool. We used precision and recall as performance metrics to evaluate tools since it's one of the most common metrics for tools' evaluation. Precision and Recall for tools are calculated as $Precision = TP/(TP + FP)$, and $Recall = TP/(TP + FN)$.

5. Results

Results from the tools' analysis are presented in Table 1. We group the results by tools' quality indicators. If the tool does not support a specific category, the dash symbol "-" is used in the table. The ARM tool identified 7 potential issues that fit the option quality indicator category and 10 for the weak phrase quality indicator. The same results for these two categories came from the RCM tool. This is expected since both tools use the same terms for these two categories. The RCM tool also identified 2 issues for subjectivity and 38 issues because of vague terms. Terms "can" and "may" are used by ARM and RCM to identify issues for option quality indicator, QuARS uses them to identify weakness in requirements, along with additional terms. QuARS identified 18 issues that could cause weakness in requirements, 5 issues for subjectivity, and 42 vagueness issues. QuARS identified 21 implicity, 116 multiplicity, and 32 underspecification issues in syntactic analysis. RETA tool identified 56 issues with vague terms in requirements, 42 issues with quantifier terms, 52 pronouns, 66 complex requirements, and 51 adverbs. The RETA also uses patterns to identify plural terms and passive sentences. Because of it, almost all requirements were marked as potentially ambiguous. It is important to mention that multiple issues can be identified in one requirement.

In the Table 2 results of tool evaluation is presented. We compared tools' analysis with the analysis performed by the engineer from Alstom, who identified 74 requirements as ambiguous. RCM achieved the highest precision in our analysis, 0.68. The rest of the tools achieved similar precision between each other. ARM and QuARS precision are 0.43, while RETA's precision is 0.41. RETA also achieved the highest recall, 0.98, by identifying 177 requirements as potentially ambiguous. At first, this looks like significant results, since in literature recall is considered more important [21]. But it is essential to mention that this result was expected because RETA identifies plural words and passive sentences as an issue that could cause ambiguity. The second

Table 1
Ambiguity issues identified by each tool(“-“ stands for not supported by the tool)

Category	ARM	QuARS	RETA	RCM
Option/Optionality	7	0	-	7
Weak Phrase/Weakness	10	18	-	10
Subjectivity	-	5	-	2
Vagueness/Vague Term	-	42	56	38
Quantifier	-	-	42	-
Pronoun	-	-	52	-
Plural	-	-	301	-
Passive	-	-	159	-
Complex	-	-	66	-
Adverb	-	-	51	-
And/OR	-	-	154	-
Implicity	-	21	-	-
Multiplicity	-	116	-	-
Underspecificaiton	-	32	-	-

highest recall was achieved by QuARS 0.85, RCM follows it with 0.38, and ARM tool with only 0.08.

From the results, we can conclude that it's hard to achieve high precision in ambiguity detection using solutions based on pattern detection. Even though it is possible to achieve a very high recall with an extensive set of patterns, it will also result in many false-positive results. Similar results were also achieved in [7]. RCM tool achieved the highest precision, and this could be expected because it was developed inside Alstom.

Table 2
Summary of tool's analysis

	True Positive	False Positive	False Negative	Precision	Recall
ARM	6	8	68	0.43	0.08
QuARS	63	85	11	0.43	0.85
RETA	73	103	1	0.41	0.98
RCM	28	13	46	0.68	0.38

6. Threats to validity

In this section, we present the threats that could affect the validity of our results. According to [22], the threats to validity can be categorized as construct validity, internal validity, external validity, and reliability.

Construct validity refers to the extent to which studied operational measures represent what the researchers intended to study. The construct validity of our results is the number of tools selected for this paper. As mentioned in Section 3 many tools are not publicly available. To tackle potential threats, we've included all relevant tools that, to the best of our knowledge, were publicly available.

External validity refers to what extent results from the study can be generalized. The presented findings in this study are obtained from the analysis of system requirements used to develop an electric train propulsion system. The dataset we used for evaluation contains 180 requirements, which might not be enough to evaluate the general usage of selected tools. We do not claim that these results can be generalized in other contexts. However, we believe that these findings could be helpful for the further evaluation or usage of these tools or to gain insights for future tool development.

Internal validity refers to what extent relations between investigated factors can affect the credibility of results. Assessing whether something is ambiguous or not is a subjective activity, and it depends on the analyst's experience and interpretation. Also, to determine the level of ambiguity, it is more common to analyze and compare answers from multiple analysts. To mitigate internal validity, we asked an expert in requirements engineering with years of experience working in the railway industry to analyze requirements and establish ground truth.

Reliability refers to what extent the data and the analysis depend on the researchers who performed the study. We asked an expert to analyze the system requirements to mitigate this threat. The expert was not included in the design of the study, or in performing the calculations of the results. Also, the tools were used after the expert finished the analysis.

7. Conclusions and Future work

We analyzed four different requirement analysis tools using 180 system requirements from a larger project in train propulsion design. The tools in the study all use a pattern-based approach to identify ambiguities in the requirements. Our analysis showed that it is possible to achieve high recall by following this approach, but increased precision is challenging. The RCM tool, developed inside Alstom, had the highest precision in our analysis. In comparison, the RETA tool achieved the highest recall.

A possible explanation is that domain knowledge plays an important role in identifying ambiguities. To the best of our knowledge, no papers have investigated how domain knowledge affects ambiguity identification. Therefore, how domain knowledge and experience affect identifying ambiguous requirements is the question that we would like to examine in the future. We plan to survey industry practitioners and academics to explore these questions. This kind of study would help us better understand the problem of ambiguity and what kind of ambiguity we should focus our research on in the future. We also aim to analyze other approaches and tools and identify what techniques can achieve the highest precision and a low number of false-positive cases in our context.

8. Acknowledgments

This research work has received funding from the ECSEL Joint Undertaking (JU) under grant agreement No 101007350. The JU receives support from the European Union's Horizon 2020 research and innovation programme and Sweden, Austria, Czech Republic, Finland, France, Italy, Spain.

References

[1] Infectra, Principles of requirements engineering or requirements management 101, 2022. URL: https://www.inflectra.com/ideas/whitepaper/

principles-of-requirements-engineering.aspx.
[2] B. Rosadini, A. Ferrari, G. Gori, A. Fantechi, S. Gnesi, I. Trotta, S. Bacherini, Using NLP to detect requirements defects: An industrial experience in the railway domain, in: International Working Conference on Requirements Engineering: Foundation for Software Quality, Springer, 2017, pp. 344–360.
[3] M. Kassab, C. Neill, P. Laplante, State of practice in requirements engineering: contemporary data, Innovations in Systems and Software Engineering 10 (2014) 235–241.
[4] D. M. Fernández, S. Wagner, M. Kalinowski, M. Felderer, P. Mafra, A. Vetrò, T. Conte, M.-T. Christiansson, D. Greer, C. Lassenius, et al., Naming the pain in requirements engineering, Empirical software engineering 22 (2017) 2298–2338.
[5] C. Arora, M. Sabetzadeh, L. Briand, F. Zimmer, Automated checking of conformance to requirements templates using natural language processing, IEEE transactions on Software Engineering 41 (2015) 944–968.
[6] B. Gleich, O. Creighton, L. Kof, Ambiguity detection: Towards a tool explaining ambiguity sources, in: International Working Conference on Requirements Engineering: Foundation for Software Quality, Springer, 2010, pp. 218–232.
[7] S. F. Tjong, D. M. Berry, The design of SREE—a prototype potential ambiguity finder for requirements specifications and lessons learned, in: International Working Conference on Requirements Engineering: Foundation for Software Quality, Springer, 2013, pp. 80–95.
[8] N. Carlson, P. Laplante, The NASA automated requirements measurement tool: a reconstruction, Innovations in Systems and Software Engineering 10 (2014) 77–91.
[9] G. Lami, S. Gnesi, F. Fabbrini, M. Fusani, G. Trentanni, An automatic tool for the analysis of natural language requirements, Informe técnico, CNR Information Science and Technology Institute, Pisa, Italia, Setiembre (2004).
[10] K. Rajković, Measuring the complexity of natural language requirements in industrial control systems, 2019.
[11] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. Ajagbe, E.-V. Chioasca, R. T. Batista-Navarro, Natural language processing (NLP) for requirements engineering (RE): A systematic mapping study, ACM Computing Surveys (2020).
[12] A. Brooks, L. Krebs, B. Paulsen, Beta-testing a requirements analysis tool, ACM SIGSOFT Software Engineering Notes 39 (2014) 1–6.
[13] M. Arrabito, A. Fantechi, S. Gnesi, L. Semini, An experience with the application of three NLP tools for the analysis of natural language requirements, in: International Conference on the Quality of Information and Communications Technology, Springer, 2020, pp. 488–498.
[14] M. Q. Riaz, W. H. Butt, S. Rehman, Automatic detection of ambiguous software requirements: An insight, in: 2019 5th International Conference on Information Management (ICIM), IEEE, 2019, pp. 1–6.
[15] K. H. Oo, A. Nordin, A. R. Ismail, S. Sulaiman, An analysis of ambiguity detection techniques for software requirements specification (SRS), International Journal of Engineering & Technology 7 (2018) 501–505.
[16] A. Yadav, A. Patel, M. Shah, A comprehensive review on resolving ambiguities in natural language processing, AI Open 2 (2021) 85–92.
[17] U. S. Shah, D. C. Jinwala, Resolving ambiguities in natural language software requirements:

a comprehensive survey, ACM SIGSOFT Software Engineering Notes 40 (2015) 1–7.
[18] C. Rupp, K. Pohl, Requirements engineering fundamentals, Rocky Nook (2011).
[19] A. Mavin, P. Wilkinson, A. Harwood, M. Novak, Easy approach to requirements syntax (EARS), in: 2009 17th IEEE International Requirements Engineering Conference, IEEE, 2009, pp. 317–322.
[20] D. M. Berry, E. Kamsties, M. M. Krieger, From contract drafting to software specification: Linguistic sources of ambiguity, 2003. URL: http://se.uwaterloo.ca/~dberry/handbook/ambiguityHandbook.pdf.
[21] D. M. Berry, Empirical evaluation of tools for hairy requirements engineering tasks, Empirical Software Engineering 26 (2021) 1–77.
[22] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empirical software engineering 14 (2009) 131–164.

An Exploration-Analysis-Disambiguation Reasoning Framework for Word Sense Disambiguation with Low-Parameter LLMs

Deshan Sumanathilaka, Nicholas Micallef, Julian Hough
Department of Computer Science, Swansea University
Fabian Way, Crymlyn Burrows, Swansea, Wales, UK
{t.g.d.sumanathilaka, nicholas.micallef, julian.hough}@swansea.ac.uk

Abstract

Word Sense Disambiguation (WSD) remains a key challenge in Natural Language Processing (NLP), especially when dealing with rare or domain-specific senses that are often misinterpreted. While modern high-parameter Large Language Models (LLMs) such as GPT-4-Turbo have shown state-of-the-art WSD performance, their computational and energy demands limit scalability. This study investigates whether low-parameter LLMs (<4B parameters) can achieve comparable results through fine-tuning strategies that emphasize reasoning-driven sense identification. Using the FEWS dataset augmented with semi-automated, rationale-rich annotations, we fine-tune eight small-scale open-source LLMs (e.g. Gemma and Qwen). Our results reveal that Chain-of-Thought (CoT)-based reasoning combined with neighbour-word analysis achieves performance comparable to GPT-4-Turbo in zero-shot settings. Importantly, Gemma-3-4B and Qwen-3-4B models consistently outperform all medium-parameter baselines and state-of-the-art models on FEWS, with robust generalization to unseen senses. Furthermore, evaluation on the unseen “Fool Me If You Can” dataset confirms strong cross-domain adaptability without task-specific fine-tuning. This work demonstrates that with carefully crafted reasoning-centric fine-tuning, low-parameter LLMs can deliver accurate WSD while substantially reducing computational and energy demands.

Keywords: Word Sense Disambiguation, Low-parameter LLMs, Reasoning-driven Fine-tuning

1. Introduction

With recent advances in large language models (LLMs), Natural Language Processing (NLP) tasks such as machine translation, information retrieval, question answering, sentiment analysis, and summarization have achieved near-human performance in modern applications (Chang et al., 2024; Sumanathilaka et al., 2025c). However, to perform such linguistic tasks effectively, word sense disambiguation (WSD) plays a major role, as ambiguity can lead to incorrect information and the generation of misinformation (Singh et al., 2024). For example, during the COVID-19 pandemic, the term “*positive*” could indicate a confirmed infection or express optimism in a general context which are two drastically different meanings that require contextual understanding to avoid incorrect inferences. Understanding the meaning of a word in a sentence or phrase and disambiguating the correct sense is crucial for achieving accurate results, particularly when tackling the lexical ambiguity of an ambiguous word.

Working with words that have multiple meanings (polysemy) is a major challenge for current NLP systems, as highly polysemous words are difficult to disambiguate due to the nature of usage. For example, according to BabelNet[1], the word ‘*bank*’ has 49 meanings, including 40 as a nouns and 9 as a verb. However, in common usage, the senses “financial institution” (money bank) and “sloping land beside a body of water” (river bank) dominate. In contrast, senses including “a flight maneuver” (highly domain-specific to aviation) or “a slope in the turn of a road or track” (used mainly in engineering, road design) are rare in everyday usage. In domain-specific situations, models often struggle to disambiguate less common senses effectively (Awotunde, 2025).

Recent studies on WSD using large language models (LLMs) have revealed promising findings, including significant improvements in disambiguating rare senses and leveraging broader contextual information, paving the way for enhanced language understanding and more accurate disambiguation. For instance, Sumanathilaka et al. (2024b) evaluated multiple large-parameter LLMs for disambiguation using few-shot and knowledge-base-combined methods, highlighting the strengths of LLMs for WSD. However, other studies also reveal important limitations. Basile et al. (2025) demonstrated that while LLMs perform well in zero-shot scenarios, they cannot surpass current state-of-the-art methods without fine-tuning, suggesting that existing approaches do not fully exploit LLM capabilities or generalize across diverse WSD settings. These gaps highlight the need for research that addresses scalability, efficiency, and more effective use of LLMs for robust and generalizable disambiguation. In particular, incorporating proper inner reasoning to guide decision making is crucial, motivating the present study on reasoning-driven WSD with low-

[1] https://babelnet.org/

parameter LLMs. Inspired by these observations, we design and evaluate a smaller, energy-efficient LLMs for WSD with three primary design strategies:

1. Given a word in a sentence, the LLM must generate the correct definition.
2. Given a word in a sentence and possible senses, the model reasons using the neighbouring context closely related to the disambiguation process to identify the correct sense.
3. Given a word in a sentence and possible senses, the model reasons why a particular sense is correct and why other senses are not, to determine the correct meaning.

Following these design principles, we fine-tune energy-efficient, low-parameter open-source models with fewer than 4B parameters (Bai et al., 2024). Our study reveals that a chain-of-thought (CoT)-based reasoning process combined with neighbouring context analysis performs comparably to large models like GPT-4-Turbo, achieving high performance on unseen data. Furthermore, our advanced reasoning approach requires only 10% of the training data compared to the second approach, yet performed similarly, demonstrating its effectiveness. The main contributions of this work are:

- Building three reasoning datasets using a semi-automated approach and releasing them as open-source resources for future research.[2] [3]
- Proposing and implementing lightweight adapters following a novel EAD (Exploration, Analysis and Disambiguation) framework that can be efficiently used for WSD with small-scale models.
- Evaluating eight open-source small-parameter models with the proposed approaches and achieving superior performance compared to all medium-parameter models.

The rest of the paper is organized as follows: background and related work, which discusses the WSD evolution and related studies; methodology, which outlines the approach used to conduct this study; results and observations, followed by conclusions and future directions.

2. Background and Related Work

2.1. Evolution of WSD

WSD has a long history, evolving from rule-based methods to deep learning and LLM approaches. Early rule-based systems relied on hand-crafted grammatical and syntactic rules to exploit the local context of ambiguous words (Bowerman, 1978), but these methods were labor-intensive, domain-specific, and difficult to scale to complex languages (Palmer et al., 2006). Knowledge-based (KB) approaches emerged with resources such as WordNet, dictionaries, and thesauri incorporating algorithms such as Lesk to measure the sense overlap between gloss definitions and the surrounding context (Lesk, 1986). Advanced KB solutions such as random walks over extended lexical graphs including Extended WordNet (Agirre et al., 2014) and graph-based methods that combined word embeddings with contextual information were explored (Duarte et al., 2021). Exploiting synset definitions, hypernymy relations, and contextual features improved the accuracy (Kolte and Bhirud, 2009), with frameworks like the Synset Relation-Enhanced Framework (SREF) achieving state-of-the-art KB WSD (Wang and Wang, 2020). Filtering unnecessary semantic information (Kwon et al., 2021) and incorporating dependency parsing to extract more precise contextual knowledge (Meng, 2022) were also prominent KB innovations.

With annotated corpora, supervised Machine Learning became the most common research approach (Le and Shimazu, 2004; Al-Bayaty and Joshi, 2016; Gosal, 2015). These methods exploited features, namely target word morphology, Part of Speech (POS) tags, syntactic dependencies, and collocations. To achieve efficient WSD, supervised neural architectures incorporated richer lexical and gloss-based information to the training pipeline. Gloss-augmented neural networks jointly encoded glosses and context were examined (Luo et al., 2018). Other approaches to supervised WSD explored multiple-sense identification (Orlando et al., 2021), stacked BiLSTMs with attention (Laatar et al., 2023), and context-dependent modeling (Koppula et al., 2021).

There was a major shift in WSD research with the arrival of Transformer architectures (Vaswani, 2017), which enabled deep, attention-based contextual modeling. Transformer-based models such as BERT and GPT achieved state-of-the-art performance on WSD tasks (Huang et al., 2020), demonstrating strong generalization to new domains due to extensive pre-training on diverse corpora. For instance, SenseBERT augments BERT pre-training by requiring the model to predict masked words and their WordNet supersenses (Levine et al., 2019), while GlossBERT constructs context–gloss pairs and fine-tunes BERT (Huang et al., 2020). Moreover, (Barba et al., 2021b,a; Blevins and Zettlemoyer, 2020) reshaped the field with supervised approaches that enhance WSD performance.

Current research focuses on zero-shot and few-shot WSD with LLMs, applying in-context learning to leverage linguistic knowledge without extensive retraining (Basile et al., 2025; Yae et al.,

[2]Reasoning dataset: https://huggingface.co/datasets/deshanksuman/Reasoning_WSD_dataset

[3]Reasoning dataset for Verb: https://huggingface.co/datasets/deshanksuman/Advance_Reasoning_for_Verb_WSD

2025). These approaches combine large-scale pre-training, dynamic context modeling, and KB augmentation, representing the most powerful and versatile WSD techniques to date and thus forming the basis of this study's methods.

2.2. Large Language Models for WSD

Despite their remarkable performance across numerous NLP tasks, LLMs still face several challenges in WSD. While they excel at handling common vocabulary, recent research indicates that LLMs often misinterpret rare or domain-specific ambiguous terms, especially in cross-lingual scenarios (Cahyawijaya et al., 2024; Yae et al., 2024; Basile et al., 2025; Ortega-Martín et al., 2023; Meconi et al., 2025). For example, Cahyawijaya et al. (2024) highlighted persistent errors between languages, with a bias toward higher-resource languages. Furthermore, Yae et al. (2024) observed that model size strongly influences WSD accuracy, yet larger models demand significantly more computational resources, raising efficiency concerns. Although more parameter-efficient language models generally lack the rich contextual understanding of their larger counterparts, Basile et al. (2025) demonstrates that fine-tuning them for downstream WSD tasks substantially improves accuracy over base configurations. This approach offers the potential for energy-efficient, domain-specialized disambiguation.

Recent work has explored various strategies for leveraging LLMs in WSD. Sainz et al. (2023) reframed WSD as a textual entailment task, prompting LLMs to evaluate domain label suitability for a sentence containing an ambiguous word. This zero-shot method not only outperformed random guessing but also, in certain cases, matched or exceeded supervised WSD systems (Ortega-Martín et al., 2023). Similarly, Sumanathilaka et al. (2024a) investigated prompt engineering with in-context learning in GPT-3.5-Turbo and GPT-4, while subsequent work benchmarked the WSD performance of multiple LLMs, identifying DeepSeek R1 and GPT-4-mini as effective (Sumanathilaka et al., 2024b).

Few studies have examined the affect of functional variables in LLMs. Mainly, temperature tuning was shown to influence disambiguation accuracy (Sumanathilaka et al., 2025a; Li et al., 2025b). Model architecture is crucial, with Qorib et al. (2024) reporting that encoder-only models can outperform decoder-only designs for this task. Meanwhile, multilingual and translation-based WSD strategies continue to be explored (Kang et al., 2023; David et al., 2024; Ren et al., 2024; Abdel-Salam, 2024; Laba et al., 2023), reflecting the ongoing relevance even with LLMs, as its role has shifted from a standard pipeline component to a targeted research focus.

Table 1: Stats for fine-tuning Variants. R: Reason

Dataset	Size (K)	Input		Output	
		Max	Avg	Max	Avg
Direct sense	101	511	44.7	251	13.9
COT R	101	1915	226.1	1921	265.9
Advanced R	10	1865	212.6	2477	672.0
Verb R	4.5	1915	222.6	2607	764.4

3. Methodology

With the aim of achieving a reduced use of computational, memory, energy, and financial resources, we present the methodology employed in our work using low-parameter models (<4B parameters) for an efficient reasoning driven WSD task.

3.1. Dataset and Augmentation Process

This work employs the FEWS dataset as the basis for the experiments, which includes the sense tag list, training data, and test data (Blevins et al., 2021). FEWS was chosen because it contains less frequently used ambiguous words compared to the Semcor and Unified Evaluation Framework datasets (Raganato et al., 2017). Additionally, the distribution of training and test data, designed for evaluation in both few-shot and zero-shot settings, makes it well-suited for a fair and meaningful comparison (Goworek et al., 2025). The training set contains ≈101K samples, and each test set has 5,000 records for evaluation.

We augment the FEWS training data to support the fine-tuning process (Blevins et al., 2021). Our fine-tuning method is designed to elicit chain-of-thought (CoT) reasoning together with neighbour-word analysis to select the closest semantic candidate to determine word sense (see subsection 3.2.2). For advanced reasoning, we employ the *Virtuoso-Large*, an open source model from Arcee.ai[4] to generate rationales for correct sense assignments and for rejecting competing senses. The model was selected based on its strong empirical performance reported in Sumanathilaka et al. (2024b), where it ranked among the best-performing models, and due to the availability of open weights, ensuring transparency and reproducibility. The model was instructed to produce a structured rationale based on three factors: contextual analysis, justification to the correct sense, and reasoning for elimination of incorrect sense. The process used a human-in-the-loop approach to ensure accurate data augmentation. We conducted additional experiments to evaluate computational techniques for verb disambiguation. To enhance

[4] https://huggingface.co/arcee-ai/Virtuoso-Large

the dataset, we incorporated syntactic evidence into the reasoning chain in addition to the semantic evidence used in the previous process. Furthermore, we utilized an improved prompt to generate 4.5K annotated instances for training the verb model.

The first author systematically traced and supervised the entire data generation process, ensuring a human-in-the-loop validation framework to maintain annotation quality and reliability. In addition, a representative sample of the generated data was evaluated using an LLM-as-a-judge approach (Li et al., 2025a) (See Appendix Table 11), employing OpenAI GPT-4o and DeepSeek-V3 as independent evaluators. The evaluation yielded consistently high scores across all dimensions. DeepSeek-V3 presents a average scores of 4.906 (Contextual Analysis), 4.898 (Justification Accuracy), 4.938 (Elimination Completeness), and 4.968 (Coherence), while GPT-4o shows 4.974, 4.962, 4.964, and 4.966, respectively. These results indicate strong agreement between evaluators and confirm the high quality of the generated dataset. Table 1 presents the statistics of the training data.

3.2. Fine-tuning strategies

The fine-tuning of the models was designed to address multiple objectives, allowing the evaluation of different performance criteria. Specifically, we assessed eight models to investigate whether low-parameter LLMs can effectively learn fine-tuning for accurate sense identification in the presence of ambiguous words. This proposed framework involves sequential tasks: (i) identifying the correct word sense without reasoning, (ii) performing neighbour-word analysis to identify the correct sense, and (iii) applying advanced reasoning that incorporates contextual understanding, justification of the correct sense, and refutation of incorrect senses to improve output quality.

We propose a novel **EAD framework** for tasks (ii) and (iii) consisting of three phases: *Exploration (E)*, *Analyzing (A)*, and *Disambiguating (D)*. In the *Exploration* phase, the framework collects sense inventories associated with the ambiguous word, including its interpretations and potential synonyms. The *Analyzing* phase emphasizes reasoning, which involves neighbour word analysis for task (ii) and a deeper evaluation of correct versus incorrect sense interpretations for task (iii). Finally, the *Disambiguating* phase consolidates the outcomes of the reasoning process to determine and retrieve the finalized sense ID. The models selected for the study include Gemma-2-2B (Team et al., 2024), Gemma-3-4B (Team et al., 2025), LLaMA-3.2-1B and LLaMA-3.2-3B (Dubey et al., 2024), Qwen-2.5-3B (Yang et al., 2025b), Qwen-3-4B (Yang et al., 2025a), SmoLM-3-3B and DeepSeek-Distill-Qwen1.5B (Liu et al., 2024). A systematic evaluation of these models was conducted to identify the best candidate to next phase.

3.2.1. Direct Sense Identification

To implement the first design objective "Given a word in a sentence, the LLM must generate the correct definition", we conducted a supervised evaluation of selected pre-trained language models on their ability to disambiguate the correct meaning of ambiguous words. For this phase, we utilized the FEWS dataset, which contains sentences annotated with target words and their corresponding senses. For instance, one training example includes the sentence: "He banked the plane sharply to avoid the storm," where the word "bank" is annotated with the sense "to tilt or incline an aircraft." The examples were formatted into instruction-response pairs, where the system prompt specifies the task, the input question provides the sentence and target word, and the output corresponds to the correct sense.

The pre-trained models were first evaluated via inference on these inputs using recommended hyperparameters to assess baseline disambiguation performance. Subsequently, we fine-tuned the baseline models using a supervised instruction fine-tuning approach, following the system prompt-input-output framework described in Subsection 3.3. Model performance was quantified using BERTScore, precision, recall, and F1 metrics to measure semantic alignment between predicted and reference senses. Initial experiments demonstrated that the Qwen-2.5 3B model exhibited promising learning capabilities after the initial training phase. To further investigate the impact of hyperparameters, additional experiments were conducted by varying the number of epochs and expanding the training dataset with additional SemCor records. These experiments allowed us to identify optimal hyperparameter configurations, which informed the subsequent phases of the study and ensured improved model generalization on the WSD.

3.2.2. Neighbour Words Analysis

The second design objective of "Given a word in a sentence and possible senses, the model reasons on the neighbouring context closely related to the disambiguation process to identify the correct sense" was developed in this phase. Inspired by Guzman-Olivares et al. (2025), we employed the fine-tuning design process following an EAD framework, giving full attention to the neighbouring words and following a CoT reasoning process for sense disambiguation. For the dataset creation, we implemented a context-extraction module using a windowed semantic similarity approach. Specifically, each sentence was pre-processed to mark the ambiguous word using <WSD> tags, and spaCy was used to tokenize the preceding and following segments while filtering out stopwords and non-alphabetic tokens. From this, a fixed-size context window of up to 10 tokens on each side was extracted. The target word and its context tokens were then embedded using a sentence-transformer model, and cosine similarity scores were computed between the target embedding and each context token embedding. This allowed the contextual tokens to be ranked by semantic closeness to the target, from which the top-k (default k=5) most semantically relevant words were retained as features. While cosine similarity does not explicitly model syntactic dependencies, it serves as a salience-based heuristic to identify lexically influential neighbouring tokens within a constrained window. Importantly, the full sentence is retained during model fine-tuning, allowing the transformer architecture to capture long-range syntactic relations beyond the similarity-based feature selection. These context-sense pairs were then integrated into the dataset, ensuring that the training data explicitly encoded the most influential neighbouring words for disambiguation.

To exemplify the process, the sentence "After the match, the <WSD>bat</WSD> was placed carefully back into the player's bag" contains the target ambiguous word 'bat' which has 12 distinct noun senses according to the FEWS sense inventory. The preceding context tokens ['After', 'match'] and following context tokens ['placed', 'carefully', 'back', 'player', 'bag'] were extracted, and similarity scores were computed with 'bat'. The tokens were then ranked by their similarity values, and the top-k most semantically related neighbours were selected. In this example, the high similarity of 'match', 'player', and 'bag' to 'bat' strongly indicates the sports equipment sense, as opposed to the flying mammal or old woman senses. These ranked neighbours serve as semantically aligned cues from the local context, guiding the disambiguation process toward the correct interpretation.

We applied this process to annotate 101K FEWS and 226K SemCor data records, structuring the output in CoT reasoning, then used this dataset to fine-tune the models. Our experiments revealed that this approach was more efficient than standard fine-tuning, yielding promising results for disambiguation. However, we observed that LLaMA 1B, LLaMA 3B, and DeepSeek-distill-Qwen 1.5B did not acquire the expected level of reasoning capability. The other models demonstrated strong reasoning performance, showing robust generalization and effective disambiguation even for unseen sentences.

3.2.3. Advanced Reasoning

This phase was designed to achieve the goal: "Given a word in a sentence and its possible senses, the model reasons why a particular sense is correct and why the others are not," to determine the correct meaning. The main objective of this process is to disambiguate meaning through a deep analysis of each sense in relation to the sentence requiring disambiguation. Due to the lack of an appropriate reasoning-specific dataset, we used a large-parameter open source LLM with a carefully designed prompt. We used a dataset of 10K samples which were randomly selected from the FEWS dataset to ensure coverage across all POS tags.

We discovered that three elements are critical to effective reasoning in sense disambiguation: (i) contextual analysis, (ii) justification of the correct sense, and (iii) systematic elimination of incorrect senses. Our design was inspired by Huang et al. (2020), who reported the importance of using context-gloss pairs to train models not only to identify why a sense ID is valid but also to reject invalid ones. Adopting the setup described in section 3.3, we fine-tuned the selected model and achieved comparable results despite the small sample size, demonstrating improved reasoning performance for disambiguation. This process aligns with the EAD framework, where exploration gathers candidate senses, analysis provides reasoning for sense selection, and disambiguation yields the final sense ID.

After the initial reasoning process, we observed limitations in verb disambiguation. To address this challenge, we incorporated syntactic evidence into the reasoning process, where a verb's morphosyntax (tense, aspect, voice), immediate dependents (subjects, objects, complements), key function words (auxiliaries, particles, prepositions), and relevant dependency or constituent patterns were explicitly included in the prompt to constrain its meaning. To evaluate the effect of these modifications, we trained and tested Qwen-3 and Gemma-3 models exclusively on newly constructed training data tailored for this purpose. The evaluation was conducted using the FEWS Few-shot Development set with verb subset, consisting of 2.2K samples, ensuring the assessment was focused on verb disambiguation. Moreover, we employed a hybrid training approach that combined reasoning with neighbour, syntactic, and semantic analysis, and the results were recorded to validate the improvements in learning and disambiguation accuracy.

3.2.4. Ablation Study

The experimental findings showed that reasoning-based approaches are well-suited for handling unseen data. To evaluate these findings further, we examine the generalizability of such models to different datasets without any additional fine-tuning.

Thus, we employed the recently released “Fool Me If You Can” dataset (Ballout et al., 2024), an adversarial benchmark designed to investigate the robustness of language models in WSD. We selected this dataset for the ablation study due to the varied contextual challenges. Specifically, we use the FEWS sense-mapped version retrieved from prior work (Sumanathilaka et al., 2025b) [5]. The dataset consists of four subsets: a baseline with context-appropriate meanings similar to training data, for standard WSD evaluation; a version adding adjectives that reinforce intended meanings; another replacing these with adjectives linked to the opposite sense; and newly crafted natural sentences where context contradicts the expected meaning, creating harder disambiguation scenarios. Fine-tuning on the “Fool Me If You Can” training data was intentionally avoided to ensure that the evaluation measured the robustness and generalization capabilities of the current reasoning models when exposed to unseen data from a different dataset.

To further probe these capabilities, we also evaluated the models on two additional challenging benchmarks: hardEn (Maru et al., 2022), which comprises 476 test cases that cannot be solved by existing state-of-the-art WSD models, and 42D, a specialized dataset containing 370 records with rare and domain-specific senses as defined by BabelNet (Navigli and Ponzetto, 2010).

This design choice allows us to assess whether the models can maintain disambiguation performance under domain shift and adversarial context, rather than benefiting from dataset-specific adaptation (see Section 4.2 for results).

3.3. Study Setup

For model development, supervised fine-tuning (SFT) was employed to adapt a pre-trained LLM to WSD task. The baseline models were fine-tuned using Low-Rank Adaptation (LoRA) to enable efficient training while reducing computational overhead. Training data were pre-processed into a desired chat-style prompt–response format, with each example containing a context sentence and the ambiguous target word to be disambiguated.

Fine-tuning was conducted using Hugging Face transformers and trl libraries, with a custom prompt formatting function to standardize inputs. The datasets were tokenized using the model’s native tokenizer, and training hyperparameters: batch size $= 4$, gradient accumulation $= 8$ steps, learning rate $= 2 \times 10^{-4}$. We used AdamW optimizer and linear learning rate scheduler with a random seed of 3407 as recommended by prior work (Picard, 2023). The fine-tuning used standard supervised causal language modelling, with cross-entropy loss backpropagated through the gold output tokens.

All experiments were performed on an NVIDIA A100-PCIE-40GB GPU, ensuring sufficient memory bandwidth for efficient fine-tuning of the model without quantization in the final setup. The resulting LoRA adapters and tokenizer were saved locally and uploaded to the Hugging Face Hub for reproducibility and deployment, which can be accessed at `https://huggingface.co/deshanksuman`.

4. Results and Discussion

In the initial phase, we evaluated eight different low-parameter LLMs to assess their performance in disambiguating a word within a given sentence. Inspired by Sumanathilaka et al. (2024a), we evaluate using a subset of the FEWS test dataset containing 1,050 cases distributed across nouns, verbs, and adjectives in a 4:3:3 ratio and with 50 adverb examples, to ensure coverage across parts-of-speech.

Performance was measured using BERTScore, computed with the distilled base uncased models. BERTScore was chosen because the evaluation focused on comparing the semantic similarity between the expected meaning and the model’s predicted output, rather than exact token matching. As the initial experiments were not sense-mapped, a meaning-based metric provided a more informative estimate of model performance.

We applied an SFT approach without explicit reasoning steps to examine how well the models could learn the senses of ambiguous words. Fine-tuning revealed clear improvements in sense identification and disambiguation for the Qwen and LLaMA models (see Table 4). Gemma-3-4B showed a performance drop in the sense identification setting. However, this was not due to poor disambiguation; rather, the model tended to generate longer, more detailed sense explanations rather than concise labels. Because evaluation relied on BERTScore against short reference descriptions, the resulting outputs were more verbose, leading to lower similarity scores. Notably, this issue did not appear in the sense ID prediction setting with predefined candidates.

Given the better performance of Qwen-2.5-3B, we further examined the effect of the number of training epochs. We found that increasing the number of epochs from one to five did not yield significant improvements in performance for F1 score (1:0.568, 3: 0.552, 5:0.571). McNemar tests (McNemar, 1947) did not reveal any statistically significant differences. Thus, we limited fine-tuning to two epochs to avoid unnecessary computation overhead.

[5] https://github.com/Sumanathilaka/FOOL-ME-IF-YOU-CAN-dataset-Meets-FEWS-sense-Tags

Table 2: Bert Score based on distilled base uncased against the base model vs finetuned model. The best scores are in bold. * Deepseek-distill-Qwen 1.5B

Models	Base Inference			Instructional Fine-Tuning		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score
Gemma 2 2B	0.7720	0.7341	0.7515	0.7870	0.7781	0.7814
Gemma 3 4B	**0.7806**	0.7302	0.7534	0.7741	0.7214	0.7458
Llama 3.2 1B	0.6439	0.7503	0.6923	0.7919	0.7474	0.7681
Llama 3.2 3B	0.7616	0.7330	0.7430	**0.8079**	0.7567	0.7804
SmolLM 3	0.7687	**0.7656**	**0.7662**	0.7884	0.7636	0.7749
Qwen 2.5 3B	0.7551	0.6773	0.7130	0.8126	**0.8018**	**0.8061**
Qwen 3 4B	0.7619	0.7470	0.7531	**0.8128**	0.7970	0.8036
Deepseek 1.5B*	0.7248	0.7193	0.7212	0.7140	0.7491	0.7294

4.1. Reasoning Experiments

In the initial experiments of the second phase, we fine-tuned the eight models to evaluate their ability to learn reasoning using neighbouring word analysis. All models were initially trained for 1 epoch to assess performance and selected models were extended to 2 epochs based on performance. Notably, Llama 1B, 3B, and Deepseek-distill-Qwen 1.5B did not learn reasoning effectively and behaved primarily as text predictors, failing to follow the instructions given during inference.

As presented in Table 3, fine-tuning significantly improves performance over the baseline. For instance, Qwen-3-4B and Gemma-3-4B models showed substantial gains after 2 epochs, achieving the highest overall scores of 0.738 and 0.752, respectively. Among the parts of speech, Gemma-3-4B achieved the highest score in Adverbs (0.80) after 1 epoch, while Qwen-3-4B showed the best improvement in Adjectives (0.75) after 2 epochs. These observations indicate that fine-tuning leads to consistent improvements across all categories, confirming that small models with reasoning-based fine-tuning benefit more from extended training compared to baselines.

To evaluate the effectiveness of different reasoning strategies, we benchmark the CoT reasoning-based approach against the few-shot reasoning based approach and the advanced reasoning with correct and incorrect sense analysis. For the few-shot reasoning based approach, we employed the same fine-tuned models; however, during inference, we provided additional few-shot examples to further emphasize understanding of the target senses, in line with evidence that supplementary contextual examples could improve performance (Yang et al., 2024). Each sense was accompanied by two illustrative examples that demonstrated its meaning and interpretation within a specific context. For evaluation, the dynamic few-shot examples were retrieved from a pre-constructed knowledge base.

Table 4 indicates that the zero-shot CoT reasoning approach generally outperforms the few-shot reasoning approach in terms of overall accuracy, particularly for the Gemma-3-4B model fine-tuned over two epochs (0.75). Interestingly, this zero-shot CoT configuration also achieves the highest noun (0.81) and verb (0.71) scores across all tested settings. While the few-shot approach provides additional contextual cues through example cases, it does not consistently translate into higher performance, suggesting that the models are already able to leverage their fine-tuned reasoning capabilities without supplementary examples. The advanced reasoning with correct and incorrect sense analysis approach performs competitively, with Qwen-3-4B reaching strong adverb performance (0.80) and stable results across categories, but still not surpassing the zero-shot CoT reasoning in overall accuracy. Importantly, the advanced reasoning approach used only 10% of the training data compared to the CoT reasoning approach, yet produced comparable results. This trend underscores the robustness of the CoT reasoning paradigm, even in the absence of explicit few-shot demonstrations, and highlights its ability to generalize reasoning patterns effectively across different POS categories.

To gain a broader understanding of the performance of the proposed reasoning-based approach compared to low-parameter models, we evaluated on the FEWS test set, which contains both few-shot and zero-shot datasets. Notably, the few-shot setting included examples whose senses do appear in the training set, but only in limited numbers, whereas the zero-shot setting contained examples of senses not encountered during training. We benchmarked our results against current state-of-the-art models to assess the effectiveness of our approach. Thus, we compared against MFS, Lesk, SemEq (Yao et al., 2021), ESR (Song et al., 2021), RTWE (Zhang et al., 2023), and Gloss-GPT (Sumanathilaka et al., 2025b) (see Table 5).

The proposed models (Qwen & Gemma) show competitive performance despite having significantly fewer parameters than most competitor systems. With the Few-shot set, both models outperform traditional baselines (i.e. MFS & Lesk) by a considerable margin and achieve close to state-of-the-art results. Notably, Qwen achieves 76.52 and

Table 3: Fine Tuning with CoT based reasoning with Neighbour words analysis. The results are presented in the F1 score.

Models	Noun	Verb	Adjective	Adverb	Overall
Fine Tuned with CoT based Neighbour words analysis Approach					
SmolLM 3B (1 epoch)	0.52	0.35	0.50	0.54	0.47
Gemma-2 2B (1 epoch)	0.70	0.61	0.68	0.66	0.67
Qwen 2.5 3B (1 epoch)	0.71	0.64	0.71	0.74	0.69
Gemma 3 4B (1 epoch)	0.79	0.65	0.70	**0.80**	0.72
Qwen 3 4B (2 epochs)	0.79	0.67	**0.75**	0.68	0.74
Gemma 3 4B (2 epochs)	**0.81**	**0.71**	0.72	0.76	**0.75**
Current Baseline					
Qwen 3 4B	0.69	0.54	0.58	0.48	0.61
Gemma 3 4B	0.62	0.48	0.52	0.48	0.54
Gemma 7B	0.49	0.41	0.51	0.46	0.47
Mixtral 7B	0.43	0.32	0.46	0.42	0.41
Yi - 34B	0.65	0.51	0.57	0.52	0.58
GPT 4o-mini	0.37	0.30	0.31	0.32	0.33

Table 4: F1 Score for benchmarking Different Reasoning Strategies.

Models	Noun	Verb	Adjective	Adverb	Overall
Fine Tuned with COT based Neighbour words analysis Approach (Zero shot)					
Qwen 3 4B (2 epochs)	0.79	0.67	0.**75**	0.68	0.74**
Gemma 3 4B (2 epochs)	**0.81**	**0.71**	0.72	**0.76**	**0.75****
Fine Tuned with COT based Neighbour words analysis Approach (Few shot)					
Qwen 3 4B (2 epochs)	0.74	0.62	0.68	0.64	0.68
Gemma 3 4B (2 epochs)	**0.78**	**0.68**	**0.70**	**0.78**	**0.72**
Advanced Reasoning with Correct and Incorrect Sense Analysis					
Gemma 3 4B (2 epochs)	0.76	0.60	0.66	0.64	0.68
Qwen 2.5 3B (2 epochs)	0.74	0.61	0.68	0.66	0.68
Qwen 3 4B (2 epochs)	**0.76**	**0.67**	**0.70**	**0.80**	**0.72**

$^{***}p < 0.001$, $^{**}p < 0.01$, $^{*}p < 0.05$

Gemma 75.68, surpassing RTWE_L and approaching GlossGPT's performance. In the more challenging Zero-shot set, the models maintain strong accuracy (72.66 and 71.86), outperforming most baselines, showing robust generalization. The results show the effectiveness of our reasoning-based fine-tuning approach, even using compact models.

From Table 4, we observed that verb disambiguation remains challenging for all models. We explored different fine-tuning strategies to test model reasoning improvements (see setup in section 3.2.3). However, verb-specific models still struggle to disambiguate verbs using syntactic clues, even when fine-tuned or combined in a hybrid model that incorporates both reasoning and neighbour analysis with syntactic and semantic features. Table 6 shows that adding extra reasoning signals still fails due to being interpreted as noise.

4.2. Ablation Study

The ablation study aimed to evaluate the robustness and generalizability of the trained module by applying CoT based reasoning combined with Neighbour Words Analysis to new, unseen data of two diverse use cases.

Table 7 presents a detailed comparison of our proposed approach against several state-of-the-art WSD systems on the challenging hard WSD benchmarks introduced by Maru et al. (2022). Our method, leveraging neighbour word analysis combined with Finetuned Qwen and Gemma, demonstrates superior performance, with Qwen-4B achieving the highest F1 scores across both 42D and hardEN datasets. Notably, Qwen-4B attains an F1 score of 78.48 on the hardN benchmark, substantially outperforming existing models, while Gemma also shows strong competitive results.

Table 8 shows that the model adapts effectively to unseen data with the Fool dataset, maintaining comparable performance across Sets 1-3. This underscores the extent to which the model can handle novel scenarios and maintain robust reasoning capabilities. Importantly, in the more challenging Set 4, containing realistic sentences where contextual cues deliberately contradict the expected meaning of a homonym, the model reached state-of-the-art performance, even for the small LLM class. This demonstrates the model's capacity to address difficult disambiguation tasks that require nuanced contextual understanding. When compared to the

Table 5: F1 score for CoT based Neighbour words analysis against current techniques. FEWS test data used for evaluation. $_b$: Base, $_L$: Large, Our approach in italic (4B models)

Dataset	MFS	Lesk	$SemEq_b$	$SemEq_L$	ESR_b	ESR_L	$RTWE_L$	*Qwen*	*Gemma*	GlossGPT
Fewshot	51.5	40.9	80.1	82.3	77.8	83.4	78.4	*76.52*	*75.68*	**90.7**
Zeroshot	-	39.0	70.2	72.2	71.6	75.8	69.9	*72.66*	*71.86*	**79.5**

Table 6: F1 score of the VERB task compared using different reasoning methods. Hybrid:Neighbour Word Analysis with Syntactic & Semantic Analysis, A: Analysis

Model	Neighbour Word A.	Semantic A.	Syntactic & Semantic A.	Hybrid
Qwen3-4B	**0.679***	0.662	0.677	0.602
Gemma3-4B	**0.658****	0.601	0.628	0.544

$^{***}p < 0.001$, $^{**}p < 0.01$, $^{*}p < 0.05$

Table 7: Performance comparison (F1 score) against existing WSD models. The compared models are ARES (Scarlini et al., 2020),BEM (Blevins and Zettlemoyer, 2020), ESC (Barba et al., 2021a), EWS (Bevilacqua et al., 2020b), GEN (Bevilacqua et al., 2020a), SYN (Scozzafava et al., 2020)),CSC (Barba et al., 2021b) and SandWiCH (Guzman-Olivares et al., 2025). Our best-performing approach, Neighbour words analysis with Finetuned Qwen and Gemma, is reported in Italic.

Dataset	ARES	BEM	ESC	EWS	GEN	SYN	CSC	SandWiCH	*Gemma*	*Qwen*
42D	41.8	53.2	58.9	43.9	50.2	32.8	56.6	77.1	64.43	**78.48**
hardEN	0.0	0.0	0.0	0.0	0.0	0.0	7.35	53.4	40.71	**54.19**

Table 8: F1 score for “Fool me if you can” dataset for binary classification of sense ID. *Our approach is not fine-tuned with training data from “Fool me if you can” dataset.

Models	# Parameters	Set 1	Set 2	Set 3	Set 4
Roberta-base	125M	0.945	0.969	0.888	0.715
Bert-large	340M	0.970	0.978	0.874	0.689
T5-large	770M	0.984	0.987	0.896	0.691
FLAN-T5-large	780M	0.948	0.953	0.852	0.663
T5-xl	3B	**0.991**	0.992	**0.907**	0.710
FLAN-T5-xl	3B	0.955	0.958	0.881	0.718
Mixtral 7bx8	7B	0.987	**0.993**	0.820	0.714
Llama3 8B	8B	0.986	0.990	0.790	0.687
*Gemma 3 (Reasoning)**	4B	0.966	0.969	0.812	0.811
*Qwen 3 (Reasoning)**	4B	0.970	0.972	0.847	**0.852**

results reported in the original paper (Ballout et al., 2024), our approach not only outperformed GPT-3.5-Turbo but also revealed that the key driver of superior performance is not merely model size, but the inclusion of a well-structured reasoning process. This reinforces the importance of targeted reasoning strategies for achieving strong generalization and adaptability in language models. This is an important finding because it challenges the common assumption of larger sized models for improved performance (Yae et al., 2025). Instead, it reveals that carefully designed reasoning strategies like *CoT combined with Neighbour Words Analysis* can yield state-of-the-art results even with smaller models.

5. Conclusion and Future Work

Our study shows that low-parameter LLMs can achieve competitive and state-of-the-art performance in WSD when equipped with reasoning-based fine-tuning strategies. Through evaluations with eight <4B parameter models, we showed that CoT reasoning combined with neighbour-word analysis enables strong sense prediction even in zero-shot and domain-shift settings. Gemma-3-4B and Qwen-3-4B models consistently outperformed medium-parameter baselines and were comparable to larger models (e.g., GPT-3.5-Turbo), while maintaining robustness across adversarial datasets and hard WSD datasets.

Importantly, the proposed approaches were effective in modest computational settings, with the advanced reasoning strategy achieving competitive accuracy using only 10% of the training data compared to the CoT method. These findings support the fact that reasoning quality is a critical determinant of LLM-based WSD performance. Future work should extend this approach to multilingual settings, including low-resource languages, to assess its adaptability across linguistic contexts.

Limitations

The scope of this study is limited to eight models from different vendors, all of which have fewer than 4B parameters. While further exploration of mid-sized models may yield improved performance, the primary objective, demonstrating performance gains through enhanced reasoning has been successfully achieved and remains adaptable. The current study focuses exclusively on English WSD; however, the approach can be extended to multilingual or cross-lingual settings in future work. Due to computational constraints, training iterations were restricted to two epochs, and only a sample of the generated data was validated owing to limited resource availability. Nevertheless, the evaluated samples produced promising results consistent with the expectations for this study.

Acknowledgements

We acknowledge the support of the Super computing Wales project, which is part-funded by the European Regional Development Fund (ERDF) via Welsh Government. Hough's work is supported by the EPSRC grant EP/X009343/1 'FLUIDITY'.

Ethics consideration

This paper has been conducted in compliance with Swansea University's ethical standards.

Data & Code availability

The fine-tuned LoRA adapters used in this study are publicly available on Hugging Face[6], enabling reproducibility and further research. In addition, the full training, inference, and evaluation pipeline has been released as open-source code on GitHub[7].

Bibliographical References

Reem Abdel-Salam. 2024. rematchka at arabicnlu2024: Evaluating large language models for arabic word sense and location sense disambiguation. In *Proceedings of The Second Arabic Natural Language Processing Conference*, pages 383–392.

Eneko Agirre, Oier López De Lacalle, and Aitor Soroa. 2014. Random Walks for Knowledge-Based Word Sense Disambiguation. *Computational Linguistics*, 40(1):57–84.

Boshra F Zopon Al-Bayaty and Shashank Joshi. 2016. Comparative analysis between naïve bayes algorithm and decision tree to solve wsd using empirical approach. *Lecture Notes on Software Engineering*, 4(1):82.

Joseph Bamidele Awotunde. 2025. Word sense disambiguation in biomedical applications. In *Mining Biomedical Text, Images and Visual Features for Information Retrieval*, pages 587–605. Elsevier.

Guangji Bai, Zheng Chai, Chen Ling, Shiyu Wang, Jiaying Lu, Nan Zhang, Tingwei Shi, Ziyang Yu, Mengdan Zhu, Yifei Zhang, et al. 2024. Beyond efficiency: A systematic survey of resource-efficient large language models. *arXiv preprint arXiv:2401.00625*.

Mohamad Ballout, Anne Dedert, Nohayr Abdelmoneim, Ulf Krumnack, Gunther Heidemann, and Kai-Uwe Kühnberger. 2024. Fool me if you can! an adversarial dataset to investigate the robustness of lms in word sense disambiguation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 5042–5059.

Edoardo Barba, Tommaso Pasini, and Roberto Navigli. 2021a. ESC: Redesigning WSD with Extractive Sense Comprehension. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4661–4672, Online. Association for Computational Linguistics.

Edoardo Barba, Luigi Procopio, and Roberto Navigli. 2021b. ConSeC: Word Sense Disambiguation as Continuous Sense Comprehension. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 1492–1503, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Pierpaolo Basile, Lucia Siciliani, Elio Musacchio, and Giovanni Semeraro. 2025. Exploring the word sense disambiguation capabilities of large language models. *arXiv preprint arXiv:2503.08662*.

Michele Bevilacqua, Marco Maru, Roberto Navigli, et al. 2020a. Generationary or "how we went beyond word sense inventories and learned to gloss". In *Proceedings of the 2020 Conference*

[6] https://huggingface.co/deshanksuman

[7] https://github.com/Sumanathilaka/An-EAD-Reasoning-Framework-for-WSD-with-Low-Parameter-LLMs

on *Empirical Methods in Natural Language Processing (EMNLP)*, pages 7207–7221.

Michele Bevilacqua, Roberto Navigli, et al. 2020b. Breaking through the 80% glass ceiling: Raising the state of the art in word sense disambiguation by incorporating knowledge graph information. In *Proceedings of the conference-Association for Computational Linguistics. Meeting*, pages 2854–2864. Association for Computational Linguistics.

Terra Blevins, Mandar Joshi, and Luke Zettlemoyer. 2021. FEWS: Large-Scale, Low-Shot Word Sense Disambiguation with the Dictionary. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 455–465, Online. Association for Computational Linguistics.

Terra Blevins and Luke Zettlemoyer. 2020. Moving Down the Long Tail of Word Sense Disambiguation with Gloss Informed Bi-encoders. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1006–1017, Online. Association for Computational Linguistics.

Melissa Bowerman. 1978. The acquisition of word meaning: An investigation into some current conflicts. In *The development of communication*, pages 263–287. Wiley.

Samuel Cahyawijaya, Ruochen Zhang, Holy Lovenia, Jan Christian Blaise Cruz, Hiroki Nomoto, and Alham Fikri Aji. 2024. Thank you, stingray: Multilingual large language models can not (yet) disambiguate cross-lingual word sense. *arXiv preprint arXiv:2410.21573*.

Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology*, 15(3):1–45.

Robert David, Anna Kernerman, Ilan Kernerman, Nicolas Ferranti, and Assaf Siani. 2024. Multilingual word sense disambiguation for semantic annotations: Fusing knowledge graphs, lexical resources, and large language models. In *Proceedings of CEUR Workshop (RAGE-KG 2024)*, pages 16–22.

José Marcio Duarte, Samuel Sousa, Evangelos Milios, and Lilian Berton. 2021. Deep analysis of word sense disambiguation via semi-supervised learning and neural word representations. *Information Sciences*, 570:278–297.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407.

Gurinder Pal Singh Gosal. 2015. A naive bayes approach for word sense disambiguation. *International Journal*, 5(7).

Roksana Goworek, Harpal Karlcut, Muhammad Shezad, Nijaguna Darshana, Abhishek Mane, Syam Bondada, Raghav Sikka, Ulvi Mammadov, Rauf Allahverdiyev, Sriram Purighella, et al. 2025. Senwich: Sense-annotation of low-resource languages for wic using hybrid methods. *arXiv preprint arXiv:2505.23714*.

Daniel Guzman-Olivares, Lara Quijano-Sanchez, and Federico Liberatore. 2025. Sandwich: Semantical analysis of neighbours for disambiguating words in context ad hoc. *arXiv preprint arXiv:2503.05958*.

Luyao Huang, Chi Sun, Xipeng Qiu, and Xuanjing Huang. 2020. GlossBERT: BERT for Word Sense Disambiguation with Gloss Knowledge. ArXiv:1908.07245 [cs].

Haoqiang Kang, Terra Blevins, and Luke Zettlemoyer. 2023. Translate to disambiguate: Zero-shot multilingual word sense disambiguation with pretrained language models. *arXiv preprint arXiv:2304.13803*.

SG Kolte and SG Bhirud. 2009. Exploiting links in wordnet hierarchy for word sense disambiguation of nouns. In *Proceedings of the International Conference on Advances in Computing, Communication and Control*, pages 20–25.

Neeraja Koppula, K. Srinivasa Rao, and B. VeeraSekharReddy. 2021. Word Sense Disambiguation Using Context Dependent Methods. In *2021 5th International Conference on Trends in Electronics and Informatics (ICOEI)*, pages 1582–1590, Tirunelveli, India. IEEE.

Sunjae Kwon, Dongsuk Oh, and Youngjoong Ko. 2021. Word sense disambiguation based on context selection using knowledge-based word similarity. *Information Processing & Management*, 58(4):102551.

Rim Laatar, Chafik Aloulou, and Lamia Hadrich Belguith. 2023. Evaluation of Stacked Embeddings for Arabic Word Sense Disambiguation. *Computación y Sistemas*, 27(2).

Yurii Laba, Volodymyr Mudryi, Dmytro Chaplynskyi, Mariana Romanyshyn, and Oles Dobosevych.

2023. Contextual embeddings for ukrainian: A large language model approach to word sense disambiguation. In *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, pages 11–19.

Cuong Anh Le and Akira Shimazu. 2004. High wsd accuracy using naive bayesian classifier with rich features. In *Proceedings of the 18th Pacific Asia conference on language, information and computation*, pages 105–114.

Michael Lesk. 1986. Automatic sense disambiguation using machine readable dictionaries: how to tell a pine cone from an ice cream cone. In *Proceedings of the 5th annual international conference on Systems documentation*, pages 24–26.

Yoav Levine, Barak Lenz, Or Dagan, Ori Ram, Dan Padnos, Or Sharir, Shai Shalev-Shwartz, Amnon Shashua, and Yoav Shoham. 2019. Sensebert: Driving some sense into bert. *arXiv preprint arXiv:1908.05646*.

Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. 2025a. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2757–2791, Suzhou, China. Association for Computational Linguistics.

Lujun Li, Lama Sleem, Niccolo' Gentile, Geoffrey Nichil, and Radu State. 2025b. Exploring the impact of temperature on large language models: Hot or cold? *arXiv preprint arXiv:2506.07295*.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.

Fuli Luo, Tianyu Liu, Qiaolin Xia, Baobao Chang, and Zhifang Sui. 2018. Incorporating Glosses into Neural Word Sense Disambiguation. ArXiv:1805.08028 [cs].

Marco Maru, Simone Conia, Michele Bevilacqua, and Roberto Navigli. 2022. Nibbling at the Hard Core of Word Sense Disambiguation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4724–4737, Dublin, Ireland. Association for Computational Linguistics.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Domenico Meconi, Simone Stirpe, Federico Martelli, Leonardo Lavalle, and Roberto Navigli. 2025. Do large language models understand word senses? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33885–33904.

Fanqing Meng. 2022. Word sense disambiguation based on graph and knowledge base. In *4th EAI international conference on robotic sensor networks*, pages 31–41. Springer.

Roberto Navigli and Simone Paolo Ponzetto. 2010. Babelnet: Building a very large multilingual semantic network. In *Proceedings of the 48th annual meeting of the association for computational linguistics*, pages 216–225.

Riccardo Orlando, Simone Conia, Fabrizio Brignone, Francesco Cecconi, and Roberto Navigli. 2021. AMuSE-WSD: An All-in-one Multilingual System for Easy Word Sense Disambiguation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 298–307, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Miguel Ortega-Martín, Óscar García-Sierra, Alfonso Ardoiz, Jorge Álvarez, Juan Carlos Armenteros, and Adrián Alonso. 2023. Linguistic ambiguity analysis in ChatGPT. ArXiv:2302.06426 [cs].

Martha Palmer, Hwee Tou Ng, and Hoa Trang Dang. 2006. Evaluation of wsd systems. *Word Sense Disambiguation: Algorithms and Applications*, pages 75–106.

David Picard. 2023. Torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision.

Muhammad Qorib, Geonsik Moon, and Hwee Tou Ng. 2024. Are decoder-only language models better than encoder-only language models in understanding word meaning? In *Findings of the Association for Computational Linguistics ACL 2024*, pages 16339–16347.

Alessandro Raganato, Jose Camacho-Collados, and Roberto Navigli. 2017. Word Sense Disambiguation: A Unified Evaluation Framework and Empirical Comparison. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers*, pages 99–110, Valencia, Spain. Association for Computational Linguistics.

Zhengfei Ren, Annalina Caputo, and Gareth Jones. 2024. A few-shot learning approach for lexical semantic change detection using gpt-4. In *Proceedings of the 5th Workshop on Computational Approaches to Historical Language Change*, pages 187–192.

Oscar Sainz, Oier Lopez de Lacalle, Eneko Agirre, and German Rigau. 2023. What do language models know about word senses? zero-shot WSD with language models and domain inventories. In *Proceedings of the 12th Global Wordnet Conference*, pages 331–342, University of the Basque Country, Donostia - San Sebastian, Basque Country. Global Wordnet Association.

Bianca Scarlini, Tommaso Pasini, and Roberto Navigli. 2020. With More Contexts Comes Better Performance: Contextualized Sense Embeddings for All-Round Word SenseDisambiguation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP-2020)*. Association for Computational Linguistics.

Federico Scozzafava, Marco Maru, Fabrizio Brignone, Giovanni Torrisi, and Roberto Navigli. 2020. Personalized PageRank with syntagmatic information for multilingual word sense disambiguation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 37–46, Online. Association for Computational Linguistics.

Tajinder Singh, Madhu Kumari, and Daya Sagar Gupta. 2024. Context-based persuasion analysis of sentiment polarity disambiguation in social media text streams. *New Generation Computing*, 42(4):497–531.

Yang Song, Xin Cai Ong, Hwee Tou Ng, and Qian Lin. 2021. Improved Word Sense Disambiguation with Enhanced Sense Representations. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4311–4320, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Deshan Sumanathilaka, Nicholas Micallef, and Julian Hough. 2024a. Assessing gpt's potential for word sense disambiguation: A quantitative evaluation on prompt engineering techniques. In *2024 IEEE 15th Control and System Graduate Research Colloquium (ICSGRC)*, pages 204–209. IEEE.

Deshan Sumanathilaka, Nicholas Micallef, and Julian Hough. 2025a. Exploring the impact of temperature on large language models: A case study for classification task based on word sense disambiguation. In *2025 7th International Conference on Natural Language Processing (ICNLP)*, pages 178–182.

Deshan Sumanathilaka, Nicholas Micallef, and Julian Hough. 2025b. GlossGPT: GPT for Word Sense Disambiguation using Few-shot Chain-of-Thought Prompting. *Procedia Computer Science*.

Deshan Koshala Sumanathilaka, Nicholas Micallef, and Julian Hough. 2024b. Can LLMs assist with ambiguity? a quantitative evaluation of various large language models on word sense disambiguation. In *Proceedings of the First International Conference on Natural Language Processing and Artificial Intelligence for Cyber Security*, pages 97–108, Lancaster, UK. International Conference on Natural Language Processing and Artificial Intelligence for Cyber Security.

Deshan Koshala Sumanathilaka, Nicholas Micallef, and Julian Hough. 2025c. Prompt balance matters: Understanding how imbalanced few-shot learning affects multilingual sense disambiguation in LLMs. In *Proceedings of the Workshop on Beyond English: Natural Language Processing for all Languages in an Era of Large Language Models*, pages 7–15, Varna, Bulgaria. INCOMA Ltd., Shoumen, BULGARIA.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.

Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.

A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.

Ming Wang and Yinglin Wang. 2020. A Synset Relation-enhanced Framework with a Try-again Mechanism for Word Sense Disambiguation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6229–6240, Online. Association for Computational Linguistics.

Jung H Yae, Nolan C Skelly, Neil C Ranly, and Phillip M LaCasse. 2024. Leveraging large language models for word sense disambiguation. *Neural Computing and Applications*, pages 1–18.

Jung H Yae, Nolan C Skelly, Neil C Ranly, and Phillip M LaCasse. 2025. Leveraging large language models for word sense disambiguation. *Neural Computing and Applications*, 37(6):4093–4110.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025a. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, et al. 2025b. Qwen2. 5-1m technical report. *arXiv preprint arXiv:2501.15383*.

Changbing Yang, Garrett Nicolai, and Miikka Silfverberg. 2024. Multiple sources are better than one: Incorporating external knowledge in low-resource glossing. *arXiv preprint arXiv:2406.11085*.

Wenlin Yao, Xiaoman Pan, Lifeng Jin, Jianshu Chen, Dian Yu, and Dong Yu. 2021. Connect-the-Dots: Bridging Semantics between Words and Definitions via Aligning Word Sense Inventories. ArXiv:2110.14091 [cs].

Xuefeng Zhang, Richong Zhang, Xiaoyang Li, Fanshuang Kong, Junfan Chen, Samuel Mensah, and Yongyi Mao. 2023. Word Sense Disambiguation by Refining Target Word Embedding. In *Proceedings of the ACM Web Conference 2023*, pages 1405–1414, Austin TX USA. ACM.

6. Appendix

Table 9: Prompt used to elicit rationales for sense selection during fine-tuning.

You are a helpful assistant tasked with simulating the human thinking process for Word Sense Disambiguation (WSD). Given a sentence containing an ambiguous word, along with the expected sense ID, your goal is to generate a detailed and logical reasoning process as a human would.
Your response should include:

- Contextual Analysis – Explain how the surrounding context in the sentence helps determine the correct meaning of the ambiguous word.
- Justification of the Correct Sense ID – Provide a clear explanation of why the expected sense ID is appropriate in this context.
- Elimination of Incorrect Senses – Briefly describe why the other possible sense IDs do not fit the given context.

Be thorough, logical, and emulate how a human would naturally think through the disambiguation. Do not include any additional information or instructions outside of the reasoning process.

Table 10: Prompt used to elicit rationales for Verb sense selection during extended study.

You are a helpful assistant tasked with simulating the human thinking process for Word Sense Disambiguation (WSD).
Given a sentence containing an ambiguous word, along with the expected sense ID, your goal is to generate a detailed and logical reasoning process as a human would.
The given ambiguous word is a VERB.
Your response should include:

1. Syntactic Evidence — Summarize the verb's morphosyntax (tense/aspect/voice), immediate dependents (subject, objects, complements), key function words (auxiliaries, particles, prepositions), and any relevant dependency/constituent patterns that constrain its meaning.
2. Semantic Evidence — Describe selectional preferences and semantic roles implied by the verb, plausible paraphrases, collocations, and context/topic cues (entities, events) that support a specific sense.
3. Decision — State the chosen sense ID and give a clear justification that ties the syntactic and semantic cues to that sense.
4. Elimination of Alternatives — Briefly state why other possible sense IDs do not fit given the observed syntax and semantics.

Be thorough, logical, and emulate how a human would naturally think through the disambiguation. Do not include any additional information or instructions outside of the reasoning process.

Table 11: Prompt used for LLM-as-a-Judge evaluation of WSD reasoning quality

You are an expert evaluator assessing the quality of Word Sense Disambiguation (WSD) reasoning generated by an AI system.

Task Context:

- Original sentence, ambiguous word, and sense definitions: {input_text}
- Correct sense ID: {senseid}
- Generated Reasoning to evaluate: {reasoning}

Evaluation Instructions:
Evaluate the generated reasoning on the following four dimensions using a 1-5 scale:

1. **Contextual Analysis Quality**: Does the reasoning identify and explain relevant contextual clues effectively?
   - 5: Comprehensively identifies all relevant contextual clues and explains their disambiguation role
   - 4: Identifies most key contextual elements with clear explanations
   - 3: Identifies some context but misses important clues or lacks depth
   - 2: Minimal contextual analysis with superficial observations
   - 1: Incorrect or irrelevant contextual analysis
2. **Sense ID Justification Accuracy**: Is the justification for the correct sense logically sound and semantically accurate?
   - 5: Provides precise, logically sound justification with clear semantic connections
   - 4: Good justification with minor gaps in reasoning
   - 3: Adequate justification but lacks depth or contains minor logical errors
   - 2: Weak justification with significant logical gaps
   - 1: Incorrect or nonsensical justification
3. **Elimination Reasoning Completeness**: Does it systematically eliminate alternative senses with clear reasoning?
   - 5: Systematically eliminates all alternative senses with clear reasoning
   - 4: Eliminates most alternatives with good reasoning
   - 3: Partial elimination with some reasoning gaps
   - 2: Minimal elimination or weak reasoning
   - 1: No elimination or incorrect reasoning
4. **Overall Coherence and Human-likeness**: Does the reasoning flow naturally like human thinking?
   - 5: Reads like natural human reasoning, logically structured and comprehensive
   - 4: Good flow with minor artificiality
   - 3: Adequate but somewhat mechanical or repetitive
   - 2: Disjointed or overly formulaic
   - 1: Incoherent or completely artificial

Output Format (JSON):

```
{
  "contextual_analysis_score": 1-5,
  "justification_accuracy_score": 1-5,
  "elimination_completeness_score": 1-5,
  "coherence_score": 1-5
}
```

A Survey on Lexical Ambiguity Detection and Word Sense Disambiguation

Miuru Abeysiriwardana
School of Computing
Informatics Institute of Technology
Colombo 06, Sri Lanka
miuru.2019450@iit.ac.lk

Deshan Sumanathilaka
Department of Computer Science
Swansea University
Swansea, Wales, United Kingdom
deshankoshala@gmail.com

***Abstract*— This paper explores techniques that focus on understanding and resolving ambiguity in language within the field of natural language processing (NLP), highlighting the complexity of linguistic phenomena such as polysemy and homonymy and their implications for computational models. Focusing extensively on Word Sense Disambiguation (WSD), it outlines diverse approaches ranging from deep learning techniques to leveraging lexical resources and knowledge graphs like WordNet. The paper introduces cutting-edge methodologies like word sense extension (WSE) and neuromyotonic approaches, enhancing disambiguation accuracy by predicting new word senses. It examines specific applications in biomedical disambiguation and language-specific optimisation and discusses the significance of cognitive metaphors in discourse analysis. The research identifies persistent challenges in the field, such as the scarcity of sense-annotated corpora and the complexity of informal clinical texts. It concludes by suggesting future directions, including using large language models, visual WSD, and multilingual WSD systems, emphasising the ongoing evolution in addressing lexical complexities in NLP. This thinking perspective highlights the advancement in this field to enable computers to understand language more accurately.**

Keywords*— *Ambiguity, Natural Language Processing, Word Sense Disambiguation, Word Embedding, Word Sense Extension

I. Introduction

The field of Natural Language Processing (NLP) has seen groundbreaking improvements, particularly with the advent of deep learning and transformer-based models. These models, capable of learning complex patterns in language by training on large amounts of data, have greatly enhanced our ability to deal with the subtleties and complexities of language ambiguity [1]. Language ambiguity is a persistent challenge in NLP because human language can have multiple meanings depending on the context of a single word or phrase [1]. This is where the evolution of NLP comes into play, from rule-based systems, which relied on manually coded rules, to modern, sophisticated models. These modern models leverage machine learning algorithms to learn from large amounts of data, reflecting a variety of approaches. Each approach comes with its unique challenges and achievements. For instance, while transformer-based models have shown impressive performance on various tasks, and to train them effectively, a substantial amount of data and computational resources are required, which can be prohibitive for many applications. Despite these advancements, language ambiguity remains a complex problem, driving ongoing research and development in NLP technologies. This ongoing research and development in NLP are focusing more on the refinement of these models, targeting the creation of more intelligent, context-aware, and reliable systems, with promising results in enhancing the reasoning capabilities [2], marking a significant step in Artificial intelligence (AI) with the promise of more natural human-computer interactions in the years to come. As we make further progress in enhancing and perfecting these models, we can anticipate even more remarkable breakthroughs in NLP.

A. *Language Ambiguity*

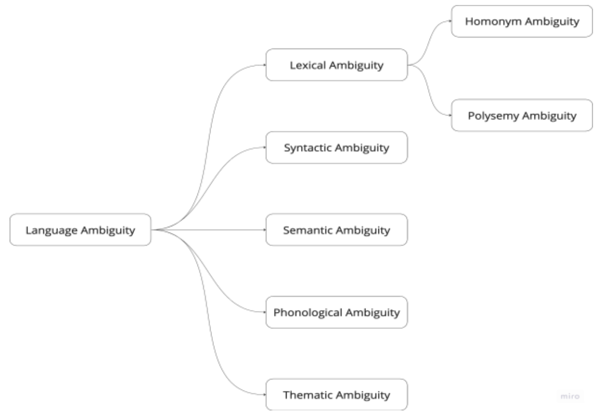

Fig. 1. Types of Ambiguity [1]

The language is a complex and constantly evolving system, frequently including words with multiple meanings. This poses significant challenges in the field of natural language processing (NLP), where the primary target is to facilitate computers in comprehending and interpreting human language with precision. Fig. 1 explores the broader topic of language ambiguity, which serves as a classification system for the complexities of language that can present challenges in communication and understanding. It is particularly relevant in the field of natural language processing, where disambiguation is a critical task. The major problem this paper intends to tackle is lexical ambiguity, where a given word can have various interpretations, making it especially difficult for computational models to process and understand the language effectively.

B. *Lexical Ambiguity*

In Lexical ambiguity, homonymy, refers to a string of sounds or characters corresponding to more than one word and/or meaning[3]. Lexical ambiguity can be confusing in a sentence, as a word can have multiple meanings. For example, the word “green” in the sentence “That horse is green” can mean untrained or the color green [3]. Lexical ambiguity and syntactic ambiguity are two distinct concepts. Lexical ambiguity refers to the presence of multiple meanings for a single word or phrase. On the other hand, syntactic ambiguity occurs when a sentence or phrase can be interpreted in multiple ways due to its structure or grammar

[3]. For instance, take the more popular sentence, “I saw the man with a telescope.” This sentence can be interpreted in two distinct ways: it might suggest that the narrator observed a man who was holding a telescope, or it could imply that the narrator used a telescope to see a man. This example clearly illustrates syntactic ambiguity, where the sentence structure leads to multiple interpretations. However, it's important to note that this differs from lexical ambiguity, which involves ambiguity in the meaning of individual words or phrases, not the overall sentence structure [1].

Polysemy refers to a single word with multiple meanings [4], [5]. The multiple meanings are listed under one entry in a dictionary [4]. An example of polysemy is the word ‘dish’ [6]. It can be seen in dictionaries that the word ‘dish’ has multiple definitions or polysemous meanings. The entry, “It’s your turn to wash the dishes,” refers to a kind of plate [4]. In terminological contexts, homonymy refers to the phenomenon where a single form represents different yet interconnected concepts [7]. This is distinct from homonymy in broader literary language, where it typically manifests as varied polysemantic formations lacking shared semantic elements [7].

The writer in [7] categorises terminological homonymy into three distinct forms: brunch, intersystem, and inter-scientific [7]. These categories highlight how a single term can possess multiple interpretations in different contexts or frameworks [7]. Take, for instance, the term “bank” in English, which is an example of a homonym. Depending on the situation, “bank” might refer to a financial establishment for depositing or borrowing money, or it could mean the area adjacent to a water body [6]. The same word, “bank”, represents these two unrelated ideas, thereby qualifying as a homonym [7]. In NLP, detecting and resolving such lexical ambiguities are vital for enhancing the effectiveness of various technologies, including machine translation, information retrieval, sentiment analysis, and conversational AI [1]. The capacity of these systems to accurately interpret context and clarify word meanings is fundamental to their precision and dependability [8].

C. *Word Sense Disambiguation*

Determining the precise interpretation of words in context, Word Sense Disambiguation (WSD) stands as a central obstacle in the realm of Natural Language Processing. It delves beyond simply recognising ambiguous meanings, instead pinpointing and applying the appropriate sense of a word within its usage [8]. Consider the term "mouse" in a sentence like "A mouse involves an item operated by hand, equipped with one or more switches" – here, it would be inferred to mean the computer accessory [9]. While multiple methodologies have been examined within this domain, notably for English, the progress of WSD studies for South Asian tongues, such as Urdu, is still in a nascent phase [9].

Recently, various deep learning techniques have been applied with notable success to tasks in Natural Language Processing. A particular study scrutinised multiple deep learning strategies, such as basic recurrent neural networks, long short-term memory networks, gated recurrent units, bidirectional short-term memory, and ensemble learning approaches, for their effectiveness in Word Sense Disambiguation (WSD) for the Urdu language [8]. Using two standard corpora to gauge performance, this research revealed that these deep learning techniques surpassed previous achievements for the comprehensive WSD challenge in the Urdu language [8]. Additionally, a different method involved the integration of definitions from dictionaries to bolster the WSD function in neural language models [10]. This technique proved to be adaptable to various tasks with minimal need for extra parameters. Upon being assessed across 15 subsequent tasks, its performance was superior to other advanced methods on the SemEval and Senseval datasets and marked an improvement over the baseline results on the GLUE benchmark [10].

Neil Aspinall served as the Chief Executive of Apple Corps Ltd., a company distinct from Apple Inc., known for its technology products, and Apple Leisure Group, a travel and hospitality conglomerate and the fruit. Fig. 2. illustrates the process of assigning senses to the term ‘Apple,’ which is inherently ambiguous, with the most prominent interpretations including the companies and the common fruit. The primary objective of a Word Sense Disambiguation (WSD) system is to accurately determine the intended meaning of 'Apple' in each sentence by analysing surrounding contextual cues.

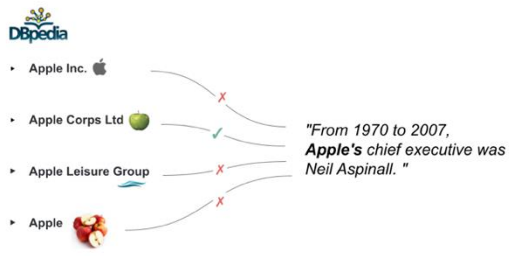

Fig. 2. Examples of WSD [11]

Fig 3 contains the functional WSD architecture proposed by [12].

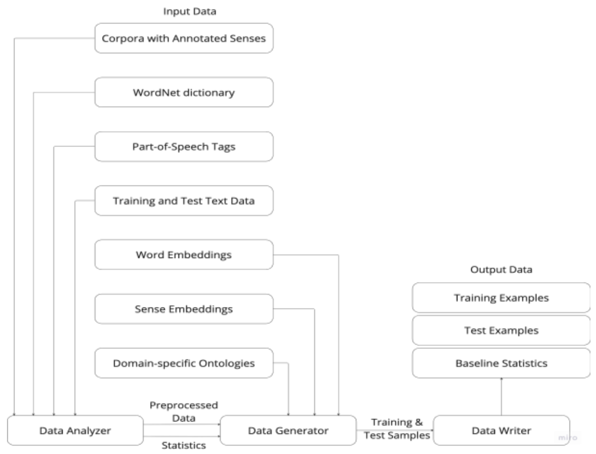

Fig. 3. WSD Functional Architecture [12]

The procedure begins with the system importing a dictionary derived from WordNet along with the training and testing textual materials into the Data Analyzer component. This component then transforms the imported data into a distinct intermediary structure and simultaneously computes elementary statistics pertinent to sentences and words [12]. These statistical analyses are instrumental in aiding users in determining the parameters needed to form training and test instances generated by the system. The Data Generator component, an essential part of the system, allows users to selectively extract diverse attributes from the training text data according to their set parameters. This component also produces training and test instances by amalgamating these attributes following the user's preferred sequence and structure. Following this, the created instances are transferred to the Data Writer component. This component is tasked with storing these instances as files for training and testing in a user-specified format and includes various baseline accuracy metrics to evaluate the WSD classifier [12]. These files are subsequently ready for use in external systems designed to learn and model classification patterns for the comprehensive all-words WSD task [12].

II. RELATED WORK

A. *Word Sense Extention*

A paradigm shift in NLP is observed in the concept of word sense extension (WSE). Instead of traditional disambiguation, this approach allows words to spawn new senses in novel contexts [13]. WSE creates a framework that blends cognitive models with learning schemes by dividing a word with multiple meanings into pseudo-tokens, each representing a different sense [13]. This method has demonstrated potential in forecasting likely new meanings for English words, particularly in transformer-based WSD models that handle infrequent word senses [13].

B. *Neurosymbolic Methodology for WSD Systems*

Another breakthrough in WSD is achieved through a novel neurosymbolic methodology [14], surpassing the 80% F1 score ceiling and reaching 90%. This methodology integrates supervised learning with symbolic reasoning, utilizing hypernym relations and pre-trained word embeddings [15]. Such neurosymbolic approaches have opened new vistas for full-fledged multilingual WSD systems [15]. A recent study in Scientific Reports introduces a novel technique for creating word sense embeddings [16], especially for words that are polysemous. This method employs a bidirectional long short-term memory neural network and a self-attention mechanism. [16], this method enhances the accuracy of word sense induction and the quality of sense embeddings [16]. This advancement significantly contributes to the representation of word senses in computational linguistics. These developments reflect the dynamic nature of research in lexical ambiguity detection and WSD. Incorporating novel methodologies like WSE and neurosymbolic approaches, along with advancements in word sense embeddings, highlights the ongoing evolution and increasing sophistication in handling the complexities of language in NLP. These contributions enhance the understanding of lexical ambiguity and pave the way for more accurate and contextually relevant NLP applications.

C. *Tailoring Contextual Embeddings for WSD*

In addition, there are multiple other exciting prospects in the field of WSD research. One notable example is the study by [17], which introduces a technique for WSD that involves tailoring contextual embeddings specifically for this task, relying exclusively on lexical information. The central concept of this method involves narrowing the semantic gap between related senses and contexts while distancing dissimilar or unrelated ones. This strategy has demonstrated superior results over past methods that modify contextual embeddings. Furthermore, it has reached a higher performance in knowledge-based WSD, particularly when combined with a reranking heuristic incorporating the sense inventory [17].

D. *Neurosymbolic Approaches for Reasoning within Graph Structures*

Additionally, the study by [14] offers an extensive review of neurosymbolic approaches for reasoning within graph structures. [14] introduces an innovative categorisation system that allows for the classification and discussion of the primary applications of these methods. They also suggest various potential paths for the future development of this emerging research area [14].

E. *AMuSE-WSD: A Multilingual System for WSD*

The "All-in-one Multilingual System for Easy Word Sense Disambiguation" (AMuSE-WSD) marks a notable breakthrough in the field of WSD [18]. Recently, WSD has garnered increased interest thanks to the impressive capabilities of deep learning methods in this area, particularly when enhanced by advanced pre-trained language models. Previously, there was a lack of comprehensive, user-friendly systems for leveraging these advancements, posing a challenge for researchers. AMuSE-WSD fills this void as the inaugural comprehensive system delivers top-tier sense information across 40 languages via a cutting-edge neural WSD model [18]. This innovation is pivotal for embedding semantic understanding into practical applications and spurring further research in lexical semantics. Before AMuSE-WSD, users looking to apply WSD in downstream tasks had to depend on existing systems mostly based on graph-centric heuristics or traditional machine-learning techniques. AMuSE-WSD revolutionises this by offering a leading-edge neural model for WSD [18]. By providing quality sense information in various languages, it is a significant resource for researchers and professionals in the field. This advancement underscores research's dynamic and evolving nature in detecting lexical ambiguity and advancing WSD, illustrating the growing complexity and refinement in processing language within Natural Language Processing (NLP). Such advancements are crucial in deepening the understanding of lexical ambiguity and set the stage for NLP applications that are more precise and context sensitive.

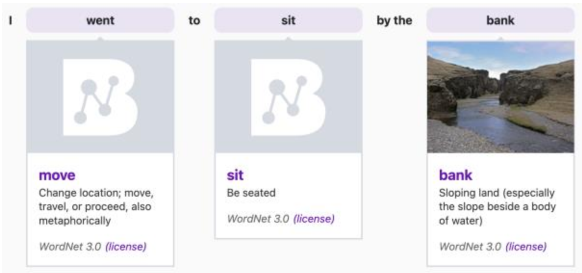

Fig. 4. Example 1 from AMuSE-WSD[18]

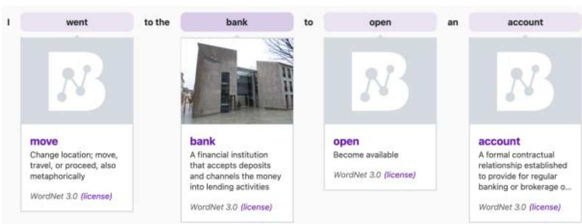

Fig. 5. Example 2 from AMuSE-WSD[18]

The sentences “I went to sit by the bank” and “I went to the bank to open an account” were used in the AMuSE-WSD system [18], as presented in Fig 4. And Fig 5. The system could accurately identify the intended reference of the word “bank” by considering the context. In the first sentence, “bank” refers to the edge of a river, while in the second, it refers to a financial institution. This demonstrates the system’s ability to understand the nuances of language and interpret words based on their contextual meaning. It’s a significant advancement in the field of NLP, highlighting the system’s effectiveness in handling language ambiguity.

III. FINDINGS AND ANALYSIS

Deep Learning for Urdu WSD [9]: The study explores a range of deep learning architectures to resolve the complexities of discerning word senses in Urdu. It examines a suite of techniques, including gated recurrent units, long-short-term memory networks, bidirectional long-short-term memory networks, simple recurrent neural networks, and ensemble learning strategies, as explained in Fig 6. where 4 deep learning methods are involved [9], for a selected ambiguous word, the 4 given models would assign the senses, and the final predicted sense is dependent upon a vote from derived outputs from these 4 models, and if they all predict different senses, then the final decision would be the sense assigned by the deep learning model with the highest accuracy. The study found that modern deep learning models are better at accurately determining word meanings in Urdu for the All-Words WSD task compared to earlier techniques [9].

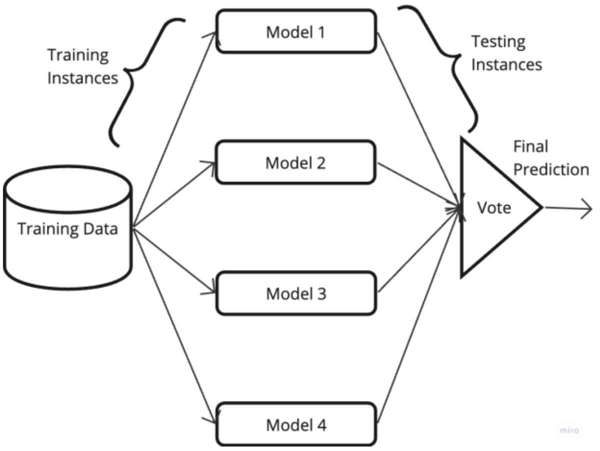

Fig. 6. URDU WSD Ensemble Learning Approach [9]

Biomedical Term Clarity Using Attention Neural Networks: This method introduces the use of attention neural networks to refine the precision in clarifying biomedical terminology [19]. It leverages the structure of words, grammatical roles, and contextual semantic cues from surrounding words to inform its features, achieving a notable average accuracy rate of 91.38% in distinguishing word senses [19]. Analysis of Metaphors in Cognition and Discourse: The study explores cognitive and discourse analysis of metaphors, examining the application of computer-aided tools such as Word Sense Disambiguation in detecting metaphorical language within discursive exchanges [20].

Employing the WordNet Knowledge Graph in WSD A novel approach employing the WordNet knowledge graph has been proposed to improve Word Sense Disambiguation (WSD) efforts. This method utilizes enhancements to the Lesk algorithm to better manage exact word matches and concise definitions, examines semantic similarity measures' relevance in WSD, and underscores three heuristic models: Most Frequent Sense, one sense per discourse, and one sense per collocation [21].

Optimizing Association Rules for WSD: The article presents a novel approach using an artificial immune system to refine association rules applicable to WSD [22]. It introduces a statistically normalized algorithm tailored for the Afaan Oromo language, designed to distinguish between meanings of ambiguous words independently of established rules or pre-labelled data. The algorithm exhibits encouraging outcomes, achieving an F-measure accuracy of 80.76% [22].

TABLE I. COMPARATIVE ASSESSMENT OF EXISTING WORKS

Approach	Goals	Established Tools	Limitations
Investigation of deep learning methods. [9]	To outperform previous results for the Urdu All-Words WSD task.	Ensemble Learning, Gated Recurrent Units, Bidirectional Long-Short Term Memory, Recurrent Neural Networks, Long-Short Term Memory.	data requirements and computational complexity
Attention neural network-based method. [19]	Improve disambiguation accuracy of biomedical words.	Morphology, part of speech, and semantic information features.	challenges in integrating diverse feature sets
Computer-assisted analysis of metaphors in discourse. [12].	Identify metaphorical features in discursive communication.	WSD algorithms.	detecting nuanced metaphorical usage.
Use of the WordNet knowledge graph. [13]	Novel approach for WSD	WordNet knowledge graph	limitations of WordNet's coverage and structure.
Artificial immune system for optimizing association rules.[22]	Perform WSD for the Afaan Oromo language without predefined rules or annotated datasets.	Normalized statistical algorithm.	Language-specific challenges and the generalizability of the method.

Table I. presents a comprehensive overview of diverse approaches in computational linguistics and natural language processing, each characterized by unique goals, methodologies, and challenges. These vary from deep learning techniques targeted to enhance Urdu alphabet recognition to innovative uses of neural networks for biomedical text disambiguation, from metaphor analysis in discourse to WordNet for word sense disambiguation (WSD). A recurrent theme across the above approaches includes the challenges and the subtleties of natural language.

TABLE II. MODEL AND DATASET COMPARISON

Approach	Model(s) Used	Dataset Used	Language	Accuracy	F1 Score
Investigation of deep learning methods. [9]	Ensemble of Simple RNN, LSTM, GRU, Bi-LSTM	ULS-WSD-18, UAW-WSD-18	Urdu	63.25% (UAW-WSD-18), 72.63% (ULS-WSD-18)	0.66
Resolving Amharic Lexical Ambiguity using Neural Word Embedding. [23]	joint supervised and unsupervised approach based on distributional semantic space for Amharic word sense disambiguation	Ambiguous Words, Amharic Annotated Corpora, Dictionary/Gloss	Amharic	70% (supervised) 60% (unsupervised) 86% (joint)	82.3% (supervised) 85.7% (unsupervised) 92.5% (joint)
Disambiguating spatial prepositions [24]	Bidirectional Encoder Representation from Transformer-based XLNet transformer	natural language expressions annotated for geo-spatial sense detection	English	0.96 precision	0.94
Artificial immune system for optimizing association rules.[22]	Normalized Statistical Algorithm	Afaan Oromo ambiguous words	Afaan Oromo	-	80.76%

Table II. contains the dataset and model comparison of four different WSD approaches. Each method showcases the application of advanced NLP techniques to disambiguate word meanings in various languages, targeting different levels of resource availability and linguistic complexity.

IV. CHALLENGES AND FUTURE DIRECTIONS

A. Challenges in Lexical Ambiguity Detection and WSD

A semantic similarity-based method for WSD is developed using training sets from resources like SemCor, OMSTI, and the Adaptive-Lexical Resource [25]. The latter is a training compilation for supervised learning that focuses on words with multiple meanings [25]. A major challenge presents itself with polysemous words, where their interpretation is heavily context dependent. Training, therefore, necessitates focusing on the contextual nuances of these ambiguous terms [25]. Another problem that arises is related to the evaluation of uncertainty [26], [27]. Uncertainty evaluation is critical in various machine learning deployments, especially when safety is paramount. Current approaches to assess the calibration of uncertainty in regression are inadequate, as they fail to effectively separate useful uncertainty predictions from those that are not [27]. Additionally, there is a scarcity of large, manually annotated sense corpora for WSD [28], [29]. For instance, SemCor, the most extensive manually annotated sense corpus available, tags words with 33,760 different senses, representing only about 16% of WordNet's sense inventory [28]. There are efforts to mitigate this limitation by developing new sense-annotated corpora through automatic or semi-automatic means [29].

Significant challenges are associated with the informalities of clinical texts [30], such as typographical errors, inconsistent and incomplete texts, and non-standardized texts. The difficulty with the lack of clinical resources, such as clinical sense inventories, and privacy concerns for conducting research with clinical text are also notable [30].

B. Future Directions

Utilisation of Large Language Models (LLMs) increases when enhancing phrases and clarifying ambiguities associated with specific words as knowledge bases [31]. An emerging area of interest lies in investigating cutting-edge transformer-based techniques for multimodal information retrieval [31]. The task of Visual Word Sense Disambiguation (VWSD) presents a unique challenge in selecting the most appropriate image from several options to accurately reflect the meaning of an ambiguous word in its context [32]. This task is currently being examined as a single-modal issue, transforming it into text-to-text, image-to-image retrieval, and question-answering formats to maximise the potential of the relevant models [32].

In the area of uncertainty estimation, future efforts are focused on developing new definitions that overcome existing shortcomings, alongside evaluating these using straightforward histogram-based methods [27]. Discussions are also underway regarding the practical constraints of existing methods in critical real-world scenarios, such as mission-critical and safety-sensitive applications [27]. Regarding Word Sense Disambiguation (WSD), there is a shift towards more refined results through adaptive word embeddings, particularly when these embeddings are crafted based on the context of the word [25].

The trend for Multilingual Word Sense Disambiguation (MWSD) is to create knowledge-driven and supervised MWSD systems [33]. Efforts are being made to develop unified sense representations that span multiple languages [33]. Additionally, the challenge of limited annotations in MWSD is being addressed by transferring knowledge from languages with abundant resources to those with fewer resources.

V. CONCLUSION

The study on Word Sense Disambiguation (WSD) in natural language processing discusses several challenges, including the complexity of training models for context-dependent polysemous words and the limited availability of sense-annotated corpora. Addressing these, new methods, such as adaptive word embedding and the development of new automated and semi-automated corpora, are emerging. The field is advancing with Large Language Models (LLMs) for phrase enhancement and ambiguity resolution and exploring novel areas like Visual Word Sense Disambiguation (VWSD) for image-based context interpretation. Future directions also include refining uncertainty estimation in machine learning, which is crucial for safety-critical applications, and expanding into Multilingual Word Sense Disambiguation, focusing on unified sense representations and knowledge transfer between languages. These evolving strategies show the continuous progression in tackling lexical complexities in NLP, enhancing the capability for more accurate, context-aware language processing.

References

[1] A. Yadav, A. Patel, and M. Shah, "A comprehensive review on resolving ambiguities in natural language processing," *AI Open*, vol. 2, pp. 85–92, 2021, doi: 10.1016/j.aiopen.2021.05.001.

[2] R. Hong, X. Pang, and C. Zhang, "Advances in Reasoning by Prompting Large Language Models: A Survey," *Cybern. Intell.*, Dec. 2023, doi: 10.26599/CAI.2024.9390004.

[3] "Lexical Ambiguity," obo. Accessed: Nov. 27, 2023. [Online]. Available: https://www.oxfordbibliographies.com/display/document/obo-9780199772810/obo-9780199772810-0309.xml

[4] "Polysemy: Definition, Meaning & Examples | Vaia," Hello Vaia. Accessed: Nov. 27, 2023. [Online]. Available: https://www.hellovaia.com/explanations/english/lexis-and-semantics/polysemy/

[5] A. Moldovan, "Descriptions and Tests for Polysemy," *Axiomathes*, vol. 31, no. 3, pp. 229–249, Jun. 2021, doi: 10.1007/s10516-019-09445-y.

[6] "What is Polysemy? – A Survey of Current Research and Results," *Pragmat. Flex. Word Mean.*, p. 175, 2001.

[7] V. D. Tabanakova, "Term 'Homonymy' As A Semantic Category," *Eur. Proc. Soc. Behav. Sci.*, vol. Language and Technology in the Interdisciplinary Paradigm, Feb. 2021, doi: 10.15405/epsbs.2021.12.25.

[8] P. Nanjundan and E. Z. Mathews, "An Analysis of Word Sense Disambiguation (WSD)," in *Proceedings of the International Health Informatics Conference*, S. Jain, S. Groppe, and N. Mihindukulasooriya, Eds., in Lecture Notes in Electrical Engineering. Singapore: Springer Nature, 2023, pp. 251–259. doi: 10.1007/978-981-19-9090-8_22.

[9] A. Saeed, R. M. A. Nawab, and M. Stevenson, "Investigating the Feasibility of Deep Learning Methods for Urdu Word Sense Disambiguation," *ACM Trans. Asian Low-Resour. Lang. Inf. Process.*, vol. 21, no. 2, p. 38:1-38:16, Oct. 2021, doi: 10.1145/3477578.

[10] A. Nesterov, L. Hollink, and J. van Ossenbruggen, "How Contentious Terms About People and Cultures are Used in Linked Open Data." arXiv, Nov. 13, 2023. Accessed: Nov. 26, 2023. [Online]. Available: http://arxiv.org/abs/2311.10757

[11] "About: Apple." Accessed: Nov. 29, 2023. [Online]. Available: https://dbpedia.org/page/Apple

[12] G. Agre, D. Petrov, and S. Keskinova, "Word Sense Disambiguation Studio: A Flexible System for WSD Feature Extraction," *Information*, vol. 10, p. 97, Mar. 2019, doi: 10.3390/info10030097.

[13] "Word Sense Extension," ar5iv. Accessed: Nov. 25, 2023. [Online]. Available: https://ar5iv.labs.arxiv.org/html/2306.05609

[14] L. N. DeLong, R. F. Mir, M. Whyte, Z. Ji, and J. D. Fleuriot, "Neurosymbolic AI for Reasoning on Graph Structures: A Survey." arXiv, Feb. 14, 2023. Accessed: Nov. 29, 2023. [Online]. Available: http://arxiv.org/abs/2302.07200

[15] "Word Sense Disambiguation as a Game of Neurosymbolic Darts," ar5iv. Accessed: Nov. 26, 2023. [Online]. Available: https://ar5iv.labs.arxiv.org/html/2307.16663

[16] Y. Sun and J. Platoš, "A method for constructing word sense embeddings based on word sense induction," *Sci. Rep.*, vol. 13, no. 1, Art. no. 1, Aug. 2023, doi: 10.1038/s41598-023-40062-3.

[17] S. Mizuki and N. Okazaki, "Semantic Specialization for Knowledge-based Word Sense Disambiguation." arXiv, Apr. 22, 2023. Accessed: Nov. 29, 2023. [Online]. Available: http://arxiv.org/abs/2304.11340

[18] R. Orlando, S. Conia, F. Brignone, F. Cecconi, and R. Navigli, "AMuSE-WSD: An All-in-one Multilingual System for Easy Word Sense Disambiguation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 298–307. doi: 10.18653/v1/2021.emnlp-demo.34.

[19] C.-X. Zhang, S.-Y. Pang, X.-Y. Gao, J.-Q. Lu, and B. Yu, "Attention Neural Network for Biomedical Word Sense Disambiguation," *Discrete Dyn. Nat. Soc.*, vol. 2022, pp. 1–14, Jan. 2022, doi: 10.1155/2022/6182058.

[20] O. Kalinin, "Cognitive Metaphor and Discourse: Research Methods and Paradigms," *Vestn. Volgogr. Gos. Univ. Ser. 2 Jazyk.*, no. 5, pp. 108–121, Jan. 2022, doi: 10.15688/jvolsu2.2021.5.9.

[21] M. AlMousa, R. Benlamri, and R. Khoury, "A novel word sense disambiguation approach using WordNet knowledge graph," *Comput. Speech Lang.*, vol. 74, p. 101337, Jul. 2022, doi: 10.1016/j.csl.2021.101337.

[22] Department of Information Technology, College of Computing and Informatics, Wolkite University, Ethiopia and A. A. Abafogi, "Normalized Statistical Algorithm for Afaan Oromo Word Sense Disambiguation," *Int. J. Intell. Syst. Appl.*, vol. 13, no. 6, pp. 40–50, Dec. 2021, doi: 10.5815/ijisa.2021.06.04.

[23] G. Demlew and D. Yohannes, "Resolving Amharic Lexical Ambiguity using Neural Word Embedding," in *2022 International Conference on Information and Communication Technology for Development for Africa (ICT4DA)*, Bahir Dar, Ethiopia: IEEE, Nov. 2022, pp. 55–60. doi: 10.1109/ICT4DA56482.2022.9971344.

[24] M. A. Radke, A. Gupta, K. Stock, and C. B. Jones, "Disambiguating spatial prepositions: The case of geo-spatial sense detection," *Trans. GIS*, vol. 26, no. 6, pp. 2621–2650, 2022, doi: 10.1111/tgis.12976.

[25] C. D. Kokane, S. D. Babar, P. N. Mahalle, and S. P. Patil, "Word Sense Disambiguation: Adaptive Word Embedding with Adaptive-Lexical Resource," in *Proceedings of International Conference on Data Analytics and Insights, ICDAI 2023*, N. Chaki, N. D. Roy, P. Debnath, and K. Saeed, Eds., in Lecture Notes in Networks and Systems. Singapore: Springer Nature, 2023, pp. 421–429. doi: 10.1007/978-981-99-3878-0_36.

[26] Z. Liu and Y. Liu, "Ambiguity Meets Uncertainty: Investigating Uncertainty Estimation for Word Sense Disambiguation," 2023, doi: 10.48550/ARXIV.2305.13119.

[27] D. Levi, L. Gispan, N. Giladi, and E. Fetaya, "Evaluating and Calibrating Uncertainty Prediction in Regression Tasks," *Sensors*, vol. 22, no. 15, Art. no. 15, Jan. 2022, doi: 10.3390/s22155540.

[28] B. Scarlini, T. Pasini, and R. Navigli, "Sense-Annotated Corpora for Word Sense Disambiguation in Multiple Languages and Domains," in *Proceedings of the Twelfth Language Resources and Evaluation Conference*, N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, A. Moreno, J. Odijk, and S. Piperidis, Eds., Marseille, France: European Language Resources Association, May 2020, pp. 5905–5911. Accessed: Nov. 07, 2023. [Online]. Available: https://aclanthology.org/2020.lrec-1.723

[29] L. Vial, B. Lecouteux, and D. Schwab, "Sense Vocabulary Compression through the Semantic Knowledge of WordNet for Neural Word Sense Disambiguation." arXiv, Aug. 27, 2019. Accessed: Nov. 28, 2023. [Online]. Available: http://arxiv.org/abs/1905.05677

[30] S. Moon, B. McInnes, and G. B. Melton, "Challenges and Practical Approaches with Word Sense Disambiguation of Acronyms and Abbreviations in the Clinical Domain," *Healthc. Inform. Res.*, vol. 21, no. 1, pp. 35–42, Jan. 2015, doi: 10.4258/hir.2015.21.1.35.

[31] A. Kritharoula, M. Lymperaiou, and G. Stamou, "Large Language Models and Multimodal Retrieval for Visual Word Sense Disambiguation." arXiv, Oct. 21, 2023. doi: 10.48550/arXiv.2310.14025.

[32] A. Raganato, I. Calixto, A. Ushio, J. Camacho-Collados, and M. T. Pilehvar, "SemEval-2023 Task 1: Visual Word Sense Disambiguation," in *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*, A. Kr. Ojha, A. S. Doğruöz, G. Da San Martino, H. Tayyar Madabushi, R. Kumar, and E. Sartori, Eds., Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 2227–2234. doi: 10.18653/v1/2023.semeval-1.308.

[33] Y. Su, H. Zhang, Y. Song, and T. Zhang, "Multilingual Word Sense Disambiguation with Unified Sense Representation." arXiv, Oct. 13, 2022. doi: 10.48550/arXiv.2210.07447.

Ambiguity Detection and Uncertainty Calibration for Question Answering with Large Language Models

Zhengyan Shi[1*] **Giuseppe Castellucci**[2] **Simone Filice**[3*] **Saar Kuzi**[2]
Elad Kravi[4*] **Eugene Agichtein**[2] **Oleg Rokhlenko**[2] **Shervin Malmasi**[2]
[1]University College London [2]Amazon [3]Technology Innovation Institute [4]Meta
`michaelszx117,filice.simone,eladkravi@gmail.com`
`giusecas,eugeneag,olegro,malmasi@amazon.com`

Abstract

Large Language Models (LLMs) have demonstrated excellent capabilities in Question Answering (QA) tasks, yet their ability to identify and address ambiguous questions remains underdeveloped. Ambiguities in user queries often lead to inaccurate or misleading answers, undermining user trust in these systems. Despite prior attempts using prompt-based methods, performance has largely been equivalent to random guessing, leaving a significant gap in effective ambiguity detection. To address this, we propose a novel framework for detecting ambiguous questions within LLM-based QA systems. We first prompt an LLM to generate multiple answers to a question, and then analyze them to infer the ambiguity. We propose to use a lightweight Random Forest model, trained on a bootstrapped and shuffled 6-shot examples dataset. Experimental results on ASQA, PACIFIC, and ABG-COQA datasets demonstrate the effectiveness of our approach, with accuracy up to 70.8%. Furthermore, our framework enhances the confidence calibration of LLM outputs, leading to more trustworthy QA systems that are able to handle complex questions.

1 Introduction

Recent advancements in Large Language Models (LLM) (Chung et al., 2022; Touvron et al., 2023; OpenAI, 2023) have significantly improved their capabilities in Question Answering (QA). However, users often ask under-specified questions that can have multiple interpretations (Min et al., 2020; Sun et al., 2023). Those ambiguities typically lead to inaccurate or misleading answers, which undermine the user trust in the systems (Ovalle et al., 2023). Identifying questions requiring clarification is thus a crucial task to build trustworthy NLP systems.

Recent studies (Cole et al., 2023; Deng et al., 2023) explored how LLMs can detect question ambiguity with prompting (*e.g.,* binary prompts where LLM answers with ‘`Yes`’ or ‘`No`’). These works found that the prompting strategy is ineffective and performs at random guessing levels.

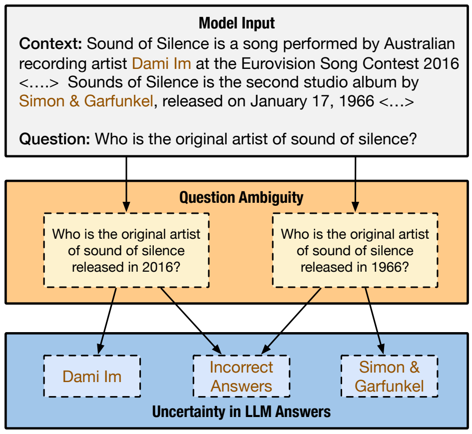

Figure 1: Ambiguity can either originate from the inherent ambiguity in the question (denotational uncertainty) or stem from the model's own indecision about potential answers (epistemic uncertainty).

In light of these findings, we propose to address the problem from a different angle by analyzing the responses of the LLM to the potentially ambiguous question. Intuitively, as illustrated in Figure 1, if an LLM provides multiple plausible responses, such as “*Dami Im*” and “*Simon & Garfunkel*” for the question “*Who is the original artist of Sound of Silence?*”, it can suggest ambiguity in the user question. Therefore, we hypothesize that understanding the variance of the LLM outputs can assist in detecting the ambiguity of questions.

A straightforward implementation would be to prompt the LLM to generate many possible answers to the question and then measure the entropy (*i.e.,* uncertainty) over the answers (Kuhn et al., 2023; Lin et al., 2023). The entropy can serve as a proxy for the question ambiguity: when the LLM insists on a single answer, the entropy will be 0 (indicating *non-ambiguity*); instead, if the LLM

*Work done while working at Amazon.

Proceedings of the 5th Workshop on Trustworthy NLP (TrustNLP 2025), pages 41–55

is confident about multiple answers, the resulting entropy would increase towards 1 (thus indicating *ambiguity*). However, LLMs often produce incorrect, incomplete, or misleading answers, due to a lack of specific knowledge, hallucination, or other underlying factors (Tian et al., 2023; Bang et al., 2023). In Figure 1, such LLMs' outputs, labeled as "*incorrect answers*", amplify the measured entropy. Therefore, a more refined interpretation model is necessary to discern the question ambiguity.

In this work, we propose a novel framework to detect ambiguity in questions in LLM-based QA systems in low-resource settings. As shown in Figure 2, our framework first prompts an LLM to generate multiple answers to a question given some contextual information, *i.e.,* supporting evidence in a retrieval-augmented setting (Lewis et al., 2020); we prompt the LLM through self-consistency prompting (Wang et al., 2022). Then, we use an interpreter model to analyze the answers with various distributional features of the LLM responses to infer the ambiguity. We found that a Random Forest (RF) model, trained on a diverse range of LLM output patterns simulated through *bootstrapping* based on a very few-shot example set, is capable of accurately identifying ambiguity in questions. This approach outperforms various baselines including self-interpretation by the LLM *itself*, a RoBERTa-based classifier, and different prompting strategies. In particular, we conduct experiments on the ASQA (Stelmakh et al., 2022), Pacific (Deng et al., 2022), and Abg-CoQA (Guo et al., 2021) datasets, and show that our proposed framework substantially improves the performance of the ambiguity detection task, with accuracy levels up to 70.8%; this is a substantial improvement over the existing prompt-based approaches, which barely surpass a random baseline. Our evaluation also shows that the prediction probabilities derived from the RF are reliable indicators of the model's accuracy, which effectively reduces the likelihood of providing incorrect or misleading answers, thus improving the trustworthiness of the resulting system. Our analysis also explores the benefits of bootstrapping few-shot examples and reveals that our approach delivers much fewer false positives, compared to the heuristic method using entropy.

In summary, the contributions of this work are: i) we introduce a novel framework for ambiguity detection in LLM-based QA systems by prompting the LLM to generate multiple answers which are then analyzed by an RF model, trained using bootstrapping; ii) experiments on the ASQA, Pacific, and Abg-CoQA datasets show that the proposed framework considerably enhances the performance of the ambiguity detection task; iii) our study reveals that prediction probabilities generated by the RF model are reliable indicators of the model's accuracy. This aspect is crucial as it minimizes the likelihood of providing incorrect responses, improving the reliability of the resulting QA systems.

2 Related Work

Ambiguous Question Answering and Clarification. Ambiguity is an element of human language, which has led to numerous studies including in instruction following (Shi et al., 2022a), conversational search (Keyvan and Huang, 2022; Aliannejadi et al., 2019), product search (Chen et al., 2023, 2024), and question answering (Shao and Huang, 2022; Sun et al., 2023; Lee et al., 2023; Ji et al., 2024; Zhang et al., 2024; Wu et al., 2024). Previous studies (Min et al., 2020; Shi et al., 2022b; Cole et al., 2023) emphasize the importance of grounding the ambiguity detection task within a relevant context, as the definition of "ambiguous" is inherently subjective. While the ClariQ dataset (Aliannejadi et al., 2021) is one of the pioneering datasets for query ambiguity, it does not offer a grounding context, leading to some inconsistent annotations (see Appendix §B). Similarly, AmbigQA (Min et al., 2020) and WebQuestionsSP (Yih et al., 2016) do not provide annotated context. In this research, we focus on a context-enhanced setting.

Uncertainty Estimation. Estimating uncertainty/confidence is crucial for assessing the reliability of LLMs (Gal and Ghahramani, 2016; Yang et al., 2024a; Geng et al., 2024; Zhou et al., 2023a). Ideally, a perfectly calibrated confidence estimation reflects the true likelihood of the prediction being correct (Niculescu-Mizil and Caruana, 2005; Guo et al., 2017). Earlier studies (Murray and Chiang, 2018; Malinin and Gales, 2020; Jiang et al., 2021) often used the token probability from the language model to calculate the marginal probability of a sequence and use it to estimate the model confidence. Recent works have raised the question of whether post-training (Ouyang et al., 2022; Wei et al., 2022a) might negatively impact model calibration (OpenAI, 2023). Many efforts have been made to calibrate uncertainty in LLMs. Kadavath et al. (2022) estimated the LLM confidence using the likelihood of the "True" token

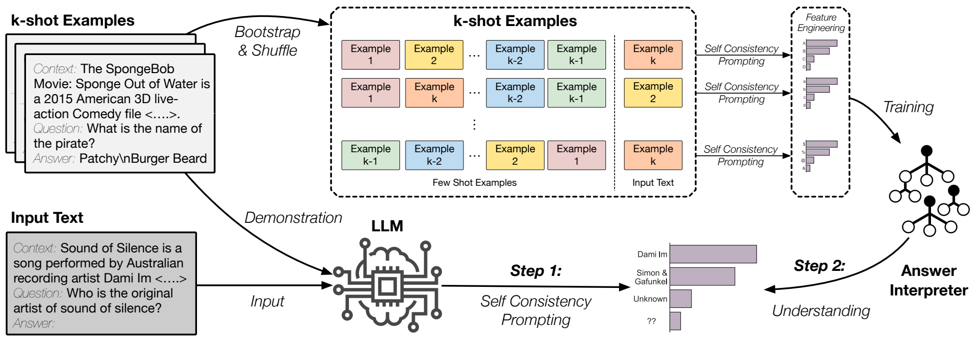

Figure 2: Overview of our framework. Given a question, we first retrieve a set of supporting pieces of evidence with a retrieval engine. Then, we perform two steps: (i) generate all feasible answers using self-consistency prompting; (ii) adopt an interpreter to infer the ambiguity in the question. The interpreter is trained with a *bootstrapping and shuffling* technique of 6 examples over distributional features from the generated answers.

when prompted to validate the correctness of its prior response. Other works prompted LLMs to generate their confidence (Mielke et al., 2022; Lin et al., 2022; Tian et al., 2023; Zhou et al., 2023b). Additionally, Si et al. (2023) considered the frequency of the answer as a proxy for confidence. Another line of work assumes that when the LLM generates a broad range of semantically varied answers, it indicates a high level of uncertainty (Lin et al., 2023; Nikitin et al., 2024; Shi et al., 2024). They measure the uncertainty via entropy over answers sampled from the model output distribution. After identifying semantically different answers a, the overall uncertainty can be represented as $H(q) = -\sum_a p(a|q) \log p(a|q)$. However, these approaches assume the existence of a single correct answer.

Answer Calibration. Understanding when to trust an LLM is essential for building safer AI systems (Amodei et al., 2016; Hendrycks et al., 2021; Zhao et al., 2020; Maynez et al., 2020; Portillo Wightman et al., 2023; Yang et al., 2024b). Selective question answering is a popular approach for addressing this problem (Chow, 1957; El-Yaniv et al., 2010; Kamath et al., 2020; Zhang et al., 2021). Specifically, the idea is to assign the confidence $s(q)$ for answering the question q. A threshold τ is used to decide whether to answer it, ask for clarification, or abstain from answering. An accurate uncertainty estimation may help reduce the risk of generating false or unfounded outputs.

3 Task Formulation

We focus on the scenario where we prompt an LLM to get an accurate answer to an unambiguous question or detect an ambiguous question under the few-shot setting. More specifically, given a user's question q, the QA system has access to some external information c relevant to the question. The contextual information is either provided with the question (e.g., a document-grounded conversation) or is retrieved from a collection of documents $\mathcal{D}$ (*i.e.,* retrieval-augmented QA). Given c and q, the goal of the QA system is to (1) find an accurate answer a to an unambiguous question; or (2) request clarification when the question is ambiguous (*i.e.,* has multiple plausible interpretations).[1]

In this paper, we focus on two tasks: *ambiguity detection* and *confidence calibration*. The goal of the ambiguity detection task is to identify whether a given question is ambiguous. As for confidence calibration, the goal is to measure the quality of the confidence estimation, which is crucial for avoiding inaccurate, incomplete, or misleading answers.

4 Our Approach

In this section, we describe our framework (see Figure 2), aiming to (i) identify ambiguous questions; and (ii) avoid providing incorrect, incomplete, or misleading answers. We first prompt the LLM to generate several answers (*answer-oriented prompting*; see §4.1) and then deduce the ambiguity by analyzing cues from the LLM outputs (see §4.2).

4.1 Self-consistency Prompting for Multiple Plausible Answers

Differently from previous work which prompted the LLM to generate a single answer using standard self-consistency prompting (Wang et al., 2022; Kuhn et al., 2023; Si et al., 2023), we prompt the

[1]In this paper we do not tackle the problem of generating a clarification question, which we leave for future research.

LLM to list all plausible answers, separated by a delimiter (*e.g.,* '\n'). Given a question q and a corresponding context c, this can be represented as:

$$P_{\mathsf{LLM}}(\mathcal{A} \mid \text{prompt}, c, q), \tag{1}$$

where $a \in \mathcal{A}$ represents a single answer. This process repeats m times by sampling from the LLM's decoder with a temperature value t (the number of answers $|\mathcal{A}|$ can be varied across different samples). Subsequently, we group all generated answers from the m sampling outputs using exact text matching, which is sufficient as the answers are typically short phrases, and categorize them to generate a collection $\mathcal{A}_m = \{(a_1, f_1), \ldots, (a_n, f_n)\}$. Here, each element $(a_i, f_i) \in \mathcal{A}_m$ represents an individual answer and its corresponding occurrence frequency across the m LLM outputs.

4.2 LLM Outputs Analysis

The next phase is to analyze the LLM outputs. The objectives of this step are twofold: (i) recognize when the LLM is confident about a single answer (indicating *non-ambiguity*); (ii) determine when the LLM is confident about multiple answers (indicating *ambiguity*). Intuitively, when an LLM repeatedly generates the same answer, it implies a high confidence level and likelihood of correctness (Wang et al., 2022). Conversely, a variety of low-frequency occurring answers may indicate either low confidence or potential inaccuracies (Kuhn et al., 2023). Therefore, we hypothesize that by examining the frequency of the LLM answers, we can infer the ambiguity in the question. In this work, we utilize an RF model to analyze the LLM outputs. The RF input is a set of features derived from the set $\mathcal{A}_m$ of answers and their frequencies. The model is used to predict a score reflecting the probability that the question is ambiguous.

Random Forest Few-shot Training. A notable challenge in training the interpreter model is the limited amount of data available. To overcome this, we adopt a *bootstrapping and shuffling* strategy using few-shot examples to create an expanded training dataset with N examples. Specifically, as shown in Figure 2, given k examples in the demonstration data, we *bootstrap* by selecting up to $k-1$ examples from these to form a new demonstration, while using the remaining example as the input and its corresponding label (*i.e.,* ambiguous or non-ambiguous) as the ground truth. Next, we *shuffle* the examples to generate additional demonstrations. This allows us to form a diverse collection of demonstration-input pairs that are fed into the LLM to produce a set of answer-frequency $\mathcal{A}_m$ for training. Then, we construct specific features to capture the distributional patterns of the answers.

Feature Extraction. We hypothesize that frequently occurring answers are more likely to be accurate, regardless of semantic meaning. This leads us to assume that discarding less common answers, which might be incorrect, can help in better assessing the question's ambiguity. However, different models of varying sizes exhibit distinct patterns in generating erroneous answers, making it challenging to set a fixed frequency threshold. To address this, we compute the entropy over answers with different frequencies. Specifically, we calculate the entropy of answers occurring more than m times, denoted as e_m. We find that using a binary value as the feature enhances model performance. Thus we define binary features $f_{e_m,t} \triangleq \mathbf{1}_{e_m>t}(e_m)$, where t represents a threshold within the range $[0, 1]$. We then generate a feature set by choosing various values for m and t. These features and the corresponding labels are used to train the random forest model, which serves as an interpreter to analyze $\mathcal{A}_m$. The advantage of the RF model with our bootstrapping and shuffling strategy against more sophisticated models is to simulate and learn different potential answer distributions, rather than relying on the semantic content.

4.3 Calibration for Question Answering

Another focus of this study is estimating the model certainty when answering questions. Our approach uses two types of confidence estimation. First, we assess the model confidence in determining whether a question is ambiguous using the probability estimation from an interpreter model, which we denote as $c_{\text{amb}} \propto P(\text{ambiguity}|x; \Theta)$, where x is the input to the interpreter model and Θ represents its parameters. Secondly, to estimate the model confidence in a specific answer a, we use a conditional probability formula $c_{\text{a}} \propto f_a \cdot P(\neg\text{ambiguity}|x; \Theta)$, where f_a is the frequency ratio of the answer a to the total frequency of all answers. Our hypothesis is that a high probability assigned by the model to either a single or multiple correct answers could signify a greater chance of accuracy. Conversely, a probability that reflects indecision or difficulty in distinguishing between these scenarios might indicate potential inaccuracies.

Models	ASQA				PACIFIC				ABG-COQA			
	P ↑	R ↑	F_1 ↑	Acc ↑	P ↑	R ↑	F_1 ↑	Acc ↑	P ↑	R ↑	F_1 ↑	Acc ↑
**Supervised Learning and Random Baselines*												
RANDOM	$57.64_{2.3}$	$51.66_{1.3}$	$54.48_{1.7}$	$50.76_{2.3}$	$55.34_{1.2}$	$54.1_{3.7}$	$54.68_{2.4}$	$53.47_{1.5}$	$42.95_{1.9}$	$37.36_{1.7}$	$50.52_{2.4}$	$50.54_{1.7}$
ROBERTA-L (Full)	$62.08_{6.3}$	$94.54_{4.2}$	$71.81_{1.4}$	$73.12_{3.6}$	$67.16_{2.1}$	$86.76_{1.5}$	$75.69_{1.8}$	$73.33_{3.6}$	$67.54_{2.1}$	$81.93_{6.8}$	$73.81_{1.4}$	$75.12_{1.2}$
ROBERTA-L (6-shot)	$50.86_{1.5}$	$61.42_{6.3}$	$55.57_{3.4}$	$58.01_{1.6}$	$63.81_{2.2}$	$33.74_{5.6}$	$43.75_{4.5}$	$45.87_{0.8}$	$46.70_{0.8}$	$71.11_{3.4}$	$56.36_{1.6}$	$47.20_{1.1}$
**Binary Prompting (Standard Few-shot Prompting for Ambiguity)*												
FLAN-T5-XL	$62.59_{1.8}$	$62.50_{7.6}$	$62.19_{3.0}$	$57.09_{0.6}$	$25.85_{1.3}$	$39.19_{3.6}$	$31.14_{2.1}$	$36.28_{0.4}$	$32.90_{0.1}$	$29.23_{0.0}$	$30.96_{0.1}$	$32.20_{0.2}$
FLAN-T5-XXL	$59.99_{0.1}$	$81.31_{0.9}$	$69.04_{0.4}$	$58.43_{0.3}$	$14.11_{3.8}$	$11.46_{5.6}$	$12.46_{5.0}$	$43.57_{3.3}$	$38.21_{0.8}$	$30.00_{1.5}$	$33.60_{1.3}$	$38.40_{0.4}$
LLAMA-2-7B	$56.67_{1.4}$	$90.81_{8.4}$	$69.70_{3.4}$	$55.32_{3.1}$	$36.22_{1.0}$	$94.79_{8.7}$	$52.37_{2.5}$	$36.76_{0.1}$	$51.44_{32.0}$	$6.15_{6.5}$	$10.50_{10.7}$	$50.10_{2.2}$
LLAMA-2-13B	$58.32_{0.4}$	$85.36_{1.9}$	$69.28_{0.3}$	$56.85_{0.3}$	$30.83_{1.0}$	$27.86_{8.6}$	$28.70_{5.3}$	$50.67_{2.9}$	$56.18_{7.4}$	$29.42_{8.8}$	$37.07_{7.4}$	$50.10_{1.4}$
LLAMA-2-70B	$52.59_{2.6}$	$54.62_{5.3}$	$53.85_{4.9}$	$50.80_{2.7}$	$38.64_{1.0}$	$44.27_{6.0}$	$41.26_{2.9}$	$53.55_{1.2}$	$48.56_{3.8}$	$32.69_{12.8}$	$37.86_{8.3}$	$47.00_{2.1}$
**Answer-Oriented Prompting with Random Forest (Ours)*												
FLAN-T5-XL	$57.00_{0.2}$	$94.58_{2.4}$	$71.13_{0.9}$	$56.31_{0.7}$	$45.70_{1.6}$	$77.87_{3.7}$	$57.93_{3.4}$	$57.42_{0.5}$	$60.96_{3.2}$	$62.16_{3.6}$	$61.44_{2.3}$	$59.45_{2.8}$
FLAN-T5-XXL	$61.14_{0.7}$	$77.37_{6.6}$	$68.29_{2.8}$	$59.48_{1.1}$	$47.81_{1.9}$	$72.90_{7.6}$	$57.71_{6.4}$	$59.94_{3.1}$	$67.73_{4.5}$	$60.31_{14.1}$	$62.47_{7.2}$	$63.71_{2.7}$
LLAMA-2-7B	$59.84_{1.7}$	$91.64_{8.8}$	$73.18_{3.1}$	$61.39_{2.2}$	$39.13_{3.1}$	$75.82_{6.5}$	$51.46_{2.1}$	$47.19_{0.7}$	$60.75_{1.3}$	$57.18_{6.1}$	$58.80_{3.8}$	$58.60_{2.2}$
LLAMA-2-13B	$61.42_{6.8}$	$88.67_{8.9}$	$72.30_{4.2}$	$61.25_{4.7}$	$42.19_{10.8}$	$71.00_{4.1}$	$53.89_{3.0}$	$54.78_{1.2}$	$60.29_{1.9}$	$58.33_{5.4}$	$59.21_{3.5}$	$58.40_{2.4}$
LLAMA-2-70B	$64.04_{3.4}$	$88.67_{4.1}$	$73.98_{2.2}$	$64.57_{2.2}$	$58.16_{2.7}$	$72.40_{8.2}$	$58.16_{4.9}$	$61.61_{4.0}$	$73.95_{3.2}$	$67.69_{3.6}$	$70.68_{2.8}$	$70.80_{2.3}$

Table 1: Ambiguity detection task results on the development set. We report the average performance with standard deviation across 3 random seeds. The best prompting performance for each column is highlighted in blue.

Methods	ASQA	PACIFIC	ABG-COQA
**Binary Prompting*			
Standard Prompting	$48.76_{4.7}$	$53.55_{1.2}$	$47.00_{2.1}$
CoT Prompting	$56.91_{3.4}$	$44.41_{3.0}$	$48.93_{1.5}$
Self-Consistency	$53.66_{6.0}$	$52.71_{2.1}$	$45.90_{3.6}$
**Answer-oriented Prompting*			
LLM-*itself*	$44.08_{1.1}$	$45.66_{2.1}$	$46.84_{2.2}$
ROBERTA-L	$54.27_{5.6}$	$61.55_{1.9}$	$55.56_{2.1}$
Frequency Heuristic	$57.42_{1.3}$	$54.75_{2.3}$	$59.70_{3.4}$
Heuristic Method	$61.57_{2.3}$	$59.86_{4.2}$	$62.00_{2.3}$
Sampling Repetition	$51.61_{1.7}$	$54.55_{5.0}$	$51.64_{1.5}$
Sampling Diversity	$50.85_{4.6}$	$50.84_{2.1}$	$47.03_{2.6}$
Random Forest (ours)	$64.57_{2.2}$	$61.61_{4.0}$	$70.80_{2.3}$

Table 2: Ambiguity detection accuracy on the dev set (3 seeds average) with different prompting using LLAMA-2-70B. The best performance is marked in blue.

5 Experimental Setup

5.1 Datasets

We experimented with three datasets, including ASQA (Stelmakh et al., 2022) , PACIFIC (Deng et al., 2022), and ABG-COQA (Guo et al., 2021). ASQA was created based on AmbigQA (Min et al., 2020) by adding a context to each question and long-form answers. PACIFIC is a QA dataset in the financial domain, constructed based on the TAT-QA dataset (Zhu et al., 2021) where the context is in the form of tables and text. ABG-COQA, which was built on top of the CoQA dataset (Reddy et al., 2019), consists of narratives and corresponding ambiguous questions. Following prior studies (Deng et al., 2023; Cole et al., 2023; Tian et al., 2023), we use the development sets for evaluation. See more details and examples in Appendix §B.

5.2 Implementation Details

We experimented with a range of LLMs with different sizes, including encoder-decoder, i.e., Flan-T5 (3B, 11B) (Chung et al., 2022) and decoder-only, i.e., LLAMA-2 (7B, 13B, 70B) (Touvron et al., 2023); for LLAMA-2, we used the CHAT variant. We set the number of few-shot examples to 6 in all models and prompting strategies due to the limited length of the model input. We used the oracle context as the input, except for our experiments with noisy contexts over the ASQA dataset. For those experiments, we utilized evidence retrieved by a Dense Passage Retrieval (DPR) model (Karpukhin et al., 2020); the retrieval corpus is the English Wikipedia dump of 12/20/2018 and the documents are split into chunks of 100 words (Karpukhin et al., 2020). Examples of the different prompts and further implementation details can be found in Appendix §C and §D, respectively.

5.3 Baselines

Ambiguity Detection. The first set of baselines is based on *Binary Prompting* (Deng et al., 2023) where the idea is to prompt the LLM to perform binary classification to determine question ambiguity. We evaluated different prompting strategies for binary prompting, including **Standard** prompting (Brown et al., 2020), **Chain-of-Thought (CoT)** prompting (Wei et al., 2022b), and **Self-Consistency** prompting (Wang et al., 2022). The second set of baselines is based on *Answer-oriented Prompting*, where we prompt the LLM to generate multiple answers for a question and then detect ambiguity based on the analysis of these answers. In our approach, we use a Random Forest model[2] to analyze the answers. To test the effectiveness of other models, we experimented with the following baselines. (i) **Heuristic Method**: a

[2]Details on the Random Forest training are provided in Appendix C.

question is predicted as ambiguous if the entropy of the generated answers exceeds a certain threshold. (ii) **Frequency Heuristic**: a question is predicted as ambiguous if there are multiple high-frequency answers. We experiment with various thresholds to define 'high frequency'. (iii) **LLM-*itself***: prompting the model for question ambiguity binary classification based on the concatenation of all generated answers, the original context, the question, and some few-shot demonstrations. (iv) **RoBERTa-L**: we train a RoBERTa-L model with the bootstrapping dataset generated in §4 and use it for prediction based on the same inputs as in LLM-*itself*. (v) **Sampling Repetition** and **Sampling Diversity** measure the frequency of the most confident answer and count the number of unique answers among samples from the LM respectively. Following Cole et al. (2023), we report the best performance among different values of *Num Disagreements* and *Num Answers*.

Confidence Calibration. We use the following approaches as baselines: **Self-consistency Confidence** (Si et al., 2023) uses the frequency of the most frequent answer from self-consistency prompting as the confidence score. **Sampling Diversity** estimates the confidence in inverse proportion to the number of distinct samples. Specifically, the score is zero if every sample differs from the others. We also use the **Verbalized Confidence** approach (Mielke et al., 2022; Tian et al., 2023) which concatenates the most frequent answer to the original context and question, and prompts the LLM to express its confidence in the range of 0 to 100. **`P(True)`** (Kadavath et al., 2022) concatenates the most frequent answer to the original context and question, and prompts the LLM to determine whether the answer is true. Then, the confidence score is computed based on the logit probability associated with the "`True`" token. The methods described above focus on assessing the confidence of a single answer. Therefore, for a more comprehensive evaluation, we also consider approaches that estimate the model confidence based on multiple answers. For **LLM-*itself***, we prompt the LLM with all generated answers, the original context, and the question. Then, unlike the ambiguity detection task, the LLM is prompted to express its confidence towards multi-correct answers in the range of 0 to 100. For **RoBERTa-L**, the approach is similar, but it uses the logits from the RoBERTa model to quantify confidence. Finally, the **Heuristic** method uses entropy as a measure of confidence.

Methods	P ↑	R ↑	F_1 ↑	Acc ↑
Llama-2-7B	$59.84_{1.7}$	$91.64_{8.8}$	$73.18_{3.1}$	$61.39_{2.2}$
`w/ Top-3`	$57.80_{0.3}$	$94.93_{0.1}$	$71.85_{0.3}$	$57.24_{0.6}$
Llama-2-13B	$61.42_{6.8}$	$88.67_{8.9}$	$72.30_{4.2}$	$61.25_{4.7}$
`w/ Top-3`	$57.99_{0.3}$	$96.58_{2.1}$	$72.45_{0.3}$	$57.80_{0.2}$
Llama-2-70B	$64.04_{3.4}$	$88.67_{4.1}$	$73.98_{2.2}$	$64.57_{2.2}$
`w/ Top-3`	$58.62_{1.2}$	$94.66_{1.4}$	$72.39_{0.7}$	$58.60_{1.1}$

Table 3: Results on the ambiguity detection task using retrieved passages on the ASQA dataset. `w/ Top-3` represents using the top-3 retrieved documents rather than the oracle context. We report the accuracy of the development set across three random seeds. The best performance for each column is highlighted in blue.

5.4 Evaluation Metrics

For the ambiguity detection task, we use Precision, Recall, F_1, and Accuracy for evaluation. For the confidence calibration task, we report the Accuracy of whether the model provides the correct answer to unambiguous user questions or accurately identifies the question ambiguity. For the confidence calibration task, we report the Expected Calibration Error (ECE) to measure the discrepancy between the predicted accuracy (*i.e.,* confidence) and its actual performance. Specifically, the predictions are divided into M uniform bins B_m w.r.t. confidence scores. Then, we compute the average absolute difference between the confidence (`cnf`) and the actual accuracy (`acc`) for each bin over n samples:

$$ECE = \sum_{m=1}^{M} \frac{|B_m|}{n} |\texttt{acc}(B_m) - \texttt{cnf}(B_m)| \qquad (2)$$

Due to the limitations of ECE stemming from its bucketing approach (Si et al., 2023), we also report the Brier score (Brier, 1950). We also evaluate how the system performs when it selectively responds based on its confidence. Acc@50 indicates the accuracy of questions if the QA system only answers questions with the top 50% confidence scores.

6 Experimental Results

6.1 Ambiguity Detection

Tables 1 and 2 present experimental results on the ambiguity detection task using various LLMs, and different prompting strategies with the ground truth context. Table 3 shows the more realistic scenario results when using the context retrieved with a DPR model. In particular, we use top-3 retrieved documents instead of the ground-truth documents.

Method	ASQA				PACIFIC				ABG-COQA			
	Acc ↑	Acc@50 ↑	ECE ↓	Brier ↓	Acc ↑	Acc@50 ↑	ECE ↓	Brier ↓	Acc ↑	Acc@50 ↑	ECE ↓	Brier ↓
**Single Answer Assumption*												
Verbalization	25.51	29.48	43.10	38.40	35.89	35.14	23.26	28.92	31.60	30.51	34.78	36.24
P(True)	25.51	44.64	28.58	28.61	35.89	40.15	14.00	24.26	31.60	50.85	25.48	28.29
Self-Consistency	25.51	62.37	28.09	24.02	35.89	49.62	9.15	21.18	31.60	52.00	25.23	26.33
Sampling Diversity	25.51	62.84	26.40	23.54	35.89	48.45	9.54	21.94	31.60	54.65	22.47	25.91
**Ambiguous Question Answering*												
LLM-*itself*	40.12	43.01	21.81	25.81	31.73	32.91	12.73	25.83	41.93	43.01	25.82	28.93
ROBERTA-L	46.84	49.02	20.30	24.72	35.31	49.02	9.20	20.83	42.41	51.03	22.31	25.05
Heuristic Method	52.35	53.61	26.07	33.55	33.21	47.69	10.37	21.50	44.80	55.20	25.25	27.44
Random Forest (ours)	61.26	65.82	10.15	23.90	37.39	53.08	8.67	19.49	49.60	59.20	16.83	24.84

Table 4: Calibration results on three datasets using LLAMA-2-70B on the development set. ↑ and ↓ indicate whether higher or lower metrics are preferable, respectively. The best performance for each column is highlighted in blue.

#1. Limited Effectiveness of Binary Prompting in Ambiguity Detection. As shown in Table 1, we find that the performance of binary prompting is inconsistent across different datasets. For example, the ASQA dataset obtains a performance slightly above random guessing, while the results on the PACIFIC and ABG-COQA datasets are underwhelming. Moreover, the increased model size does not necessarily improve the performance of this strategy. For example, the LLAMA-2-70B does not perform better than LLAMA-2-7B on the ASQA dataset. These findings indicate that binary prompting might struggle to detect ambiguity consistently. In Table 2 (Top), we further evaluate the performance of different binary prompting strategies (*i.e.,* CoT and Self-consistency). We find that these strategies did not yield any performance improvement. Our findings align with the prior study (Deng et al., 2023), underscoring the difficulty of this strategy to decide if a question is ambiguous. Similarly, Cole et al. (2023) suggests that none of the prompting strategies seems particularly useful, with none surpassing the baseline precision of 53%.

#2. Improved Performance in Ambiguity Detection with Answer-Oriented Prompting and Random Forest. Table 1 presents the performance of the ambiguity detection task using our approach, which achieves the best performance across datasets and model sizes. Notably, we observe a clear trend where the effectiveness in detecting ambiguity improves with the model size. This highlights that our approach can identify cases where the LLM confidently suggests multiple answers (indicating ambiguity) versus when it leans towards a single answer (*indicating non-ambiguity*).

Table 2 (Bottom) shows the results where we explore alternative models to the Random Forest using answer-oriented prompting. We find that Random Forest emerges as the most effective technique. Moreover, we observe that LLMs lack the ability to self-interpret their outputs. This observation aligns with findings from prior studies (Valmeekam et al., 2023; Stechly et al., 2023), indicating that self-interpretation of responses remains a challenging task for the LLMs. Apart from our approach, the heuristic method based on entropy delivers the most optimal results. Please, find a detailed error analysis of these two approaches in §6.3.

#3. Noisy Contexts Experiments. Table 3 evaluates a more realistic setting, where the context is retrieved with ASQA. This experiment shows what would be the performance when the retrieved passages are noisy. The performance slightly declines when using only the retrieved context (`w/ Top-3`) across all model sizes. Still, it is within 1-2 points in the F_1 score compared to the ground truth context setting, i.e., our approach is effective in coping with noisy contexts.

#4. Low-resource Setting. Table 1 compares our approach with supervised models in low-resource settings. In fact, our model outperforms supervised models trained on the same set of 6 examples (ROBERTA-L 6-shot): these models require much more training examples to be competitive.

6.2 Confidence Calibration

#1. Our approach responds to unambiguous questions or detects ambiguity. As shown in Table 4, our approach consistently outperforms all baselines, including models like the LLM or ROBERTA. It reaches 61.26% accuracy, outperforming the closest competitor (*i.e.,* heuristic) by roughly 9% on ASQA. Similar outcomes can be observed on other datasets. Interestingly, the accuracy for *Ambiguous Question Answering* does not always outperform those with *Single Answer*

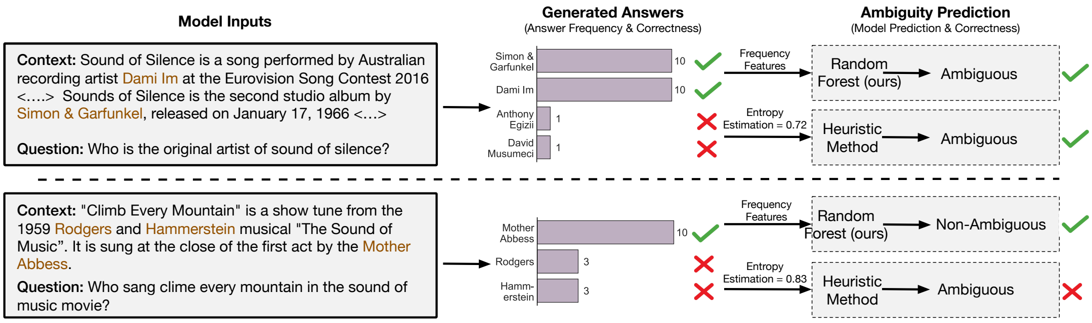

Figure 3: Our model against the entropy-based heuristic: the latter tends to have a higher entropy when the LLM produces incorrect answers. This leads to an overestimated denotational uncertainty, *i.e.,* higher false positives rate.

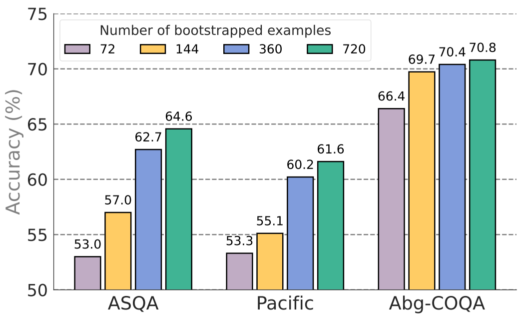

Figure 4: Impact of bootstrap size using LLAMA-2-70B. The performance increases with the bootstrap size.

Assumption on PACIFIC[3].

#2. Our approach demonstrates a superior ability to avoid incorrect, incomplete, or misleading answers. Our experiments indicate that using the Random Forest's probability, our approach generates more accurately calibrated confidence estimates. In various metrics like ECE, ACC@50, and Brier score, our method consistently outperforms other baseline methods across datasets. Our approach has thus an enhanced grasp of the trustworthiness of its answers, thereby minimizing the chances of providing incorrect information.

6.3 Further Analysis

Bootstrapping size. The main goal of the *bootstrapping and shuffling* strategy is to generate a diverse distribution of answers. Figure 4 shows the impact of the bootstrapping size on the performance. The accuracy improves with the size of the bootstrapping set: this result is impressive, given that only 6 annotated examples are initially used.

Error Analysis. Figure 3 provides case studies to compare the entropy-based heuristic and our approach on ASQA. When the LLM gives some incorrect answers, (*e.g.,* "rodgers" and "hammerstain"), the heuristic method tends to have higher entropy. In this case, the heuristic method misinterprets the source of this uncertainty to the question ambiguity, rather than its knowledge gaps or inaccuracies. This misinterpretation, often a result of the LLM's errors or 'hallucinations', leads to increased entropy values and, consequently, a higher rate of false positives. In our analysis, the heuristic method exhibits a 32.1% false positive rate and a 7.0% false negative rate. In contrast, our approach achieves a reduced false positive rate of 25.4% while obtaining a slight increase in false negatives at 10.1%.

7 Conclusion

In this work, we introduce a novel framework that enables LLMs to recognize ambiguous questions. Our approach prompts the LLM to generate multiple answers that are then analyzed through an interpreter model (i.e., Random Forest) to detect ambiguity. The Random Forest is trained with only 6 examples that are bootstrapped and shuffled to create multiple answer distributions. Our experiments on three datasets demonstrate the effectiveness of our approach in low-resource settings in identifying ambiguous questions. Furthermore, our approach has been shown to effectively refine the confidence calibration of LLM outputs: this improves the LLMs' ability to accurately interpret and respond to complex queries, contributing to more reliable and trustworthy QA systems.

[3] In PACIFIC the context documents are mainly tables with numbers; in this scenario, LLMs generally struggle, regardless of their size.

Limitations

Our research is a step forward in identifying ambiguous questions in LLM-based QA systems. However, we must recognize certain limitations, particularly regarding the dependency on model scale. The effectiveness of our method for detecting ambiguity is closely tied to the size of the LLM used. Essentially, our approach requires a robust LLM capable of accurately answering questions first, before assessing the ambiguity of these questions. If the model is smaller or prone to errors, our method may face challenges in accurately identifying ambiguities. This reliance on large-scale models brings advantages in terms of performance but also introduces scalability and resource challenges, especially in environments with limited resources. Moreover, our approach requires the LLM model to generate (possibly) all the answers to a question. This may be inefficient from a latency perspective, especially when using very large models. Finally, the current work doesn't specifically address the problem of disambiguation, which is crucial in improving trust in the NLP systems.

References

Mohammad Aliannejadi, Julia Kiseleva, Aleksandr Chuklin, Jeff Dalton, and Mikhail Burtsev. 2021. Building and evaluating open-domain dialogue corpora with clarifying questions. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4473–4484, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. 2019. Asking clarifying questions in open-domain information-seeking conversations. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR'19, page 475–484, New York, NY, USA. Association for Computing Machinery.

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*.

Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023*.

Glenn W Brier. 1950. Verification of forecasts expressed in terms of probability. *Monthly weather review*, 78(1):1–3.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, page 785–794, New York, NY, USA. Association for Computing Machinery.

Zhiyu Chen, Jason Choi, Besnik Fetahu, Oleg Rokhlenko, and Shervin Malmasi. 2023. Generate-then-retrieve: Intent-aware FAQ retrieval in product search. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, pages 763–771, Toronto, Canada. Association for Computational Linguistics.

Zhiyu Chen, Jason Ingyu Choi, Besnik Fetahu, and Shervin Malmasi. 2024. Identifying high consideration E-commerce search queries. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 563–572, Miami, Florida, US. Association for Computational Linguistics.

Chi-Keung Chow. 1957. An optimum character recognition system using decision functions. *IRE Transactions on Electronic Computers*, EC-6(4):247–254.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2022. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*.

Jeremy Cole, Michael Zhang, Daniel Gillick, Julian Eisenschlos, Bhuwan Dhingra, and Jacob Eisenstein. 2023. Selectively answering ambiguous questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 530–543, Singapore. Association for Computational Linguistics.

Yang Deng, Wenqiang Lei, Lizi Liao, and Tat-Seng Chua. 2023. Prompting and evaluating large language models for proactive dialogues: Clarification, target-guided, and non-collaboration. *arXiv preprint arXiv:2305.13626*.

Yang Deng, Wenqiang Lei, Wenxuan Zhang, Wai Lam, and Tat-Seng Chua. 2022. PACIFIC: Towards proactive conversational question answering over tabular and textual data in finance. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6970–6984, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Ran El-Yaniv et al. 2010. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11(5).

Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1050–1059, New York, New York, USA. PMLR.

Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. 2024. A survey of confidence estimation and calibration in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6577–6595.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.

Meiqi Guo, Mingda Zhang, Siva Reddy, and Malihe Alikhani. 2021. Abg-coQA: Clarifying ambiguity in conversational question answering. In *3rd Conference on Automated Knowledge Base Construction*.

Dan Hendrycks, Nicholas Carlini, John Schulman, and Jacob Steinhardt. 2021. Unsolved problems in ml safety. *arXiv preprint arXiv:2109.13916*.

Jiaming Ji, Boyuan Chen, Hantao Lou, Donghai Hong, Borong Zhang, Xuehai Pan, Tianyi Qiu, Juntao Dai, and Yaodong Yang. 2024. Aligner: Efficient alignment by learning to correct. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How Can We Know When Language Models Know? On the Calibration of Language Models for Question Answering. *Transactions of the Association for Computational Linguistics*, 9:962–977.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696, Online. Association for Computational Linguistics.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.

Kimiya Keyvan and Jimmy Xiangji Huang. 2022. How to approach ambiguous queries in conversational search: A survey of techniques, approaches, tools, and challenges. *ACM Comput. Surv.*, 55(6).

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations*.

Dongryeol Lee, Segwang Kim, Minwoo Lee, Hwanhee Lee, Joonsuk Park, Sang-Woo Lee, and Kyomin Jung. 2023. Asking clarification questions to handle ambiguity in open-domain QA. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 11526–11544, Singapore. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*.

Zhen Lin, Shubhendu Trivedi, and Jimeng Sun. 2023. Generating with confidence: Uncertainty quantification for black-box large language models. *arXiv preprint arXiv:2305.19187*.

Andrey Malinin and Mark Gales. 2020. Uncertainty estimation in autoregressive structured prediction. *arXiv preprint arXiv:2002.07650*.

Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919, Online. Association for Computational Linguistics.

Sabrina J. Mielke, Arthur Szlam, Emily Dinan, and Y-Lan Boureau. 2022. Reducing conversational agents' overconfidence through linguistic calibration. *Transactions of the Association for Computational Linguistics*, 10:857–872.

Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2020. AmbigQA: Answering ambiguous open-domain questions. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5783–5797, Online. Association for Computational Linguistics.

Kenton Murray and David Chiang. 2018. Correcting length bias in neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 212–223, Brussels, Belgium. Association for Computational Linguistics.

Alexandru Niculescu-Mizil and Rich Caruana. 2005. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, ICML '05, page 625–632, New York, NY, USA. Association for Computing Machinery.

Alexander Nikitin, Jannik Kossen, Yarin Gal, and Pekka Marttinen. 2024. Kernel language entropy: Fine-grained uncertainty quantification for llms from semantic similarities. *arXiv preprint arXiv:2405.20003*.

OpenAI. 2023. Gpt-4 technical report. *arXiv*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Gray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*.

Anaelia Ovalle, Kai-Wei Chang, Ninareh Mehrabi, Yada Pruksachatkun, Aram Galystan, Jwala Dhamala, Apurv Verma, Trista Cao, Anoop Kumar, and Rahul Gupta, editors. 2023. *Proceedings of the 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP 2023)*. Association for Computational Linguistics, Toronto, Canada.

Gwenyth Portillo Wightman, Alexandra Delucia, and Mark Dredze. 2023. Strength in numbers: Estimating confidence of large language models by prompt agreement. In *Proceedings of the 3rd Workshop on Trustworthy Natural Language Processing (TrustNLP 2023)*, pages 326–362, Toronto, Canada. Association for Computational Linguistics.

Siva Reddy, Danqi Chen, and Christopher D. Manning. 2019. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266.

Zhihong Shao and Minlie Huang. 2022. Answering open-domain multi-answer questions via a recall-then-verify framework. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1825–1838, Dublin, Ireland. Association for Computational Linguistics.

Zhengxiang Shi, Yue Feng, and Aldo Lipani. 2022a. Learning to execute actions or ask clarification questions. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 2060–2070, Seattle, United States. Association for Computational Linguistics.

Zhengxiang Shi, Jerome Ramos, To Eun Kim, Xi Wang, Hossein A Rahmani, and Aldo Lipani. 2022b. When and what to ask through world states and text instructions: Iglu nlp challenge solution. *Neural Information Processing Systems (NeurIPS) IGLU Workshop*.

Zhengyan Shi, Sander Land, Acyr Locatelli, Matthieu Geist, and Max Bartolo. 2024. Understanding likelihood over-optimisation in direct alignment algorithms. *arXiv preprint arXiv:2410.11677*.

Chenglei Si, Zhe Gan, Zhengyuan Yang, Shuohang Wang, Jianfeng Wang, Jordan Boyd-Graber, and Lijuan Wang. 2023. Prompting gpt-3 to be reliable. In *International Conference on Learning Representations (ICLR)*.

Kaya Stechly, Matthew Marquez, and Subbarao Kambhampati. 2023. Gpt-4 doesn't know it's wrong: An analysis of iterative prompting for reasoning problems. *arXiv*.

Ivan Stelmakh, Yi Luan, Bhuwan Dhingra, and Ming-Wei Chang. 2022. ASQA: Factoid questions meet long-form answers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8273–8288, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Weiwei Sun, Hengyi Cai, Hongshen Chen, Pengjie Ren, Zhumin Chen, Maarten de Rijke, and Zhaochun Ren. 2023. Answering ambiguous questions via iterative prompting. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7669–7683, Toronto, Canada. Association for Computational Linguistics.

Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D Manning. 2023. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. *arXiv preprint arXiv:2305.14975*.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Karthik Valmeekam, Matthew Marquez, and Subbarao Kambhampati. 2023. Can large language models really improve by self-critiquing their own plans? *arXiv preprint arXiv:2310.08118*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*.

Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. 2022a. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022b. Chain of thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*.

Bin Wu, Zhengyan Shi, Hossein A Rahmani, Varsha Ramineni, and Emine Yilmaz. 2024. Understanding the role of user profile in the personalization of large language models. *arXiv preprint arXiv:2406.17803*.

Adam X Yang, Maxime Robeyns, Thomas Coste, Jun Wang, Haitham Bou-Ammar, and Laurence Aitchison. 2024a. Bayesian reward models for llm alignment. *arXiv preprint arXiv:2402.13210*.

Adam X Yang, Maxime Robeyns, Xi Wang, and Laurence Aitchison. 2024b. Bayesian low-rank adaptation for large language models. In *The Twelfth International Conference on Learning Representations*.

Wen-tau Yih, Matthew Richardson, Chris Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 201–206, Berlin, Germany. Association for Computational Linguistics.

Mingtian Zhang, Shawn Lan, Peter Hayes, and David Barber. 2024. Mafin: Enhancing black-box embeddings with model augmented fine-tuning. *arXiv preprint arXiv:2402.12177*.

Shujian Zhang, Chengyue Gong, and Eunsol Choi. 2021. Knowing more about questions can help: Improving calibration in question answering. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 1958–1970, Online. Association for Computational Linguistics.

Zheng Zhao, Shay B. Cohen, and Bonnie Webber. 2020. Reducing quantity hallucinations in abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2237–2249, Online. Association for Computational Linguistics.

Han Zhou, Xingchen Wan, Lev Proleev, Diana Mincu, Jilin Chen, Katherine Heller, and Subhrajit Roy. 2023a. Batch calibration: Rethinking calibration for in-context learning and prompt engineering. *arXiv preprint arXiv:2309.17249*.

Kaitlyn Zhou, Dan Jurafsky, and Tatsunori Hashimoto. 2023b. Navigating the grey area: Expressions of overconfidence and uncertainty in language models. *arXiv preprint arXiv:2302.13439*.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3277–3287, Online. Association for Computational Linguistics.

A Retrieval Performance

The table 5 provides a comparison of retrieval performance metrics using Dense Passage Retrieval (DPR), focusing on its effectiveness in passage/document retrieval tasks. The performance is measured using the MRECALL metric at two different recall levels: 3 and 5. These recall levels indicate the number of retrieved items (passages) considered for evaluating the method's accuracy.

Method	MRECALL@3	MRECALL@5
DPR	43.46/33.70	48.66/38.08

Table 5: Performance on passage retrieval in MRECALL. The two numbers in each cell indicate performance on all questions and on questions with more than one answer, respectively.

B Datasets

Datasets used in this work. In this section, we provide details for each dataset, along with representative examples in Table 7. Following the previous work (Si et al., 2023; Tian et al., 2023), we downsample the evaluation set to assess model performance more effectively. Specifically, we sampled 638 examples from the ASQA dataset, 521 from the PACIFIC dataset, and 250 from the ABG-COQA dataset, all taken from their respective evaluation sets.

Discussion about the ClariQ dataset. Here we also discuss some potential inconsistent annotations in the ClariQ dataset. The ambiguity annotations within ClariQ can differ significantly based on the perspective of the annotators, resulting in multiple interpretations. For instance, while the query "*Find condos in Florida*" is ambiguous, "*Tell me about hotels in New York.*" is considered unambiguous. Here we provide 10 pairs of questions (20 questions in total) with inconsistent ambiguity annotations. It is noteworthy that ClariQ only consists of roughly 200 questions across both its training and development sets. Such inconsistent annotations highlight the importance of grounding the ambiguity of a question within the context. Datasets such as ASQA, PACIFIC, and ABG-COQA address this issue by grounding questions within their context.

User Query	Ambiguity
Find condos in Florida.	Yes
Tell me about hotels in New York.	No
I want to learn about rock art.	Yes
I'd like to learn about lymphoma in dogs	No
How to change the toilet in the house	Yes
how to build a fence?	No
Tell me more about USA tax for annuity	Yes
Find me information about the sales tax in Illinois.	No
How to cook pork tenderlion	Yes
How to get organised?	No
I'm looking for information on worm	Yes
I'm looking for information about South Africa	No
Tell me about vines for shade.	Yes
Tell me more on health clubs in Arkansas	No
Tell me about source of the nile	Yes
Tell me about american military university.	No
Tell me about Barbados.	Yes
Tell me more about dnr	No
Where should I order dog clean-up bags	Yes
Where can I buy pressure washers?	No

Table 6: Analysis of ClariQ dataset. We provide 10 pairs of questions with potentially inconsistent annotations.

C Implementation Details

We randomly select 6 examples from the training set for few-shot examples in demonstrations, because (1) even if the datasets we used in our experiments contain a large number of examples, our solution targets low-resource scenarios where just a bunch of annotated data are available; and (2) we wanted to be sure the examples can easily fit into the prompt of LLMs. Thus, we sample a very low number of examples (i.e., 6 examples) and demonstrate that these are sufficient to make our method work.

We follow (Kuhn et al., 2023; Cole et al., 2023; Si et al., 2023; Tian et al., 2023) to decode $m = 10$ times. For each, we generate `10` sampled outputs (temperature=`0.3,0.5,0.7`) and use exact match (after lowercasing and removing punctuation) for comparison among outputs. We do sub-string and exact matching to group the equivalent answers. While previous works use the NLI model, it does not work. We use XGboost (Chen and Guestrin, 2016) to train the Random Forest model. We performed a grid search for the hyper-parameters of the model by searching the best configuration on a development set with respect to the max depth among 1, 2, 3, 4, 5 and the number of estimators among 20, 30, 50, 100. For the feature engineering, in our experiments, we set m to 0,1,2 and t to 0.5,0.7,0.9.

To determine the confidence levels for both single and multiple answers using the **LLM-*itself***, **ROBERTA-L**, and **Heuristic** baselines, we first calculate the confidence for multiple answers, denoted as p_m. Once p_m is established, we then derive the confidence for a single answer using $p_s = 1 - p_m$. This approach assumes that the confidence in a single answer inversely correlates with the confidence in multiple answers. For the baseline **ROBERTA-L**, we concatenate the questions with the context and train them with a few labelled examples or all examples in the train sets.

D Examples of Prompting

Table 8 provides examples of prompts used in our work, including binary prompting, binary prompting with CoT, answer-oriented prompting, verbalized confidence, and self-evaluation of LLMs towards correctness. For self-consistency prompting, we repeat the above-mentioned prompt multiple times.

<table>
<tr><th>Dataset</th><th>Example</th></tr>
<tr><td>ASQA</td><td>id: 7089015503030534342
question: Who is the original artist of sound of silence?
answers: Simon & Garfunkel, Dami Im
contexts: "Sound of Silence" is a song performed by Australian recording artist Dami Im. Written by Anthony Egizii and David Musumeci of DNA Songs, it is best known as Australia's entry at the Eurovision Song Contest 2016 which was held in Stockholm, Sweden, where it finished 2nd, receiving a total of 511 points. The song also won the Marcel Bezençon Award in the composer category. The song was leaked on 10 March 2016, one day before its initial release date. It is Dami Im's fourth Australian top 20 hit and worldwide, it reached the top 40 in more than six countries after the Eurovision Song Contest 2016 Final.
Ambiguity: Yes</td></tr>
<tr><td>PACIFIC</td><td>id: e4fe0666-9c0e-43c0-9f67-538dae3092b9
question: What is the amount of total sales?
clarification question: Which year are you asking about?
answer to clarification question: 2019
contexts: "Sales by Contract Type: Substantially all of our contracts are fixed-price type contracts. Sales included in Other contract types represent cost plus and time and material type contracts. On a fixed-price type contract, we agree to perform the contractual statement of work for a predetermined sales price. On a cost-plus type contract, we are paid our allowable incurred costs plus a profit which can be fixed or variable depending on the contract's fee arrangement up to predetermined funding levels determined by the customer. On a time-and-material type contract, we are paid on the basis of direct labor hours expended at specified fixed-price hourly rates (that include wages, overhead, allowable general and administrative expenses and profit) and materials at cost. The table below presents total net sales disaggregated by contract type (in millions):
Table:
| Years Ended September 30 |
	2019	2018	2017
Fixed Price	$ 1,452.4	$ 1,146.2	$ 1,036.9
Other	44.1	56.7	70.8
Total sales	$1,496.5	$1,202.9	$1,107.7 | Ambiguity: Yes</td></tr>
<tr><td>ABG-COQA</td><td>id: 3ns0a6kxc48ribjdggweghvkamnzgl|15|2
question: What politics did Lloyd George have?
answers: Liberalism
contexts: "Wales is a country that is part of the United Kingdom and the island of Great Britain. It is bordered by England to the east, the Irish Sea to the north and west, and the Bristol Channel to the south. It had a population in 2011 of 3,063,456 and has a total area of . Wales has over of coastline and is largely mountainous, with its higher peaks in the north and central areas, including Snowdon, its highest summit. The country lies within the north temperate zone and has a changeable, maritime climate. Welsh national identity emerged among the Celtic Britons after the Roman withdrawal from Britain in the 5th century, and Wales is regarded as one of the modern Celtic nations. Llywelyn ap Gruffudd's death in 1282 marked the completion of Edward I of England's conquest of Wales, though Owain Glyndŵr briefly restored independence to Wales in the early 15th century. The whole of Wales was annexed by England and incorporated within the English legal system under the Laws in Wales Acts 1535–1542. Distinctive Welsh politics developed in the 19th century. Welsh Liberalism, exemplified in the early 20th century by Lloyd George, was displaced by the growth of socialism and the Labour Party. Welsh national feeling grew over the century; "Plaid Cymru" was formed in 1925 and the Welsh Language Society in 1962. Established under the Government of Wales Act 1998, the National Assembly for Wales holds responsibility for a range of.
Ambiguity: No</td></tr>
</table>

Table 7: Examples for ASQA, PACIFIC, and ABG-COQA datasets.

Method	Prompt Template
Binary Prompting	`Let's work this out in a step by step way to be sure we have the right answer. Please determine whether the question needs the further clarification, given the context. Note that only use information from the context to answer the question. Context: {CONTEXT}\nQuestion: {Question}.\nWhether a clarification question is needed:`
Binary Prompting (CoT)	`Let's work this out in a step by step way to be sure we have the right answer. Please determine whether the question needs the further clarification, given the context. Note that only use information from the context to answer the question. Context: {CONTEXT}\nQuestion: {Question}.\nGenerated Answers: {Answers}\nWhether a clarification question is needed:`
Answer-oriented Prompting	`Provide all the accurate responses to the question based on the given context. You must only use words that appear in the context to formulate your answer. Context: {CONTEXT}\nQuestion: {Question}.\nAll correct answers for the question are:`
Verbalized Confidence	`Let's work this out in a step by step. Please indicate your confidence level (from 0 to 100) regarding the accuracy of the provided answer, based on the given context. You must use numerical values only. Context: {CONTEXT}\nQuestion: {Question}.\nGenerated Answers: {Answers}\nAnswer: Answer.\nConfidence in accuracy:`
LLM Self-Eval	`Let's work this out in a step by step. Please determine whether the generated answer is correct or not. Context: {CONTEXT}\nQuestion: {Question}.\nGenerated Answers: {Answers}\nAnswer: Answer.\nWhether this answer is correct:`

Table 8: Prompt templates for each method evaluated. Each example will be concatenated with several demonstration examples, which contain ground-truth labels.